

Postman - Introduction

Postman adalah sebuah alat (tool) untuk menguji *Application Programming Interface* (API). API sendiri berfungsi sebagai penghubung antara dua atau lebih aplikasi, memungkinkan mereka saling berkomunikasi dan bertukar data.

Secara sederhana, API bisa diibaratkan sebagai pelayan restoran: Anda (aplikasi pengguna) menyebut pesanan (permintaan data), pelayan (API) menyampaikannya ke dapur (server), lalu mengantarkan hasilnya (respons) kembali kepada Anda.

Dengan kata lain, API adalah kumpulan aturan, fungsi, dan alat yang disediakan oleh suatu aplikasi kepada pengembang lain agar aplikasi tersebut dapat berinteraksi dengan sukses. Kita menggunakan API setiap kali mengakses aplikasi populer, seperti mengecek berita di ponsel, menggunakan Facebook, atau memesan transportasi online.

Sejarah Singkat Postman

Postman pertama kali dirancang pada tahun 2012 oleh **Abhinav Asthana**, seorang pengembang perangkat lunak dan wirausahawan. Tujuannya adalah untuk menyederhanakan proses pengembangan dan pengujian API. Sebelum ada Postman, pengujian API sering dilakukan secara manual atau menggunakan alat yang rumit. Postman hadir dengan antarmuka grafis (GUI) yang mudah digunakan, sehingga siapa pun—dari pengembang pemula hingga senior—dapat menguji API tanpa harus menulis kode dari awal.

Fitur Utama Postman

Postman memungkinkan kita untuk **mengirim dan mengamati permintaan (*request*) serta tanggapan (*response*) HTTP**. HTTP adalah protokol standar yang digunakan untuk komunikasi di web. Postman mendukung berbagai metode HTTP seperti:

- **GET** (mengambil data)
- **POST** (mengirim data baru)
- **PUT** (memperbarui data yang ada)
- **PATCH** (memperbarui sebagian data)
- **DELETE** (menghapus data)

Selain itu, Postman dapat berjalan di berbagai sistem operasi: **Linux, Windows, dan Mac**. Hasil dari permintaan yang kita buat juga bisa langsung diterjemahkan ke dalam kode berbagai bahasa pemrograman (misalnya Python, JavaScript, Java, dll.), yang sangat membantu pengembang untuk mengintegrasikan API ke dalam proyek mereka.

Mengapa Postman Diperlukan? (Elaborasi Kebutuhan)

Postman memiliki basis pengguna yang sangat besar dan menjadi alat yang populer karena alasan-alasan berikut, yang akan saya jelaskan lebih lanjut:

1. **Gratis tanpa biaya lisensi**

Postman menyediakan versi gratis dengan fitur yang sudah sangat lengkap. Cocok untuk tim kecil hingga besar. Tim yang membutuhkan fitur kolaborasi tingkat lanjut dapat berlangganan versi berbayar, tetapi untuk keperluan belajar dan pengujian dasar, versi gratis sudah lebih dari cukup.

2. **Mudah digunakan**

Cukup unduh dari situs resmi Postman, instal, lalu jalankan. Tidak perlu konfigurasi rumit. Antarmukanya intuitif, mirip dengan aplikasi web pada umumnya.

3. **Akses berbasis akun**

Setelah instalasi, Anda bisa masuk ke akun pribadi. Ini memungkinkan sinkronisasi koleksi API, lingkungan (*environment*), dan pengaturan lainnya antar perangkat (misalnya dari laptop kantor ke laptop pribadi).

4. **Mudah mengelola kumpulan pengujian (*test suites*)**

Postman memiliki fitur **Collections** (koleksi). Satu koleksi dapat berisi banyak permintaan API (misalnya: GET daftar pengguna, POST pengguna baru, DELETE pengguna). Koleksi juga bisa memiliki sub-folder untuk mengelompokkan permintaan berdasarkan modul atau fitur aplikasi. Ini sangat membantu dalam menjaga kerapian dan keterulangan pengujian.

5. **Mendukung berbagai jenis API**

Postman tidak hanya untuk API REST, tetapi juga bisa menangani **SOAP** (protokol lama yang masih digunakan di perusahaan-perusahaan besar) dan **GraphQL** (alternatif modern untuk REST). Fleksibilitas ini menjadikannya alat serba guna.

6. **Pengujian otomatis dengan checkpoint**

Kita bisa menambahkan skrip pengujian kecil pada setiap permintaan. Contohnya: periksa apakah kode respons HTTP adalah 200 (OK), atau pastikan data JSON yang dikembalikan mengandung properti tertentu. Postman akan menjalankan skrip ini setiap kali permintaan dikirim.

7. **Integrasi dengan CI/CD**

Dalam pengembangan perangkat lunak modern, kita sering menggunakan alur **CI/CD** (Integrasi Berkelanjutan / Pengiriman atau Penerapan Berkelanjutan). Postman dapat diintegrasikan dengan alat seperti Jenkins, GitLab CI, atau GitHub Actions sehingga setiap kali ada perubahan kode, pengujian API bisa berjalan otomatis.

8. Integrasi dengan Newman atau Collection Runner

Newman adalah alat baris perintah (command-line) dari Postman. Dengan Newman, kita bisa menjalankan koleksi Postman berkali-kali tanpa membuka antarmuka grafis. Ini sangat berguna untuk pengujian berulang (misalnya pengujian beban atau regresi) tanpa harus mengklik manual setiap saat.

9. Dukungan komunitas yang besar

Postman memiliki forum, dokumentasi resmi, dan ribuan tutorial online. Jika menemui masalah, besar kemungkinan sudah ada solusinya di Stack Overflow atau YouTube.

10. Konsol debugging

Postman menyediakan konsol (mirip konsol di browser) yang mencatat setiap permintaan dan respons secara detail. Jika ada kesalahan, kita bisa melihat *header* HTTP, isi pesan, dan waktu eksekusi untuk melacak bug.

11. Dukungan multi-lingkungan (*Environment*)

Kita bisa membuat beberapa lingkungan, misalnya: Pengembangan (development), Pengujian (staging), dan Produksi (production). Masing-masing lingkungan memiliki variabel yang berbeda (seperti URL dasar, kunci API, dll.). Satu koleksi yang sama bisa digunakan di semua lingkungan hanya dengan mengganti lingkungan aktif.

12. Impor/Eksport data

Kita dapat mengekspor koleksi dan lingkungan ke dalam file (biasanya format JSON) dan mengimpornya kembali ke Postman lain. Ini memudahkan berbagi pengujian antar anggota tim atau antar komputer.

Cara Kerja dengan Postman (Antarmuka Utama)

Untuk mulai menggunakan Postman, kita akan menemukan empat bagian utama dalam jendela aplikasinya:

1. **Header** (bagian atas)

Berisi menu utama, tombol untuk menyimpan, mengimpor, menjalankan koleksi, serta pengaturan akun dan lingkungan.

2. **Sidebar** (bagian kiri)

Menampilkan daftar koleksi, permintaan yang tersimpan, sejarah (*history*) permintaan yang pernah dikirim, serta lingkungan yang tersedia. Di sinilah kita memilih atau mengatur kelompok pengujian.

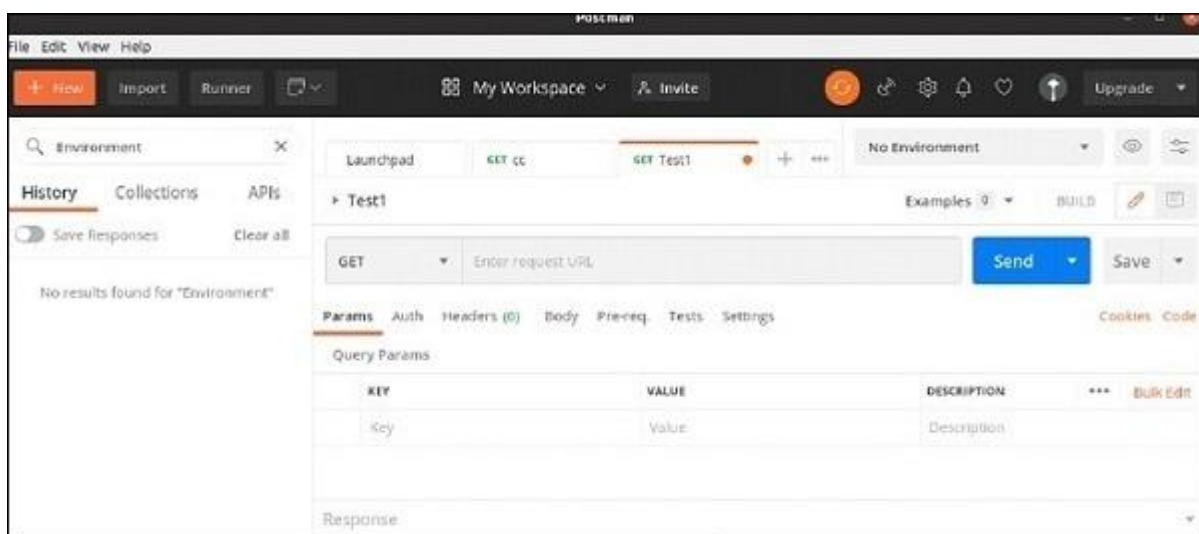
3. **Builder** (bagian tengah)

Area kerja utama untuk menyusun permintaan API. Di sini kita memilih metode HTTP (GET, POST, dll.), memasukkan URL, menambahkan *header* (misalnya Authorization), serta isi *body* (untuk metode POST/PUT). Builder juga menyediakan tab untuk menulis skrip pengujian (prasyarat Pre-request Script dan pengujian Tests).

4. **Response** (bagian bawah atau kanan, tergantung tata letak)

Menampilkan hasil dari permintaan yang dikirim. Kita bisa melihat kode status (misalnya 200 OK, 404 Not Found), waktu respons, ukuran data, dan isi respons (biasanya dalam format JSON, XML, HTML, atau teks biasa). Response juga bisa diformat agar mudah dibaca.

Jika Anda membuka Postman, Anda akan melihat antarmuka dengan sidebar di kiri, builder di tengah, dan response di sebelah kanan (atau bawah pada layar sempit).



Header

Postman terdiri dari menu-menu seperti New, Import, Runner (digunakan untuk menjalankan pengujian dengan Collection Runner), Open New, Interceptor, Sync, dan lain-lain. Header juga menampilkan nama ruang kerja (*workspace*), yaitu *My Workspace*, beserta opsi *Invite* untuk membagikannya antar tim. Bagian **Header** pada Postman adalah area paling atas dari jendela aplikasi. Header ini berfungsi sebagai pusat kendali cepat untuk berbagai tindakan penting. Mari kita bedah satu per satu komponen yang disebutkan:

1. New (Baru)

Tombol ini digunakan untuk membuat berbagai entitas baru, seperti:

- *Request* (permintaan API baru)
- *Collection* (kumpulan permintaan)
- *Environment* (lingkungan dengan variabel-variabel)
- *Mock Server* (server tiruan untuk simulasi API)
- *Monitor* (pemantauan API secara berkala)
- *Documentation* (dokumentasi API)

2. Import (Impor)

Fitur ini memungkinkan Anda membawa data dari luar ke dalam Postman, misalnya:

- File koleksi atau lingkungan dalam format JSON
- File spesifikasi API seperti OpenAPI (Swagger), RAML, atau GraphQL
- URL langsung ke file mentah
- Menempelkan teks mentah (raw text) dari clipboard

3. Runner (Pelari)

Runner adalah pintasan untuk menjalankan **Collection Runner**. Collection Runner adalah fitur untuk mengeksekusi seluruh permintaan dalam suatu koleksi secara berurutan atau berulang. Berguna untuk:

- Pengujian regresi
- Pengujian beban sederhana
- Otomatisasi skenario end-to-end

Klik Runner → pilih koleksi → atur jumlah iterasi → jalankan. Hasilnya akan ditampilkan dengan rincian permintaan mana yang berhasil atau gagal.

4. Open New (Buka Baru)

Membuka jendela atau tab baru di Postman. Pilihannya antara lain:

- *Postman Window* (jendela Postman baru)
- *Runner Window* (jendela Runner terpisah)
- *Privacy Mode Window* (jendela mode privasi, tidak menyimpan riwayat)

Berguna saat Anda ingin mengerjakan dua proyek API sekaligus tanpa mencampurkan riwayat.

5. Interceptor (Penyadap)

Interceptor adalah ekstensi browser (umumnya untuk Chrome) yang menangkap lalu lintas HTTP dari browser dan mengirimkannya ke Postman. Fungsinya:

- Menangkap *request* dan *response* yang terjadi saat Anda berselancar di web
- Memungkinkan Postman bekerja dengan *cookies* yang disimpan di browser
- Berguna untuk menganalisis API yang dipanggil oleh situs web tanpa dokumentasi resmi

6. Sync (Sinkronisasi)

Postman menyediakan fitur sinkronisasi berbasis akun. Setelah login, semua koleksi, lingkungan, dan pengaturan akan otomatis tersimpan di cloud Postman. Tombol Sync mengindikasikan status sinkronisasi (biasanya berupa ikon lingkaran atau centang). Jika ada perubahan lokal, sinkronisasi akan mengunggahnya ke server, sehingga pekerjaan Anda tetap konsisten di perangkat lain (misalnya laptop kantor dan laptop pribadi).

7. Menu Lainnya (dan lain-lain)

Header juga berisi ikon-ikon seperti:

- **Search** (pencarian cepat di dalam koleksi)
- **Notification** (notifikasi dari tim atau pembaruan Postman)
- **Account / Settings** (pengaturan akun, preferensi, dan logout)
- **Trash** (tempat sampah untuk mengembalikan entitas yang dihapus sementara)

8. Nama Workspace: My Workspace & Tombol Invite

- **Workspace** adalah ruang kerja kolaboratif. Setiap pengguna Postman memiliki *My Workspace* sebagai ruang kerja pribadi default. Di dalam workspace, Anda dapat menyimpan koleksi, lingkungan, dan server tiruan.

- **Tombol Invite** (undang) muncul di sebelah nama workspace. Dengan mengkliknya, Anda dapat mengirim undangan ke anggota tim lain (via email) untuk bergabung ke workspace tersebut. Ini memungkinkan kolaborasi: tim dapat berbagi koleksi API, dokumentasi, dan hasil pengujian secara real-time.



Menu ini digunakan untuk membuat **tiga jenis entitas utama** dalam Postman: Environment (Lingkungan), Collection (Koleksi), Request (Permintaan), Selain ketiga di atas, menu New juga menyediakan opsi lain seperti:

- *Mock Server* (server tiruan untuk simulasi respons)
- *Monitor* (pemantauan API terjadwal)
- *Documentation* (dokumentasi otomatis dari koleksi)
- *API* (mengelola definisi API secara terstruktur)

1. Menu New (Baru)

Menu ini digunakan untuk membuat **tiga jenis entitas utama** dalam Postman:

- **Environment (Lingkungan)**

Lingkungan adalah kumpulan variabel (misalnya `base_url`, `api_key`, `username`) yang nilainya bisa berbeda-beda tergantung kebutuhan (development, staging, production). Dengan membuat Environment baru, Anda dapat dengan cepat mengganti konfigurasi tanpa harus mengedit setiap permintaan satu per satu.

- **Collection (Koleksi)**

Koleksi adalah wadah yang berisi satu atau beberapa permintaan API (*request*). Anda bisa mengelompokkan permintaan berdasarkan fitur, modul, atau proyek tertentu. Koleksi juga bisa memiliki sub-folder untuk pengelompokan yang lebih rapi. Selain itu, koleksi dapat dilengkapi dengan skrip pengujian (*tests*) dan prasyarat (*pre-request scripts*).

- **Request (Permintaan)**

Permintaan API adalah inti dari pekerjaan Postman. Anda menentukan metode HTTP (GET, POST, dll.), URL, header, parameter, dan body. Setiap permintaan bisa disimpan ke dalam koleksi tertentu untuk digunakan kembali nanti.

Selain ketiga di atas, menu New juga menyediakan opsi lain seperti:

- *Mock Server* (server tiruan untuk simulasi respons)
- *Monitor* (pemantauan API terjadwal)

- *Documentation* (dokumentasi otomatis dari koleksi)
- *API* (mengelola definisi API secara terstruktur)

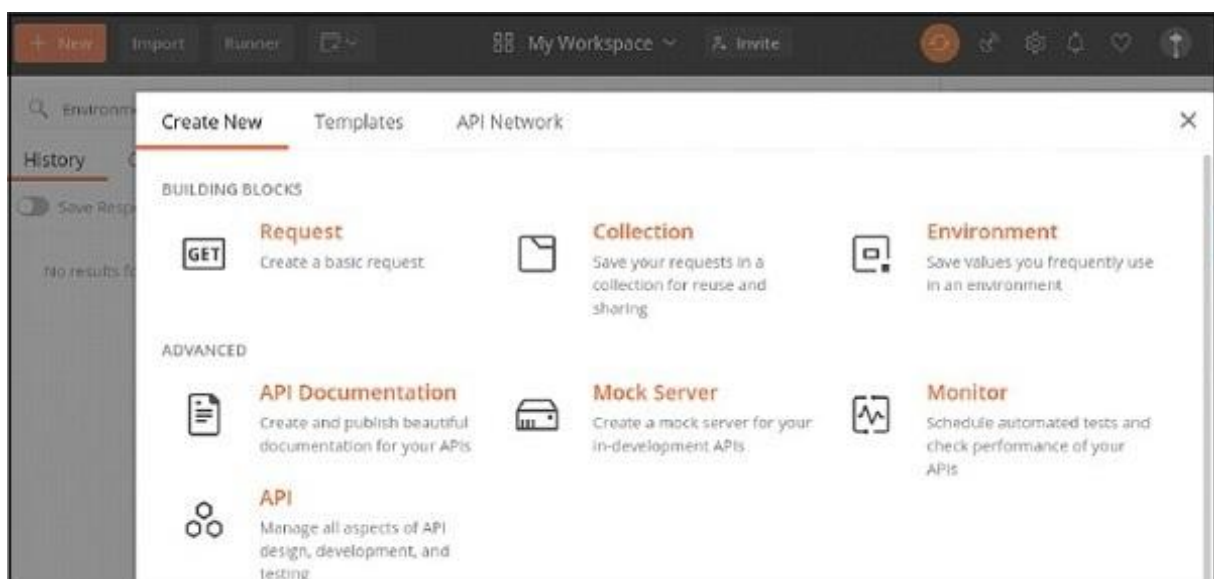
2. Menu Import (Impor)

Menu ini membantu Anda **mengimpor Environment dan Collection** dari sumber eksternal ke dalam Postman. Format yang didukung antara lain:

- **File JSON** – Hasil ekspor dari Postman lain (misalnya file .json berisi koleksi atau lingkungan).
- **File spesifikasi API** – Seperti OpenAPI (Swagger), RAML, GraphQL, atau WSDL (untuk SOAP).
- **URL publik** – Anda bisa memberikan tautan langsung ke file mentah yang dapat diakses publik.
- **Raw text** – Menempelkan teks JSON langsung ke dalam kotak dialog impor.
- **Postman API** – Mengimpor melalui koneksi ke akun Postman lain dengan izin tertentu.

Kegunaan praktis dari Import:

- Berbagi pekerjaan dengan tim: rekan kerja mengekspor koleksi → Anda mengimpornya.
- Migrasi dari alat API lain (seperti Insomnia, Paw, atau Swagger UI).
- Memulihkan cadangan (*backup*) jika terjadi kesalahan.
- Menggunakan koleksi publik dari **Postman Templates** atau **Postman API Network**.



1. Menu New (Baru)

Menu ini digunakan untuk membuat **tiga jenis entitas utama** dalam Postman:

- **Environment (Lingkungan)**

Lingkungan adalah kumpulan variabel (misalnya `base_url`, `api_key`, `username`) yang nilainya bisa berbeda-beda tergantung kebutuhan (development, staging, production). Dengan membuat Environment baru, Anda dapat dengan cepat mengganti konfigurasi tanpa harus mengedit setiap permintaan satu per satu.

- **Collection (Koleksi)**

Koleksi adalah wadah yang berisi satu atau beberapa permintaan API (*request*). Anda bisa mengelompokkan permintaan berdasarkan fitur, modul, atau proyek tertentu. Koleksi juga bisa memiliki sub-folder untuk pengelompokan yang lebih rapi. Selain itu, koleksi dapat dilengkapi dengan skrip pengujian (*tests*) dan prasyarat (*pre-request scripts*).

- **Request (Permintaan)**

Permintaan API adalah inti dari pekerjaan Postman. Anda menentukan metode HTTP (GET, POST, dll.), URL, header, parameter, dan body. Setiap permintaan bisa disimpan ke dalam koleksi tertentu untuk digunakan kembali nanti.

Selain ketiga di atas, menu New juga menyediakan opsi lain seperti:

- *Mock Server* (server tiruan untuk simulasi respons)
- *Monitor* (pemantauan API terjadwal)
- *Documentation* (dokumentasi otomatis dari koleksi)
- *API* (mengelola definisi API secara terstruktur)

2. Menu Import (Impor)

Menu ini membantu Anda **mengimpor Environment dan Collection** dari sumber eksternal ke dalam Postman. Format yang didukung antara lain:

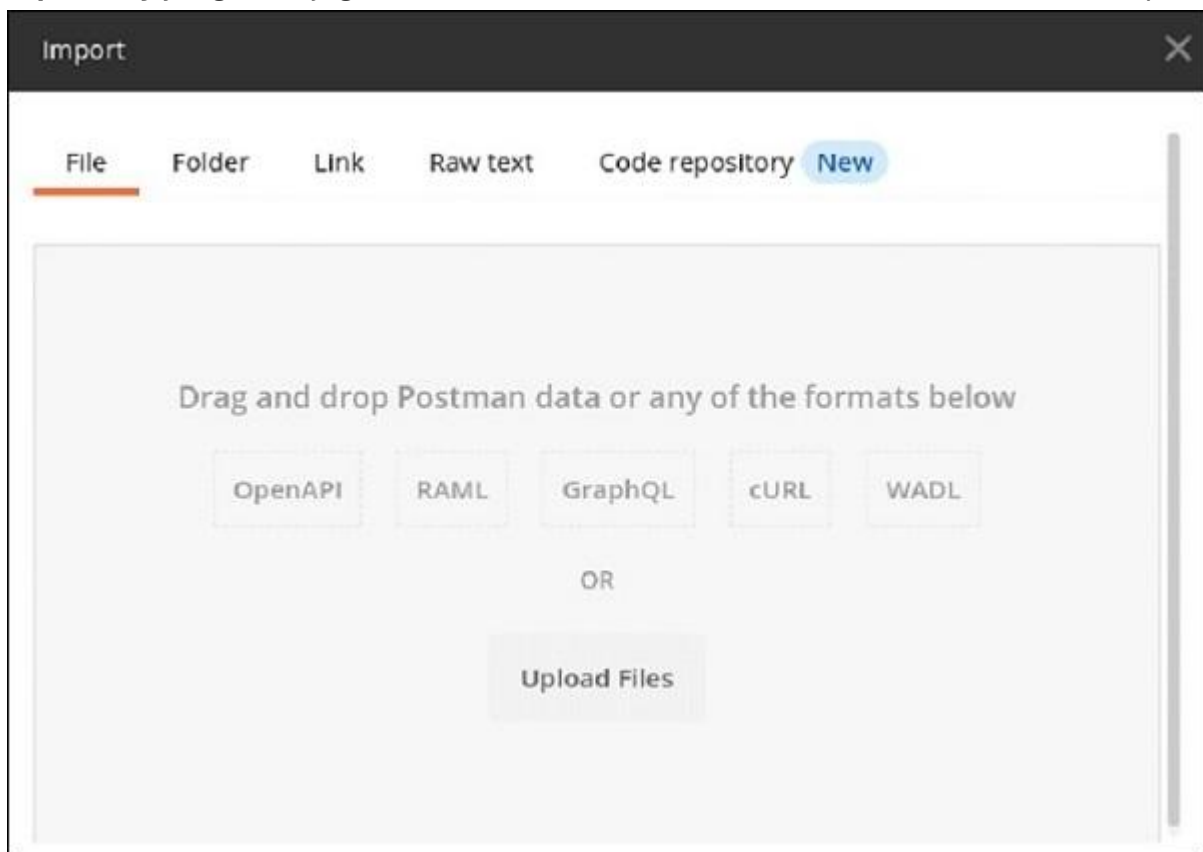
- **File JSON** – Hasil ekspor dari Postman lain (misalnya file `.json` berisi koleksi atau lingkungan).
- **File spesifikasi API** – Seperti OpenAPI (Swagger), RAML, GraphQL, atau WSDL (untuk SOAP).
- **URL publik** – Anda bisa memberikan tautan langsung ke file mentah yang dapat diakses publik.
- **Raw text** – Menempelkan teks JSON langsung ke dalam kotak dialog impor.

- **Postman API** – Mengimpor melalui koneksi ke akun Postman lain dengan izin tertentu.

Kegunaan praktis dari Import:

- Berbagi pekerjaan dengan tim: rekan kerja mengekspor koleksi → Anda mengimpornya.
- Migrasi dari alat API lain (seperti Insomnia, Paw, atau Swagger UI).
- Memulihkan cadangan (*backup*) jika terjadi kesalahan.
- Menggunakan koleksi publik dari **Postman Templates** atau **Postman API Network**.

Kita dapat mengimpor dari opsi **File**, **Folder**, **Link**, **Raw text**, atau dari **Code repository** yang juga tersedia di bawah menu Import.



Menu **Import** di Postman tidak hanya terbatas pada file JSON biasa. Postman menyediakan **berbagai sumber** untuk mengimpor data, sehingga sangat fleksibel bagi pengembang yang bekerja dengan berbagai format dan alur kerja. Berikut penjelasan masing-masing opsi:

1. File (Berkas)

- **Deskripsi:** Mengimpor satu atau beberapa berkas dari komputer lokal.

- **Format yang didukung:**

- Postman Collection (.json)
- Postman Environment (.json)
- Postman Data Dump (.json)
- Spesifikasi API: OpenAPI (Swagger) .json / .yaml, RAML, GraphQL, WADL, WSDL (untuk SOAP)
- File teks biasa (.txt) berisi raw request

- **Cara:** Klik tombol Upload Files lalu pilih berkas dari direktori.

2. Folder (Direktori)

- **Deskripsi:** Mengimpor semua berkas yang valid dari suatu folder sekaligus.
- **Kegunaan:** Cocok jika Anda memiliki banyak koleksi atau environment yang tersimpan dalam satu folder proyek. Postman akan memindai folder dan mengimpor semua berkas yang dikenali.
- **Cara:** Pilih tab Folder → klik Select Folder → pilih folder yang diinginkan.

3. Link (Tautan URL)

- **Deskripsi:** Mengimpor langsung dari URL publik yang mengarah ke file spesifikasi API atau koleksi.
- **Contoh:** Anda memiliki file OpenAPI yang dihosting di <https://contoh.com/swagger.json>. Cukup tempelkan URL tersebut, Postman akan mengambil dan mengimpornya.
- **Cara:** Pilih tab Link → masukkan URL → klik Import.

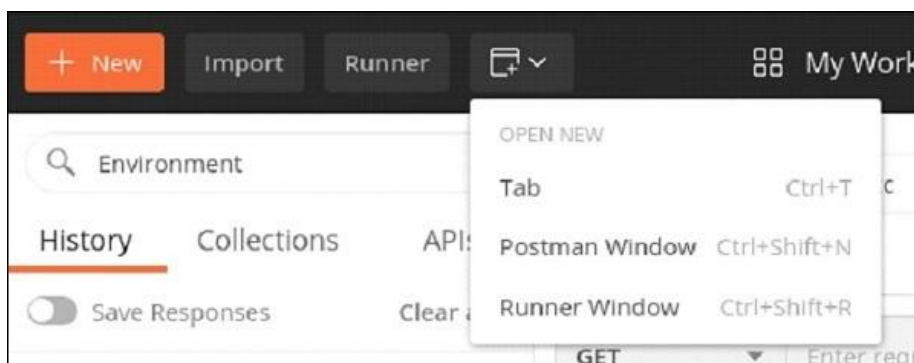
4. Raw text (Teks Mentah)

- **Deskripsi:** Menempelkan langsung kode JSON atau YAML dari clipboard ke dalam kotak teks.
- **Kegunaan:** Sangat praktis ketika seseorang mengirimkan definisi API melalui chat, email, atau dokumen. Anda cukup salin teksnya, buka Postman, tempel, lalu impor.
- **Cara:** Pilih tab Raw text → tempel teks → klik Import.

5. Code repository (Repositori Kode)

- **Deskripsi:** Mengimpor koleksi atau environment langsung dari layanan manajemen kode sumber seperti **GitHub**, **GitLab**, **Bitbucket**, atau dari **Azure DevOps**.
- **Kegunaan:** Memungkinkan integrasi alur kerja di mana koleksi API disimpan bersama kode aplikasi. Anda tidak perlu mengunduh file secara manual; Postman dapat mengambilnya langsung dari repositori (perlu otentikasi jika repositori bersifat privat).
- **Cara:**
 - Pilih tab Code Repository
 - Pilih penyedia (GitHub, GitLab, dll.)
 - Hubungkan akun dan pilih repositori serta file yang diinginkan.
 - (Catatan: Fitur ini mungkin memerlukan pengaturan token akses pribadi di beberapa versi Postman).

Di sini, **Open New** digunakan untuk membuka tab baru, jendela Postman baru, atau jendela Runner.



Menu **Open New** yang berada di Header Postman memberikan tiga opsi utama untuk membuka ruang kerja baru tanpa menutup tampilan yang sedang aktif. Ini sangat berguna untuk multitasking saat menangani banyak proyek atau pengujian API secara paralel.

Tiga Opsi dalam Open New:

1. **New Tab** (Tab Baru)
 - **Fungsi:** Membuka tab baru di dalam jendela Postman yang sama.
 - **Kegunaan:** Anda bisa mengerjakan permintaan API yang berbeda di tab terpisah tanpa mencampur riwayat atau konteks. Misalnya, satu tab untuk menguji API autentikasi, tab lain untuk API pengguna.

- **Cara:** Klik Open New → pilih New Tab (atau tekan Ctrl+T / Cmd+T).

2. Postman Window (Jendela Postman Baru)

- **Fungsi:** Membuka jendela Postman baru yang sepenuhnya terpisah dari jendela yang sedang aktif.
- **Kegunaan:** Cocok saat Anda ingin membandingkan dua koleksi secara berdampingan, atau bekerja dengan akun/workspace yang berbeda (misalnya jendela kiri untuk workspace pribadi, jendela kanan untuk workspace tim). Setiap jendela berjalan independen.
- **Cara:** Klik Open New → pilih Postman Window.

3. Runner Window (Jendela Runner)

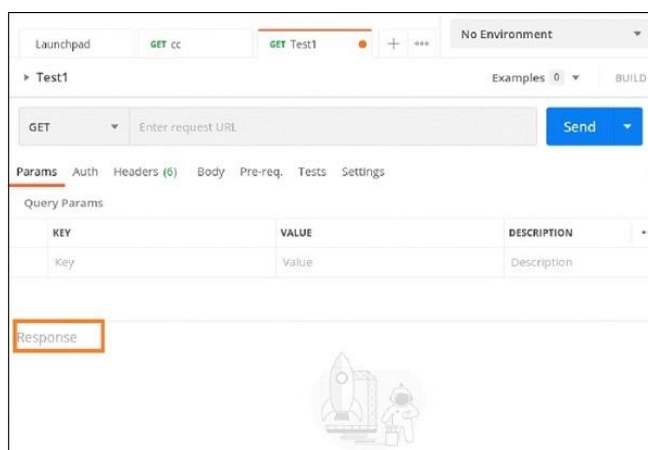
- **Fungsi:** Membuka jendela khusus untuk **Collection Runner** (menjalankan serangkaian pengujian koleksi).
- **Kegunaan:** Collection Runner biasanya dapat diakses melalui tombol Runner di Header, tetapi dengan Open New → Runner Window Anda bisa membukanya di jendela terpisah. Ini memungkinkan Anda menjalankan tes sambil tetap mengedit permintaan API di jendela utama tanpa terganggu.
- **Cara:** Klik Open New → pilih Runner Window.

Opsi Tambahan (pada beberapa versi Postman)

- **Privacy Mode Window** (Jendela Mode Privasi) – Membuka jendela yang tidak menyimpan riwayat permintaan, cocok untuk pengujian sensitif yang tidak ingin terekam.

Response

Bagian Response akan terisi nilainya hanya ketika sebuah permintaan (request) dibuat. Bagian tersebut umumnya berisi detail dari respons.



Prinsip Dasar: Response Hanya Terisi Setelah Request

Bagian **Response** di Postman bersifat **pasif** — artinya, tidak akan menampilkan data apapun sebelum Anda mengirimkan permintaan (*request*). Ini sesuai dengan sifat dasar komunikasi HTTP: klien (dalam hal ini Postman) mengirim permintaan ke server, lalu server membalas dengan respons. Tanpa permintaan, tidak ada respons.

Jadi, ketika Anda pertama kali membuka Postman atau memilih sebuah permintaan yang belum pernah dijalankan, area Response akan kosong (biasanya menampilkan pesan seperti *"Send a request to see the response"*).

Apa Saja yang Termasuk dalam "Response Details"?

Setelah Anda menekan tombol **Send**, Postman akan menampilkan informasi lengkap dari respons server. Secara umum, bagian Response mencakup komponen-komponen berikut:

1. Status Code (Kode Status)

Angka tiga digit yang menunjukkan hasil permintaan, misalnya:

- 200 OK – berhasil
- 201 Created – berhasil membuat data baru
- 400 Bad Request – permintaan salah
- 401 Unauthorized – perlu otentikasi
- 404 Not Found – sumber daya tidak ditemukan
- 500 Internal Server Error – kesalahan di sisi server

2. Response Body (Isi Respons)

Data utama yang dikirim kembali oleh server. Bisa dalam berbagai format:

- **JSON** (paling umum untuk REST API)
- **XML**
- **HTML** (biasanya untuk halaman web)
- **Text** biasa
- **Binary** (misalnya gambar, file PDF) – akan ditampilkan sebagai pratinjau atau kode hex.

3. Response Headers (Header Respons)

Metadata yang dikirim server bersama respons, misalnya:

- Content-Type: application/json
- Content-Length: 1234
- Set-Cookie: sessionId=abc123
- Cache-Control: no-cache

4. **Response Time** (Waktu Respons)

Durasi dari pengiriman request hingga diterimanya respons, biasanya dalam milidetik (ms). Berguna untuk mengukur performa API.

5. **Response Size** (Ukuran Respons)

Besar data respons (dalam byte atau KB), membantu memantau efisiensi payload.

Visualisasi di Postman

Setelah Anda mengirim request, Postman menampilkan Response dalam beberapa tab:

Tab	Fungsi
Body	Menampilkan isi respons (dengan opsi format seperti Pretty, Raw, Preview)
Headers	Menampilkan semua header respons (bisa dicari/filter)
Test Results	Menampilkan hasil skrip pengujian yang Anda tulis di tab <i>Tests</i> pada request
Cookies	Menampilkan cookie yang dikirim oleh server (jika ada)

Contoh Skenario

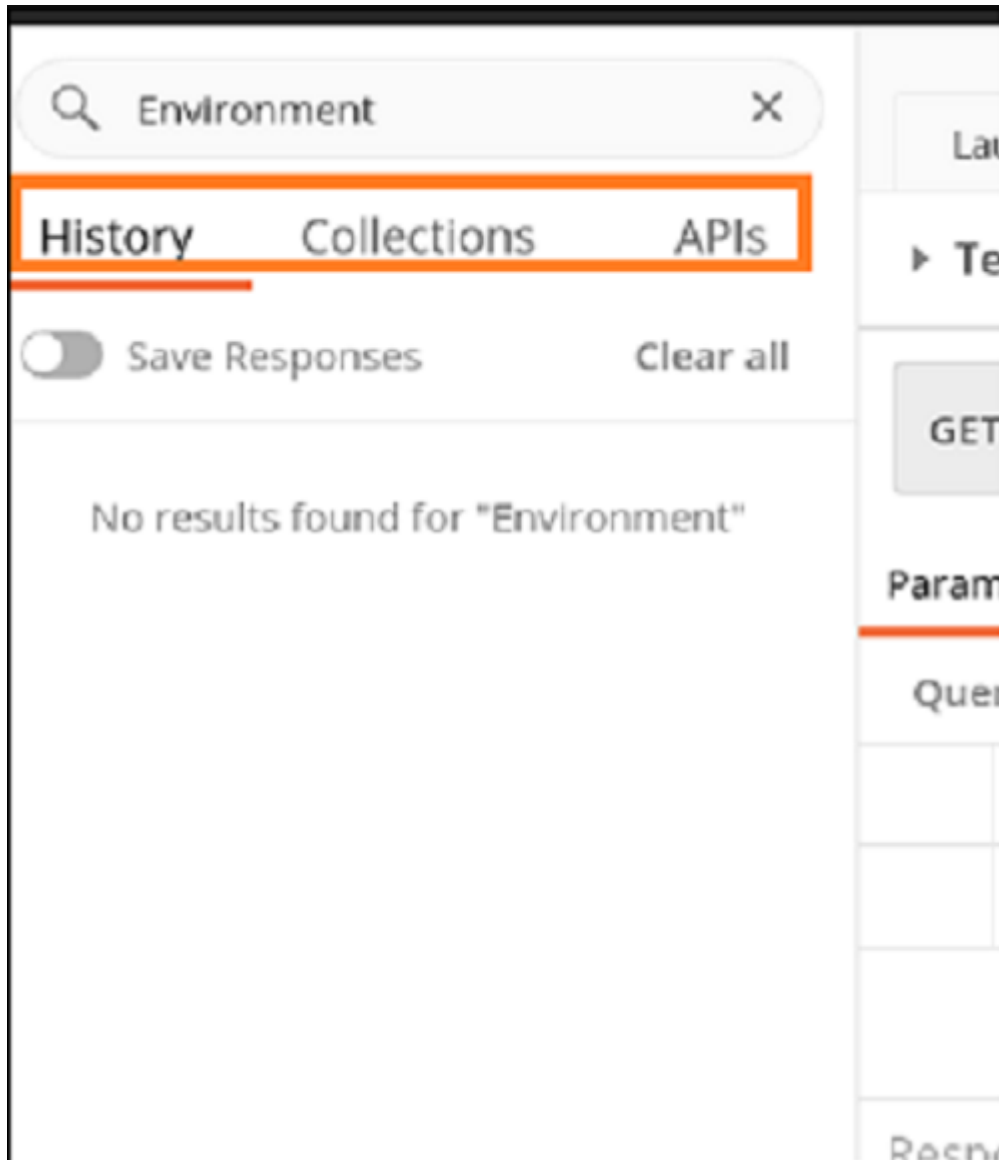
Anda membuat request GET ke <https://api.example.com/users/1>. Sebelum mengirim, area Response kosong. Setelah klik **Send**, Postman menampilkan:

- **Status:** 200 OK
- **Body:** {"id":1, "name":"John"}
- **Headers:** Content-Type: application/json, X-Powered-By: Express
- **Time:** 145 ms
- **Size:** 324 B

Jika request gagal (misalnya karena server down), Response akan tetap terisi dengan kode status error (misalnya 500), tetapi body mungkin berisi pesan kesalahan dari server.

Sidebar

Sidebar terdiri dari **Collections** (digunakan untuk mengelola pengujian, berisi folder, sub-folder, dan permintaan), **History** (mencatat semua permintaan API yang pernah dibuat di masa lalu), dan **APIs**.



Sidebar adalah panel di sisi kiri jendela Postman. Fungsinya sebagai **navigasi utama** untuk mengakses semua koleksi, riwayat permintaan, dan definisi API. Sidebar membantu Anda tetap terorganisir, terutama ketika menangani banyak proyek API sekaligus.

Berikut penjelasan mendetail dari tiga komponen utama yang disebutkan:

1. Collections (Koleksi)

Collections adalah **wadah hierarkis** untuk mengelompokkan permintaan API yang terkait. Di dalam Collections, Anda dapat:

- **Membuat folder dan sub-folder** – Misalnya, folder Pengguna berisi request GET /users, POST /users, DELETE /users/{id}. Sub-folder bisa digunakan untuk fitur yang lebih spesifik, misalnya Pengguna/Admin dan Pengguna/Reguler.
- **Menyimpan request** – Setiap request (GET, POST, PUT, dll.) disimpan dengan nama, URL, header, body, dan skrip pengujian.
- **Menjalankan pengujian koleksi** – Dengan Collection Runner, Anda bisa mengeksekusi semua request dalam koleksi secara berurutan.
- **Mengelola variabel koleksi** – Variabel yang berlaku hanya untuk koleksi tersebut (misalnya base_url, auth_token).
- **Mendokumentasikan API** – Postman dapat menghasilkan dokumentasi otomatis dari koleksi.

Kegunaan praktis: Collections sangat penting untuk **test suite** (kumpulan pengujian). Tim QA biasanya membuat koleksi berisi skenario positif, negatif, dan edge case untuk API yang sedang dikembangkan.

2. History (Riwayat)

History adalah **catatan otomatis** dari setiap permintaan API yang pernah Anda kirim melalui Postman, tanpa perlu menyimpannya secara eksplisit ke koleksi.

- **Merekam semua request** – Termasuk metode HTTP, URL, parameter, header, dan body yang dikirim.
- **Berdasarkan waktu** – Biasanya diurutkan dari yang terbaru ke terlama.
- **Dapat di-save ke koleksi** – Jika suatu request di History dirasa penting, Anda bisa menyimpannya ke Collection tertentu dengan klik ikon simpan.
- **Dapat dihapus** – Bisa menghapus satu entri atau seluruh riwayat (misalnya untuk alasan privasi).
- **Kelompok berdasarkan sesi** – Postman biasanya menampilkan riwayat per hari atau per waktu.

Kegunaan praktis: History sangat membantu ketika Anda lupa menyimpan request, atau sedang melakukan eksplorasi API secara cepat dan ingin melihat kembali permintaan yang berhasil. Juga berguna untuk *debugging* ulang tanpa harus menulis ulang request dari nol.

3. APIs (API)

Bagian **APIs** di Sidebar (tersedia di versi Postman modern) adalah fitur yang lebih **terstruktur dan berorientasi pada siklus hidup API** (API lifecycle). Berbeda

dengan Collections yang fokus pada pengujian dan permintaan, bagian APIs memungkinkan Anda untuk:

- **Mendefinisikan spesifikasi API** – Menggunakan format OpenAPI (Swagger), RAML, atau GraphQL.
- **Mengelola beberapa versi API** – Misalnya v1, v2, v3 dari API yang sama.
- **Membuat Mock Server** – Simulasi respons berdasarkan spesifikasi.
- **Menghubungkan dengan Collections** – Anda bisa menghasilkan Collection dari definisi API, atau sebaliknya.
- **Dokumentasi terpusat** – Menyediakan halaman dokumentasi yang lebih rapi untuk seluruh tim.
- **Memantau perubahan** – Melacak perubahan spesifikasi seiring waktu.

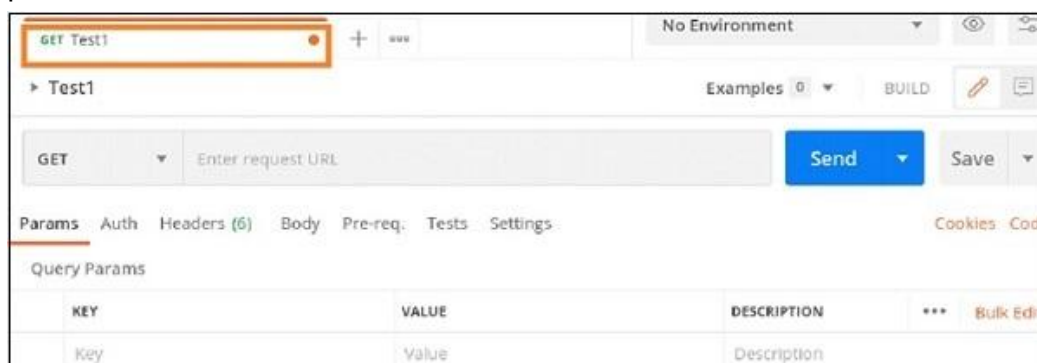
Perbedaan utama antara Collections dan APIs:

Collections	APIs
Berisi request-request konkret (dengan URL, header, dll.)	Berisi definisi abstrak (skema, model data, endpoint)
Fokus pada pengujian dan eksekusi	Fokus pada desain, dokumentasi, dan versioning
Lebih fleksibel untuk eksplorasi	Lebih formal untuk kolaborasi tim

Catatan: Pada beberapa versi Postman yang lebih lama, bagian "APIs" mungkin tidak muncul atau digantikan oleh "Templates". Namun pada Postman versi terbaru (v10 ke atas), fitur APIs menjadi bagian penting dari platform.

Builder

Builder adalah bagian terpenting dari aplikasi Postman. Bagian ini memiliki tab permintaan (request tab) dan menampilkan nama permintaan saat ini. Secara bawaan, tulisan 'Untitled Request' akan muncul jika tidak ada judul yang diberikan pada sebuah permintaan.



Mengapa Builder Disebut Bagian Terpenting?

Builder adalah area kerja utama di Postman. Di sinilah semua aktivitas inti pembuatan, pengeditan, dan pengiriman permintaan API berlangsung. Tanpa Builder, Anda tidak bisa menyusun request sama sekali. Oleh karena itu, Builder bisa dianggap sebagai "jantung" dari aplikasi Postman.

Komponen Utama dalam Builder

Builder terdiri dari beberapa elemen penting:

1. Request Tab (Tab Permintaan)

- Setiap permintaan yang Anda buka akan muncul dalam sebuah **tab** terpisah di bagian atas Builder (mirip seperti tab di browser web).
- Anda bisa membuka banyak tab sekaligus, masing-masing berisi request yang berbeda.
- Tab memudahkan peralihan cepat antara berbagai request tanpa kehilangan konteks.
- Tab biasanya menampilkan nama request dan metode HTTP (misalnya GET, POST) sebagai ikon atau label.

2. Nama Permintaan Saat Ini

- Di sebelah kiri dalam Builder (biasanya di atas area URL), terdapat **kolom atau label yang menampilkan nama request** yang sedang aktif.
- Nama ini penting untuk mengidentifikasi request ketika disimpan ke dalam Collection.
- Jika Anda bekerja dengan banyak request, memberi nama yang deskriptif (misalnya Get Semua Pengguna, Login Admin) akan sangat membantu navigasi.

3. "Untitled Request" sebagai Nama Bawaan

- Ketika Anda membuat request baru (melalui New → Request atau tombol + di samping tab), Postman secara otomatis memberi nama **"Untitled Request"**.
- Ini adalah nama sementara sampai Anda menyimpan request tersebut ke dalam Collection dan memberinya judul yang bermakna.
- Jika Anda langsung mengirim request tanpa menyimpan, nama tetap "Untitled Request" hingga Anda menyimpannya.

Elemen Lain dalam Builder (Selain yang Disebut)

Untuk melengkapi pemahaman, berikut elemen penting lain yang ada di area Builder:

Elemen	Fungsi
Dropdown Metode HTTP	Memilih metode: GET, POST, PUT, PATCH, DELETE, dll.
URL Field	Tempat memasukkan endpoint API (misalnya <code>https://api.example.com/users</code>)
Params	Menambahkan parameter query (misalnya <code>?page=1&limit=10</code>)
Authorization	Mengatur jenis autentikasi (Bearer Token, Basic Auth, API Key, dll.)
Headers	Menambahkan header HTTP kustom (misalnya Content-Type, Accept)
Body	Mengisi data untuk request POST/PUT (raw JSON, form-data, x-www-form-urlencoded, dll.)
Pre-request Script	Menulis kode JavaScript yang dijalankan SEBELUM request dikirim (misalnya untuk menyiapkan variabel)
Tests	Menulis kode JavaScript yang dijalankan SETELAH respons diterima (untuk pengujian otomatis)
Send Button	Tombol utama untuk mengirim request ke server

Bagian **Builder** juga berisi tipe request (GET, POST, PUT, dan sebagainya) serta URL. Sebuah request dijalankan dengan tombol **Send**. Jika ada perubahan yang dilakukan pada sebuah request, kita dapat menyimpannya dengan tombol **Save**.



Bagian **Builder** memiliki tab-tab seperti **Params**, **Authorization**, **Headers**, **Body**, **Pre-req.**, **Tests**, dan **Settings**. Parameter dari sebuah permintaan dalam bentuk pasangan kunci-nilai (key-value) diletakkan di dalam tab **Params**. **Authorization** untuk sebuah API dengan nama pengguna, kata sandi, token, dan sebagainya berada di dalam tab **Authorization**.

Header permintaan dan *body* permintaan didefinisikan masing-masing di dalam tab **Headers** dan **Body**. Terkadang, ada *pre-condition scripts* (skrip prasyarat) yang

harus dijalankan sebelum sebuah permintaan. Skrip tersebut diletakkan di dalam tab **Pre-req..**

Tab **Tests** berisi skrip yang dijalankan ketika sebuah permintaan dipicu. Ini membantu untuk memvalidasi apakah API bekerja dengan benar serta apakah data dan kode respons yang diperoleh sudah tepat.

1. Tipe Request (Request Type)

Di bagian Builder, tepat di sebelah kiri kolom URL, terdapat **dropdown** yang berisi berbagai metode HTTP. Metode ini menentukan jenis operasi yang ingin Anda lakukan terhadap sumber daya di server. Metode yang paling umum antara lain:

- **GET** → mengambil data (membaca)
- **POST** → mengirim data baru (menciptakan)
- **PUT** → memperbarui seluruh data yang ada (mengganti)
- **PATCH** → memperbarui sebagian data
- **DELETE** → menghapus data

Memilih tipe request yang tepat sangat penting karena setiap metode memiliki semantik yang berbeda. Misalnya, GET seharusnya tidak mengubah data di server (idempotent), sementara POST biasanya digunakan untuk tindakan yang tidak idempoten (seperti membuat pesanan baru). Postman juga mendukung metode lain seperti HEAD, OPTIONS, dan TRACE sesuai kebutuhan.

2. URL (Uniform Resource Locator)

Kolom **URL** (atau *endpoint*) adalah tempat Anda memasukkan alamat server API yang ingin dihubungi. Contoh:

```
https://api.example.com/v1/users/123
```

- URL dapat berisi **parameter path** (misalnya /users/123), **parameter query** (misalnya ?page=2&limit=10), dan **fragmen** (jarang digunakan di API).
- Postman menyediakan fitur **auto-complete** dari riwayat dan koleksi.
- Anda juga dapat menggunakan **variabel** di dalam URL, seperti {{base_url}}/users, yang nilainya diambil dari Environment atau Collection.

3. Tombol Send (Kirim)

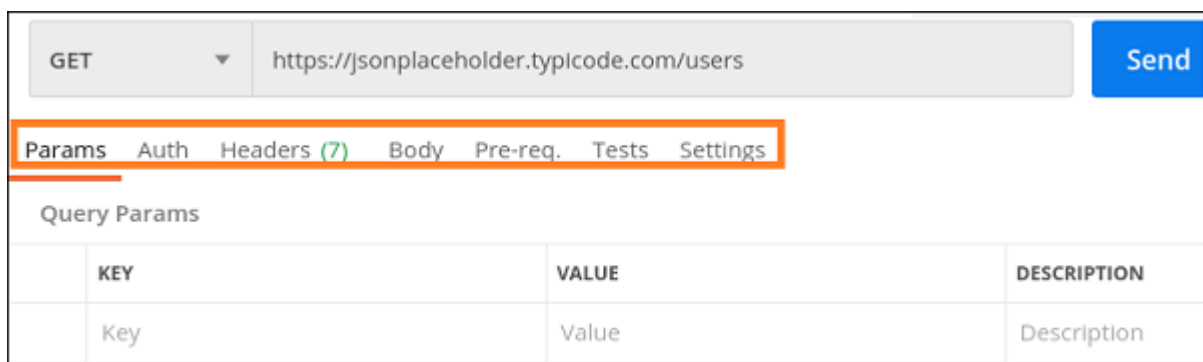
Tombol **Send** berwarna biru (secara default) terletak di sebelah kanan kolom URL. Fungsinya adalah **mengirim request** yang telah Anda susun ke server.

- Saat Anda mengklik Send, Postman akan membangun pesan HTTP berdasarkan metode, URL, header, body, dan pengaturan lainnya, lalu mengirimkannya ke server.
- Setelah server merespons, Postman akan menampilkan hasilnya di bagian **Response** (status code, body, header, waktu respons, dll.).
- Selama proses pengiriman, Anda akan melihat indikator loading. Jika koneksi bermasalah atau server tidak merespons, Postman akan menampilkan pesan error (misalnya Could not get any response).

4. Tombol Save (Simpan)

Tombol **Save** (biasanya bergambar floppy disk atau bertuliskan "Save") digunakan untuk **menyimpan perubahan** pada request yang sedang aktif. Ini berlaku jika Anda telah melakukan modifikasi pada request yang sudah ada (misalnya mengubah URL, menambah header, mengedit body, atau mengubah skrip pengujian).

- Jika request tersebut **belum pernah disimpan** (masih bernama "Untitled Request" dan belum menjadi bagian dari Collection), mengklik Save akan memunculkan dialog untuk memilih **Collection** tujuan dan memberi nama request.
- Jika request **sudah disimpan** sebelumnya, klik Save akan langsung memperbarui entri yang ada di Collection tanpa memunculkan dialog.
- Pintasan keyboard: Ctrl + S (Windows/Linux) atau Cmd + S (Mac) untuk menyimpan.
- Penting untuk **sering menyimpan** agar pekerjaan Anda tidak hilang, terutama sebelum menutup tab atau keluar dari Postman.



1. Tab Params (Parameter)

- **Fungsi:** Menambahkan **parameter query** ke dalam URL.
- **Bentuk:** Pasangan *key-value* (kunci-nilai). Contoh: page = 2, limit = 10 akan menghasilkan ?page=2&limit=10 di akhir URL.

- Parameter query biasanya digunakan untuk **filtering**, **pagination**, atau **sorting**.
- Postman secara otomatis akan menyusun parameter ini ke dalam URL. Anda juga bisa langsung menuliskan parameter di kolom URL, dan Postman akan memindahkannya ke tab Params secara otomatis.
- Ada dua mode: **Query Params** (untuk parameter di URL) dan **Path Variables** (untuk parameter di dalam jalur URL, misalnya /users/:id). Path Variables biasanya dikelola melalui kolom URL dengan sintaks :namaVariabel.

2. Tab Authorization (Otentikasi)

- **Fungsi:** Mengatur metode otentikasi yang diperlukan oleh API.
- **Jenis metode yang umum:**
 - **No Auth:** Tidak ada otentikasi.
 - **Bearer Token:** Token (biasanya JWT) dimasukkan ke header Authorization: Bearer <token>.
 - **Basic Auth:** Mengirim username dan password (dienkode base64).
 - **API Key:** Mengirim kunci API melalui header, parameter query, atau cookie.
 - **OAuth 1.0 / OAuth 2.0:** Otentikasi yang lebih kompleks, banyak digunakan oleh API modern (Google, Facebook, dll.).
 - Memilih metode yang tepat sangat penting agar server dapat mengidentifikasi siapa yang mengakses API.
 - Postman menyediakan antarmuka yang mudah untuk mengisi credential (username, password, token) tanpa perlu menyusun header secara manual.
 - Untuk OAuth 2.0, Postman bahkan bisa membantu mendapatkan *access token* secara otomatis.

3. Tab Headers (Header)

- **Fungsi:** Menentukan **header HTTP** yang akan dikirim bersama permintaan.
- **Contoh header:**
 - Content-Type: application/json → memberi tahu server bahwa body berformat JSON.

- Accept: application/json → memberi tahu server bahwa klien menginginkan respons dalam format JSON.
- User-Agent: Postman → identifikasi klien.
- Cookie: session=abc123 (walaupun biasanya lebih mudah dikelola melalui tab Cookies).
- Header dapat berupa pasangan key-value. Postman secara otomatis menambahkan beberapa header default (seperti Host), tetapi Anda bisa menghapus atau menambahkannya.
- Beberapa header bersifat wajib tergantung API (misalnya Authorization jika tidak menggunakan tab Authorization khusus).
- Anda juga bisa menggunakan variabel di dalam nilai header, misalnya {{auth_token}}.

4. Tab Body (Isi Permintaan)

- **Fungsi:** Mengisi data yang akan dikirim bersama permintaan, biasanya untuk metode **POST, PUT, PATCH**.
- **Mode yang tersedia:**
 - **none:** Tidak ada body (untuk GET, DELETE biasanya).
 - **form-data:** Untuk mengunggah file atau data formulir (mirip HTML form dengan multipart/form-data).
 - **x-www-form-urlencoded:** Data formulir dengan encoding URL (key-value, mirip query string tapi di body).
 - **raw:** Teks mentah, biasanya JSON, XML, atau teks biasa. Pilihan format (JSON, XML, HTML, Text) akan otomatis menambahkan header Content-Type yang sesuai.
 - **binary:** Mengunggah file biner (gambar, PDF, dll.).
 - **GraphQL:** Mode khusus untuk menulis query GraphQL.
 - Tab Body hanya aktif untuk metode yang mengizinkan body (POST, PUT, PATCH). Untuk GET, tab Body biasanya dinonaktifkan atau diabaikan oleh server.
 - Memilih mode yang tepat sangat penting: gunakan raw → JSON untuk REST API modern, gunakan form-data jika perlu unggah file.

5. Tab Pre-req. (Pre-request Script)

- **Fungsi:** Menulis **kode JavaScript** yang akan dieksekusi **sebelum** permintaan dikirim.
- **Kegunaan umum:**
 - Menyiapkan variabel lingkungan (misalnya mengambil timestamp untuk signature).
 - Menghasilkan token atau hash yang diperlukan untuk otentikasi.
 - Mengubah body atau header secara dinamis.
 - Mengambil data dari respons sebelumnya (jika ada rangkaian permintaan).
 - Skrip ini berjalan di sisi klien (Postman sandbox) dan memiliki akses ke objek pm (Postman object) seperti pm.environment, pm.variables, pm.request.
 - Contoh sederhana: pm.variables.set("currentTime", Date.now()); lalu gunakan {{currentTime}} di dalam URL atau header.
 - Pre-req. sangat berguna untuk **otomatisasi** dan **dinamisasi** permintaan.

6. Tab Tests (Pengujian)

- **Fungsi:** Menulis **kode JavaScript** yang akan dieksekusi **setelah** respons diterima.
- **Kegunaan utama:**
 - Memvalidasi kode status HTTP (misalnya mengharapkan 200 OK).
 - Memeriksa isi respons (misalnya apakah JSON mengandung properti tertentu, apakah nilai sesuai).
 - Memeriksa waktu respons (tidak boleh lebih dari 200 ms).
 - Menyimpan data dari respons ke variabel lingkungan untuk digunakan di permintaan berikutnya.
 - Postman menyediakan pustaka pm.test untuk mendefinisikan pengujian. Contoh:

```
pm.test("Status code is 200", function () {  
    pm.response.to.have.status(200);  
});  
pm.test("Response has user id", function () {  
    var jsonData = pm.response.json();  
    pm.expect(jsonData).to.have.property("id");  
});
```

- Hasil pengujian akan ditampilkan di tab **Test Results** pada bagian Response.
- Fitur ini memungkinkan **pengujian otomatis API** tanpa menulis kode eksternal. Anda bisa menjalankan koleksi berisi banyak request dengan skrip Tests dan langsung melihat mana yang lulus/gagal.

7. Tab Settings (Pengaturan) – (opsional, tergantung versi Postman)

- **Fungsi:** Mengatur perilaku khusus untuk permintaan saat ini.
- **Contoh pengaturan:**
 - **Follow redirects** (ikuti pengalihan otomatis atau tidak)
 - **Encode URL automatically** (otomatis mengencode karakter khusus)
 - **SSL certificate verification** (aktif/nonaktifkan verifikasi sertifikat)
 - **Timeout** (berapa lama menunggu respons)
 - Biasanya Anda tidak perlu mengubah setting ini untuk penggunaan umum. Namun, untuk pengujian tertentu (misalnya API internal dengan sertifikat self-signed), Anda mungkin perlu menonaktifkan verifikasi SSL.
 - Setting ini bersifat per-request, berbeda dengan pengaturan global di Postman.

Postman - Environment Variables

Variabel memberi opsi untuk menyimpan dan menggunakan ulang parameter dalam request, koleksi, skrip, dan sebagainya. Jika kita perlu mengubah suatu nilai, kita cukup melakukannya di satu tempat. Dengan demikian, variabel membantu meminimalkan kemungkinan kesalahan dan meningkatkan efisiensi.

Di Postman, **lingkungan (environment)** terdiri dari pasangan kunci-nilai (key-value). Lingkungan membantu mengidentifikasi setiap request secara terpisah. Saat kita membuat lingkungan, kita dapat mengubah pasangan kunci-nilai dan hal itu akan menghasilkan respons yang berbeda dari request yang sama.

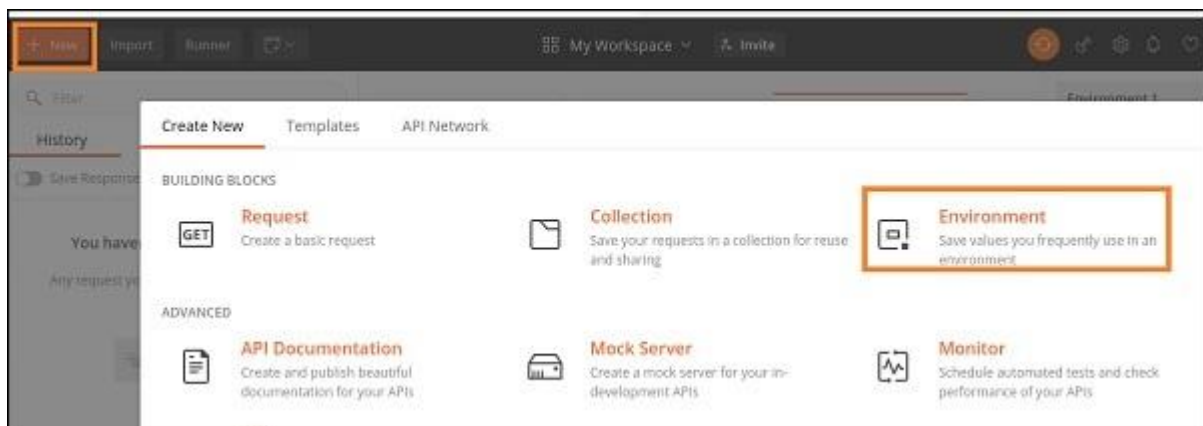
Kunci dalam pasangan kunci-nilai di lingkungan disebut sebagai **variabel lingkungan**. Kita dapat memiliki banyak lingkungan, dan masing-masing lingkungan juga dapat memiliki banyak variabel. Namun, kita hanya dapat bekerja dengan satu lingkungan dalam satu waktu.

Singkatnya, lingkungan memungkinkan eksekusi request dan koleksi dalam kumpulan data yang bervariasi. Kita dapat membuat lingkungan untuk produksi, pengujian, dan pengembangan. Masing-masing lingkungan ini akan memiliki parameter yang berbeda seperti URL, kata sandi, dan sebagainya.

Membuat Lingkungan (Create Environment)

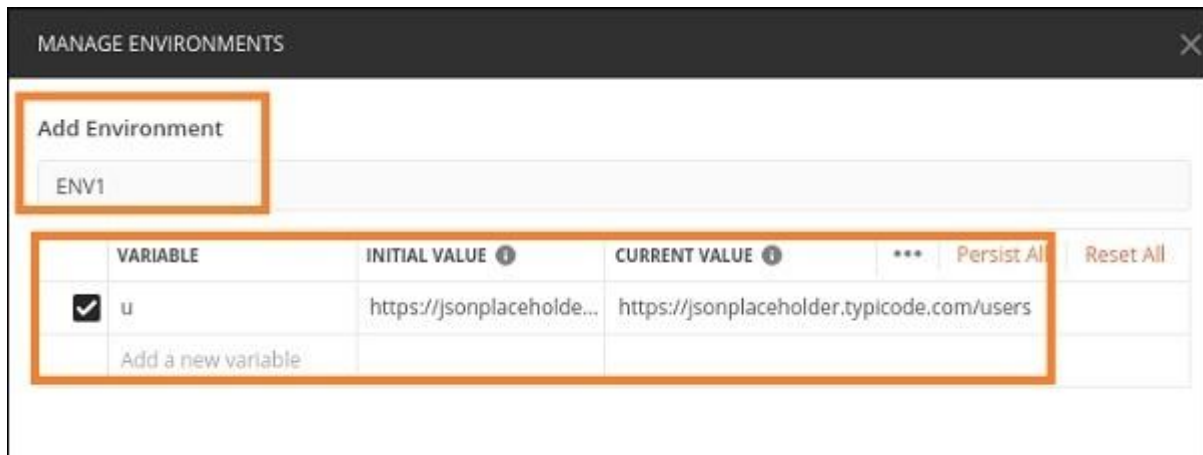
Ikuti langkah-langkah di bawah ini untuk membuat lingkungan di Postman:

Langkah 1 – Arahkan ke menu **New**, lalu klik pada **Environment**.

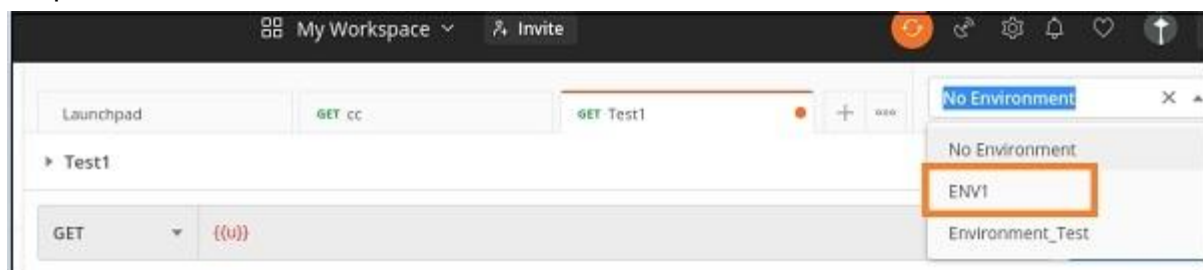


Langkah 2 – Munculan **pop-up MANAGE ENVIRONMENTS** terbuka. Kita harus memasukkan nama Environment. Kemudian, tambahkan nama variabel dan nilai.

Di sini, kita telah menambahkan variabel **u** dengan nilai <https://jsonplaceholder.typicode.com/users>. Tutup **pop-up** tersebut.

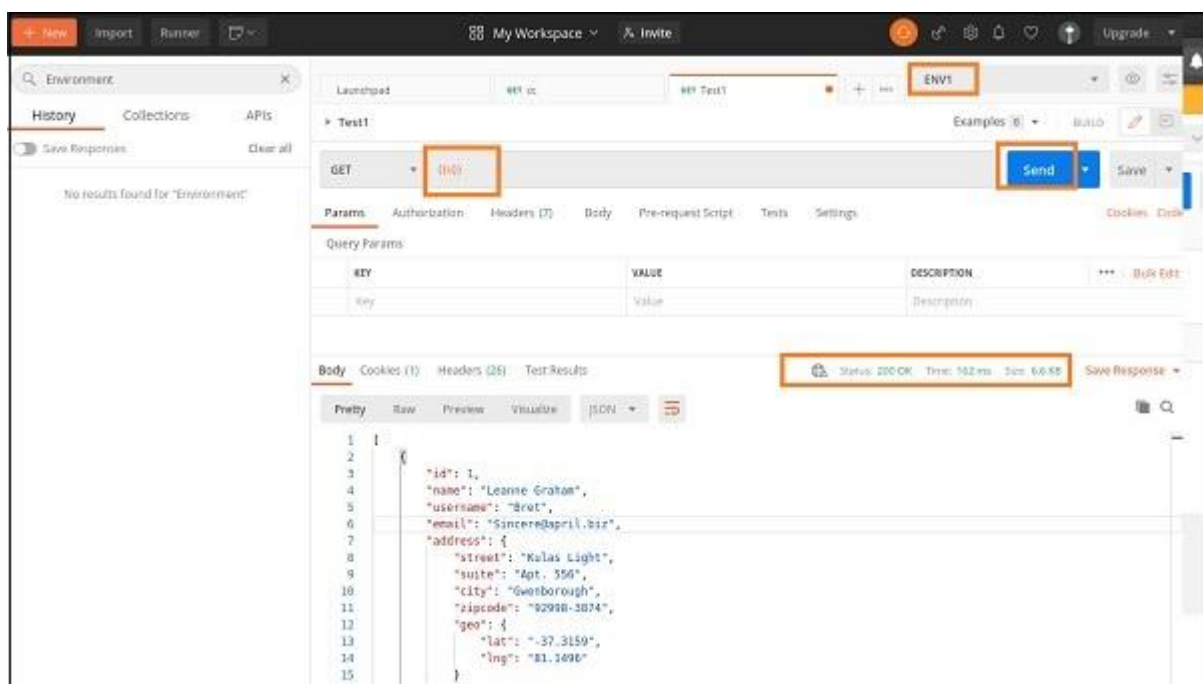


Langkah 3 – Environment baru (ENV1) akan muncul sebagai salah satu item di dalam dropdown **No Environment**.



Langkah 4 – Pilih environment **ENV1** dan masukkan {{u}} di bilah alamat. Untuk menggunakan variabel Environment dalam sebuah request, kita harus membungkusnya dengan kurung kurawal ganda ({{<nama variabel Environment>}}).

Langkah 5 – Kemudian, klik **Send**. Variabel ini dapat digunakan sebagai pengganti URL sebenarnya. Kita menerima kode respons **200 OK** (artinya request berhasil).



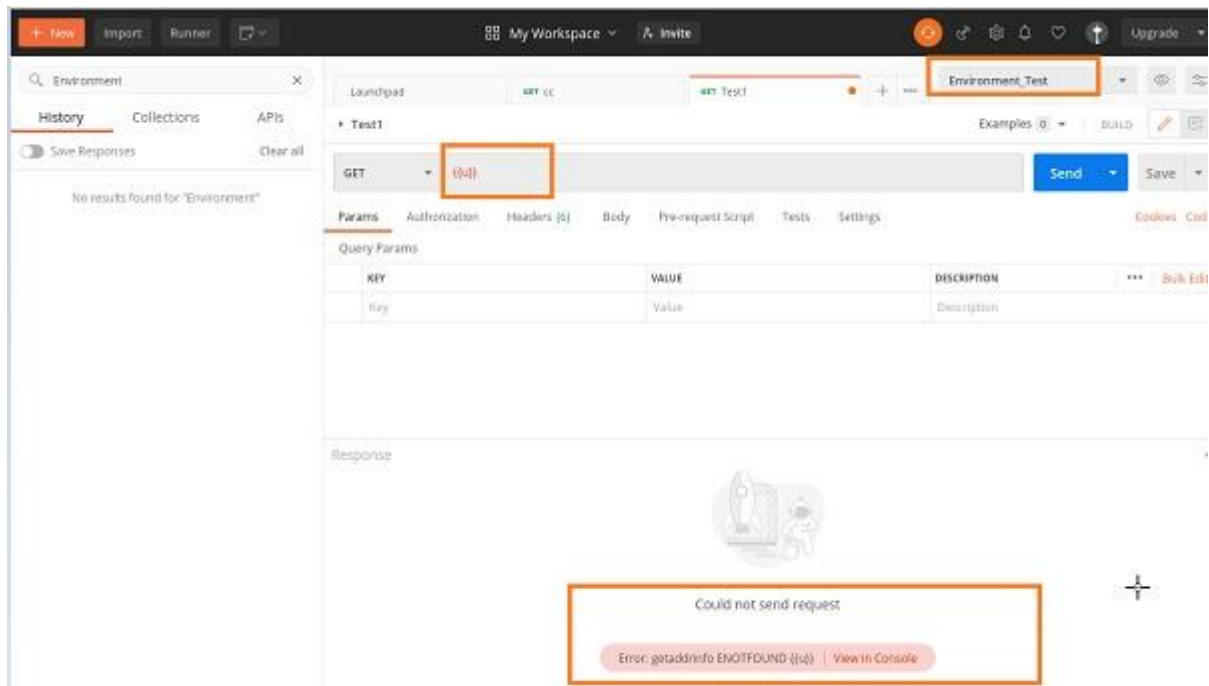
Cakupan Variabel Environment (Environment Variables Scope)

Cakupan dari sebuah variabel Environment berada di dalam environment tempat variabel tersebut dibuat. Artinya, variabel tersebut memiliki cakupan lokal yang terbatas pada environment itu. Jika kita memilih environment lain dan mencoba mengakses variabel Environment yang sama, kita akan mendapatkan error.

Pada bab ini, kita telah membuat variabel Environment u di dalam environment **ENV1** dan saat mengirim request GET, kita mendapatkan respons yang diinginkan.

Namun, jika kita mencoba menggunakan variabel Environment u yang sama dari environment lain, misalnya **Environment_Test**, kita akan menerima error.

Tampilan berikut menunjukkan error yang mungkin kita dapatkan jika menggunakan variabel Environment u yang sama dari environment lain:

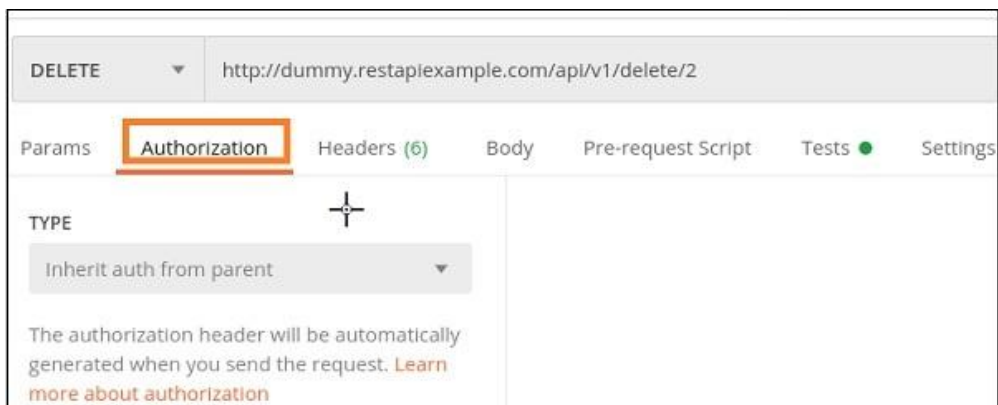


Postman - Authorization

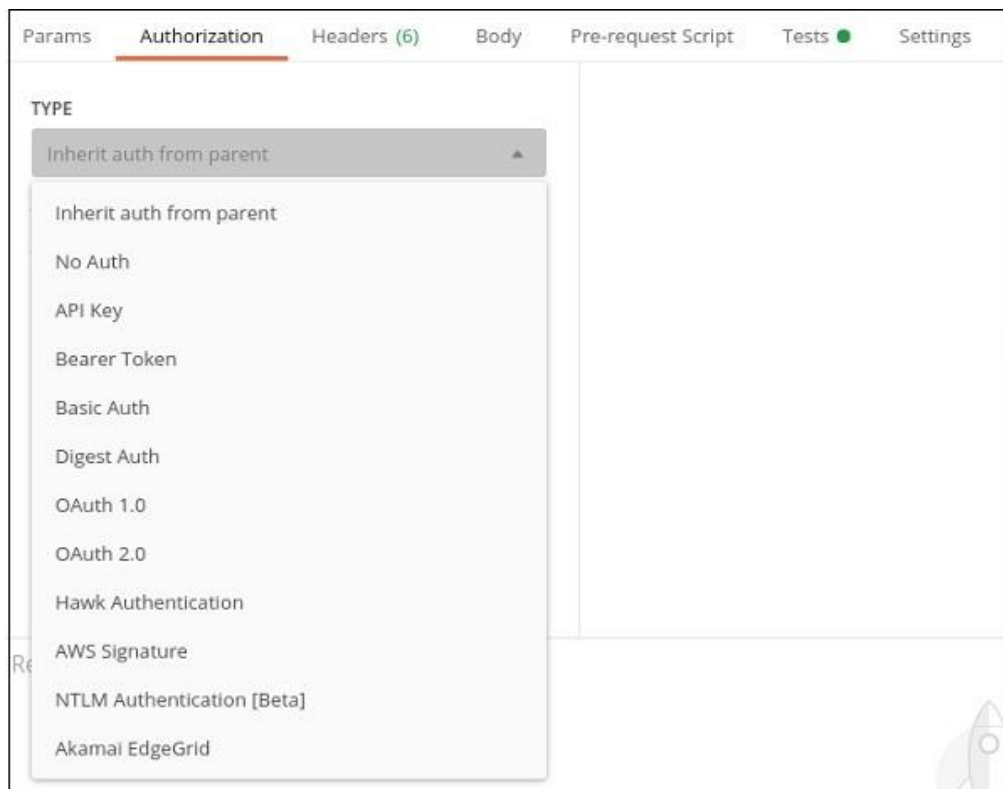
Di Postman, otorisasi dilakukan untuk memverifikasi kelayakan pengguna dalam mengakses sumber daya di server. Mungkin terdapat banyak API dalam suatu proyek, tetapi aksesnya dapat dibatasi hanya untuk pengguna tertentu yang diotorisasi.

Proses otorisasi diterapkan untuk API yang memerlukan keamanan. Otorisasi ini dilakukan untuk identifikasi dan verifikasi, apakah pengguna berhak mengakses sumber daya server.

Hal ini dilakukan di dalam tab **Authorization** di Postman, seperti yang ditunjukkan di bawah ini:



Di dalam dropdown **TYPE**, terdapat berbagai pilihan jenis Otorisasi, seperti yang ditunjukkan di bawah ini:

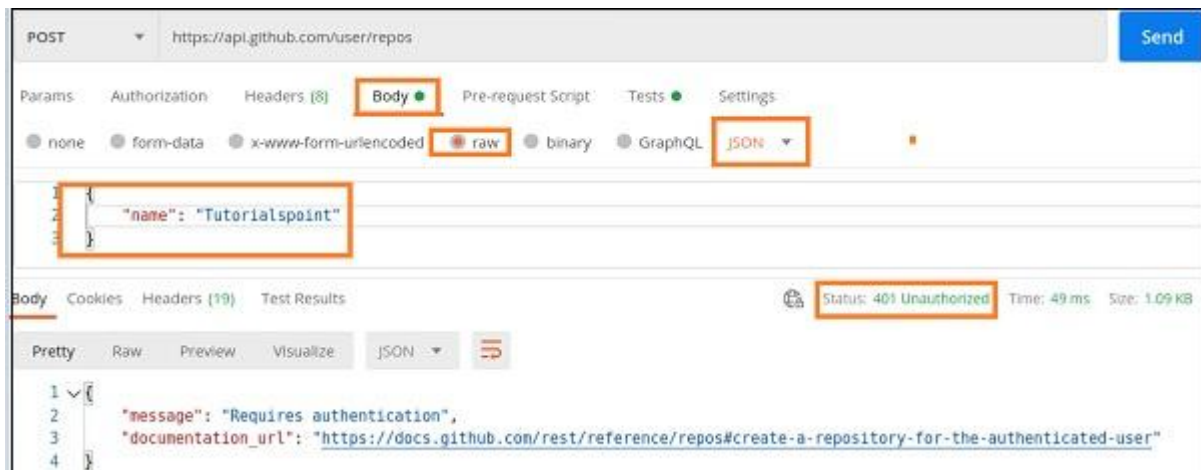


Sekarang mari kita buat request **POST** dengan API dari GitHub Developer yang memiliki endpoint `https://www.api.github.com/user/repos`. Di Postman, klik tab **Body** dan pilih opsi **raw**, lalu pilih format **JSON**.

Tambahkan body request berikut ini:

```
{
  "name" : "Tutorialspoint"
}
```

Kemudian, klik **Send**.



Response code yang diperoleh adalah **401 Unauthorized**. Ini berarti kita perlu memberikan otorisasi untuk menggunakan resource ini. Untuk melakukan otorisasi, pilih salah satu opsi dari dropdown **TYPE** di dalam tab **Authorization**.

Jenis-Jenis Otorisasi

Mari kita bahas beberapa jenis otorisasi yang penting, yaitu **Bearer Token** dan **Basic Authentication**.

Bearer Token

Untuk Otorisasi Bearer Token, kita harus memilih opsi **Bearer Token** dari dropdown **TYPE**. Setelah itu, akan muncul kolom **Token** yang perlu diisi untuk melengkapi otorisasi.

Langkah 1 – Untuk mendapatkan Token untuk GitHub API, pertama-tama masuk ke akun GitHub dengan mengklik tautan berikut: <https://github.com/login>.

Langkah 2 – Setelah masuk, klik pojok kanan atas layar dan pilih opsi **Settings**.

Apa itu Bearer Token?

Bearer Token adalah sebuah kredensial keamanan yang menandakan bahwa pemilik token ("bearer") memiliki izin untuk mengakses resource tertentu. Token ini biasanya berupa string panjang yang unik dan bersifat rahasia. Dalam konteks API modern (terutama REST API), Bearer Token sering digunakan sebagai mekanisme otentikasi yang sederhana namun aman, terutama untuk API yang menggunakan OAuth 2.0.

Cara Kerja Bearer Token di Postman

Ketika Anda memilih **Bearer Token** dari dropdown TYPE, Postman secara otomatis akan menambahkan header HTTP berikut ke dalam permintaan Anda:

```
Authorization: Bearer <token_anda>
```

Dengan demikian, Anda tidak perlu secara manual menambahkan header tersebut di tab **Headers**. Postman akan menanganinya untuk Anda.

Langkah Lengkap Mendapatkan Token GitHub (Personal Access Token)

Untuk melengkapi langkah yang disebutkan di atas, berikut adalah kelanjutan langkah-langkah mendapatkan token dari GitHub:

1. Setelah berada di halaman **Settings**, gulir ke bawah dan klik opsi **Developer settings** di menu samping kiri.
2. Klik **Personal access tokens**, lalu pilih **Tokens (classic)**.
3. Klik tombol **Generate new token**, lalu pilih **Generate new token (classic)**.
4. Beri nama token pada kolom **Note** (misalnya: "Postman Tutorial").
5. Pilih masa berlaku token (expiration) sesuai kebutuhan. Untuk keperluan pembelajaran, pilih **No expiration** atau jangka waktu tertentu.
6. Pada bagian **Select scopes**, centang **repo** (memberikan akses penuh ke repositori pribadi) atau pilih scope lain yang sesuai dengan kebutuhan Anda. Untuk membuat repositori melalui API, scope repo diperlukan.
7. Klik tombol hijau **Generate token** di bagian bawah halaman.
8. **Salin token yang dihasilkan** (Pastikan Anda menyalinnya karena token hanya akan ditampilkan sekali).
9. Tempel token yang telah disalin ke dalam kolom **Token** di tab **Authorization** Postman.
10. Klik **Send** untuk mengirim request POST Anda.

Peringatan Penting: Token ini memberikan akses ke akun GitHub Anda. Jangan pernah membagikannya, menyimpannya di kode publik, atau mengunggahnya ke repositori publik. Perlakukan token seperti kata sandi.

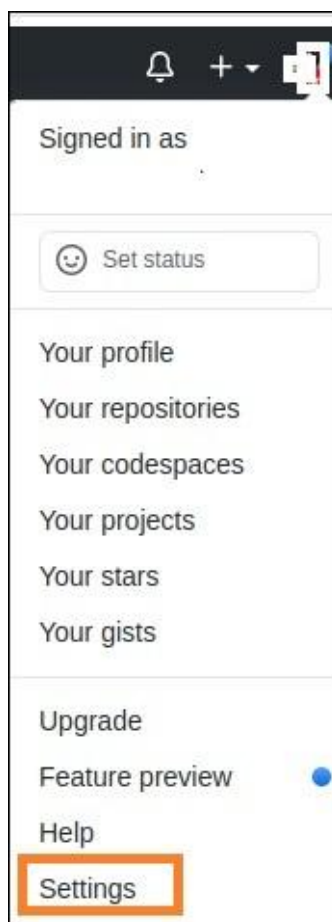
Response yang Diharapkan

Setelah mengirim request dengan Bearer Token yang valid, Anda akan mendapatkan **Response code 201 Created**. Ini menandakan bahwa repositori bernama "Tutorialspoint" telah berhasil dibuat di akun GitHub Anda.

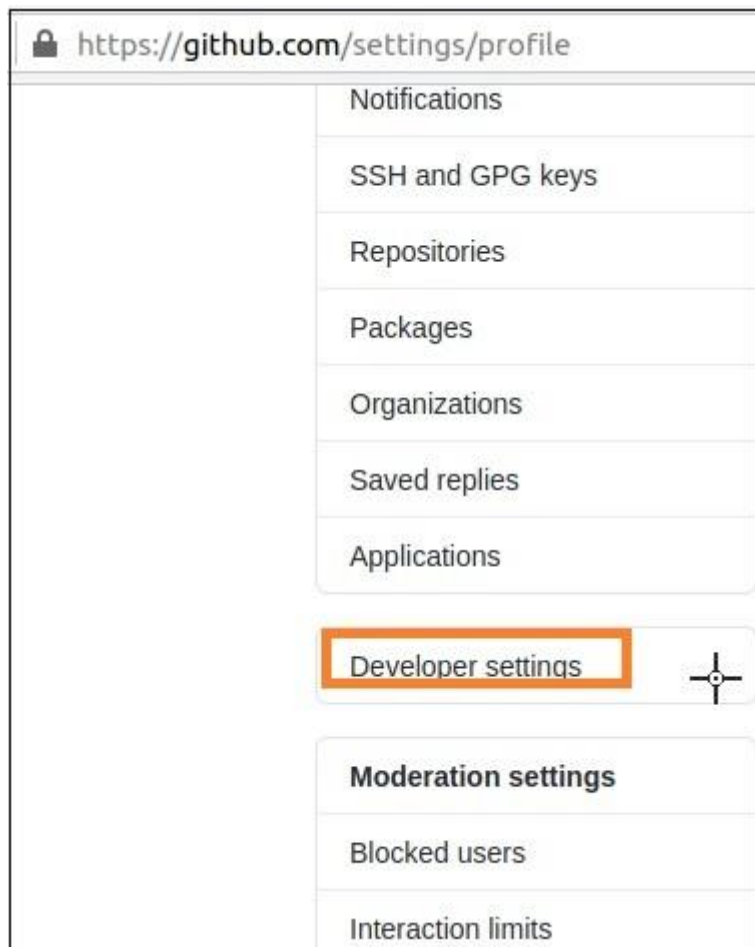
Mengapa Menggunakan Bearer Token?

Bearer Token lebih disukai daripada Basic Authentication dalam banyak skenario karena:

- **Keamanan:** Tidak perlu mengirimkan username dan password setiap kali request.
- **Fleksibilitas:** Token dapat memiliki izin terbatas (scope) dan masa berlaku.
- **Pencabutan:** Token yang terkompromi dapat dicabut tanpa mengubah password akun.



Sekarang, pilih opsi **Developer settings**.



Selanjutnya, klik **Personal access tokens**.



Sekarang, klik tombol **Generate new token**.



Berikan sebuah **Note** dan pilih opsi **repo**. Kemudian, klik **Generate Token** di bagian bawah halaman. Akhirnya, sebuah **Token** akan dibuat.

GitHub Apps

OAuth Apps

Personal access tokens

New personal access token

Personal access tokens function like ordinary OAuth access tokens. They can be used instead of a password over HTTPS, or can be used to authenticate to the API over Basic Authentication.

Note

Postman Testing

What's this token for?

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

- ☒ **repo** Full control of private repositories
 - ☒ repo:status Access commit status
 - ☒ repo_deployment Access deployment status
 - ☒ public_repo Access public repositories
 - ☒ repo:invite Access repository invitations
 - ☒ security_events Read and write security events

Salin **Token** tersebut dan tempelkan ke dalam kolom **Token** di bawah tab **Authorization** di Postman. Kemudian, klik **Send**.

Harap diperhatikan – Token ini bersifat unik untuk akun GitHub tertentu dan tidak boleh dibagikan.

Respons

Kode Respons yang diperoleh adalah **201 Created** yang berarti permintaan berhasil.

POST <https://api.github.com/user/repos>

Params **Authorization** Headers (9) Body Pre-request Script Tests Settings

TYPE

Bearer Token

The authorization header will be automatically generated when you send the

Token

Status: 201 Created Time: 1148 ms

Body Cookies Headers (25) Test Results

Pretty Raw Preview Visualize JSON

```

1 {
2   "id": 357300003,
3   "node_id": "MDEwOJlG9zaXRvcnkzNTczMDAwMDM=",
4   "name": "Tutorialspoint",
5   "full_name": "Tutorialspoint",
6   "private": false,
7   "owner": {
8     "login": "",
9     "id": 52132518,
10    "node_id": "MDQ",
11    "avatar_url": "https://avatars.githubusercontent.com/u/52132518?v=4",

```

Basic Authentication

Untuk Otorisasi *Basic Authentication*, kita harus memilih opsi **Basic Auth** dari dropdown **TYPE**, sehingga kolom **Username** dan **Password** akan ditampilkan.

Pertama, kita akan mengirim request **GET** ke sebuah endpoint (<https://postman-echo.com/basic-auth>) dengan opsi **No Auth** yang dipilih dari dropdown **TYPE**.

Harap diperhatikan – **Username** untuk endpoint di atas adalah postman dan **password** adalah password.

Apa itu Basic Authentication?

Basic Authentication adalah metode otentikasi paling sederhana yang didukung oleh protokol HTTP. Klien mengirimkan username dan password dalam bentuk teks yang digabung dengan tanda titik dua (username:password), lalu diencode menggunakan **Base64**. Hasil encode ini kemudian dimasukkan ke dalam header Authorization dengan format:

```
Authorization: Basic <base64-string>
```

Contoh: Untuk username postman dan password password, string postman:password diencode Base64 menjadi cG9zdG1hbJpwYXNzd29yZA==. Maka header yang dikirim adalah:

```
Authorization: Basic cG9zdG1hbJpwYXNzd29yZA==
```

Mengapa Percobaan Pertama dengan "No Auth" Akan Gagal?

Endpoint <https://postman-echo.com/basic-auth> adalah layanan uji dari Postman yang dirancang untuk mengembalikan respons **200 OK** hanya jika klien mengirimkan kredensial yang benar (postman / password). Jika tidak mengirimkan otentikasi sama sekali (memilih **No Auth**), server akan merespons dengan **401 Unauthorized**. Ini membuktikan bahwa API tersebut memerlukan otentikasi.

Langkah Menggunakan Basic Authentication di Postman

1. Pada tab **Authorization**, pilih **Basic Auth** dari dropdown **TYPE**.
2. Isi kolom **Username** dengan postman.
3. Isi kolom **Password** dengan password.
4. Klik **Send**.

Postman secara otomatis akan mengencode kredensial dan menambahkan header Authorization yang sesuai.

Respons yang Diharapkan

Jika kredensial benar, server akan mengembalikan:

- **Status Code:** 200 OK
- **Response Body** (JSON):

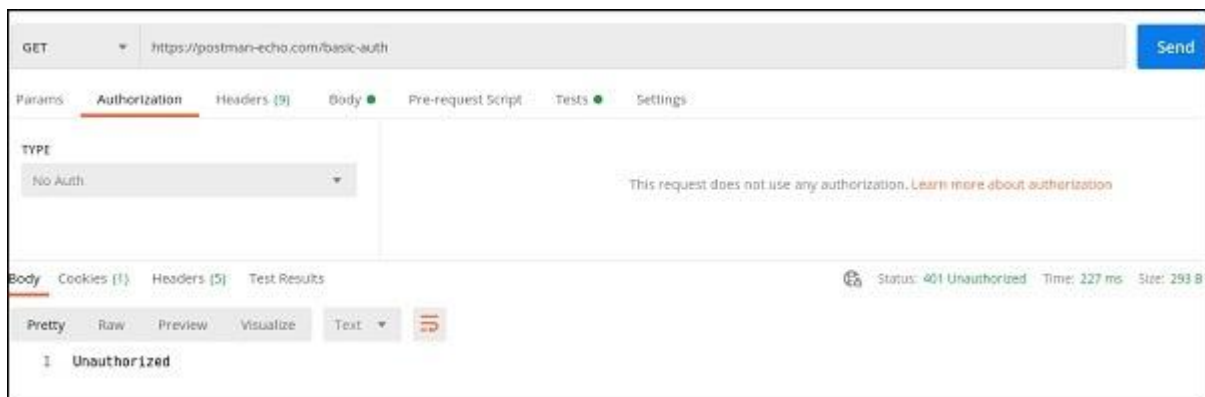
```
{  
  "authenticated": true  
}
```

Jika kredensial salah, Anda akan mendapatkan **401 Unauthorized** dengan body berisi `authenticated: false`.

Kelemahan Basic Authentication

- **Tidak aman jika hanya menggunakan HTTP biasa** (tanpa HTTPS) karena kredensial hanya diencode Base64 (bukan dienkripsi). Base64 mudah didekode.
- **Harus selalu menggunakan HTTPS** untuk mengenkripsi seluruh komunikasi.
- Tidak memiliki mekanisme *logout* atau *token expiration* yang baik. Kredensial dikirim setiap request.
- Saat ini banyak digantikan oleh Bearer Token (OAuth 2.0) atau API Key.

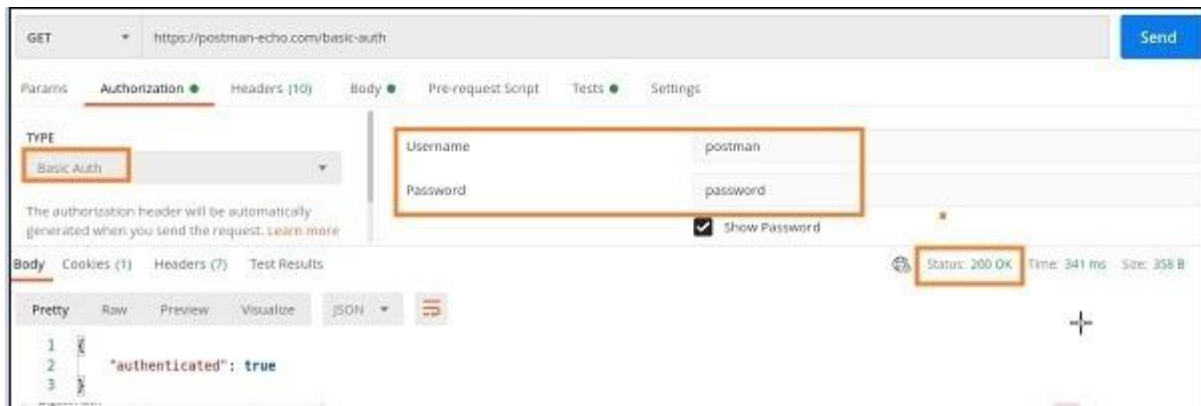
Namun, Basic Authentication masih banyak digunakan untuk API internal, sederhana, atau untuk keperluan pengujian seperti contoh di atas.



Kode Respons yang diperoleh adalah **401 Unauthorized**. Ini berarti Otorisasi tidak berhasil untuk API ini.

Sekarang, mari kita pilih opsi **Basic Auth** sebagai jenis Otorisasi, setelah itu kolom **Username** dan **Password** akan ditampilkan.

Masukkan postman untuk kolom **Username** dan password untuk kolom **Password**. Kemudian, klik **Send**.



Kode Respons yang diperoleh sekarang adalah **200 OK**, yang berarti permintaan kita telah berhasil dikirim.

No Auth

Kita juga dapat melakukan Basic Authentication menggunakan **Header** permintaan. Pertama, kita harus memilih opsi **No Auth** dari tab Authorization. Kemudian, di tab **Headers**, kita harus menambahkan pasangan kunci-nilai (key-value).

Kita akan menggunakan **key** sebagai Authorization dan **value** adalah username serta password pengguna dalam format basic <encoded credential>.

Endpoint yang digunakan dalam contoh kita adalah: <https://postman-echo.com/basic-auth>. Untuk melakukan encoding username dan password, kita akan menggunakan bantuan aplikasi pihak ketiga dengan URL: <https://www.base64encode.org>

Harap diperhatikan – **Username** untuk endpoint kita di sini adalah postman dan **password** adalah password. Masukkan postman:password di kotak edit dan klik **Encode**. Nilai hasil encoding akan muncul di bagian bawah.

Apa yang Terjadi saat Memilih "No Auth"?

Ketika Anda memilih **No Auth** di tab Authorization, Postman **tidak akan menambahkan** header Authorization secara otomatis. Ini berguna jika Anda ingin mengatur header secara manual, misalnya untuk menguji atau meniru cara kerja klien HTTP mentah.

Cara Kerja Basic Authentication Melalui Header Manual

Meskipun Postman menyediakan opsi **Basic Auth** yang otomatis menangani encoding, Anda juga bisa melakukannya secara manual untuk pemahaman yang lebih mendalam. Berikut langkah-langkahnya:

1. Di tab **Authorization**, pilih **No Auth**.
2. Buka tab **Headers**.
3. Tambahkan baris baru:

- **Key:** Authorization
- **Value:** Basic <hasil encoding Base64>

Proses Encoding Base64

- Format yang diharapkan: username:password (dengan tanda titik dua di antara username dan password).
- Untuk contoh ini: postman:password
- Hasil encoding Base64 dari postman:password adalah cG9zdG1hbjpgYXNzd29yZA==
- Maka nilai header Authorization yang lengkap menjadi: Basic cG9zdG1hbjpgYXNzd29yZA==

Cara Mendapatkan Encoding (Alternatif Selain base64encode.org)

Anda juga bisa melakukan encoding langsung di konsol browser (F12 → Console) dengan perintah:

```
btoa("postman:password")
```

Atau menggunakan terminal Linux/Mac:

```
echo -n "postman:password" | base64
```

Mengirim Request Manual dengan Header

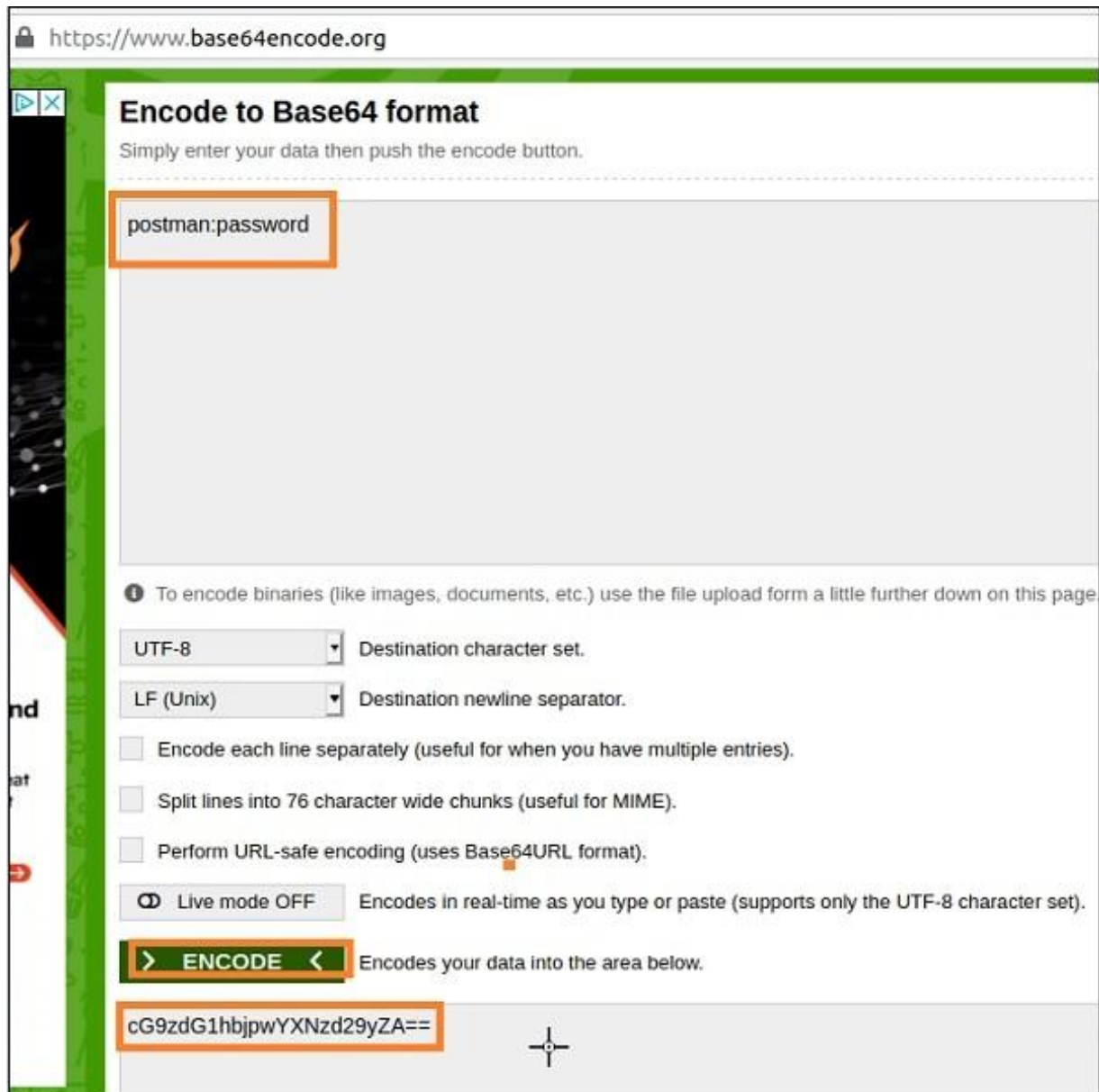
Setelah menambahkan header Authorization dengan nilai yang sudah diencode, klik **Send**. Anda akan mendapatkan respons **200 OK** sama seperti ketika menggunakan opsi **Basic Auth** otomatis.

Perbandingan: Opsi Basic Auth vs No Auth Manual

Aspek	Basic Auth (otomatis)	No Auth + Header Manual
Kemudahan	Sangat mudah, cukup isi username & password	Perlu encoding sendiri
Risiko kesalahan	Rendah	Lebih tinggi (salah format, salah encoding)
Pemahaman konsep	Kurang mendalam	Membantu memahami cara kerja HTTP Basic Auth
Penggunaan praktis	Direkomendasikan untuk sehari-hari	Untuk pembelajaran atau kasus khusus

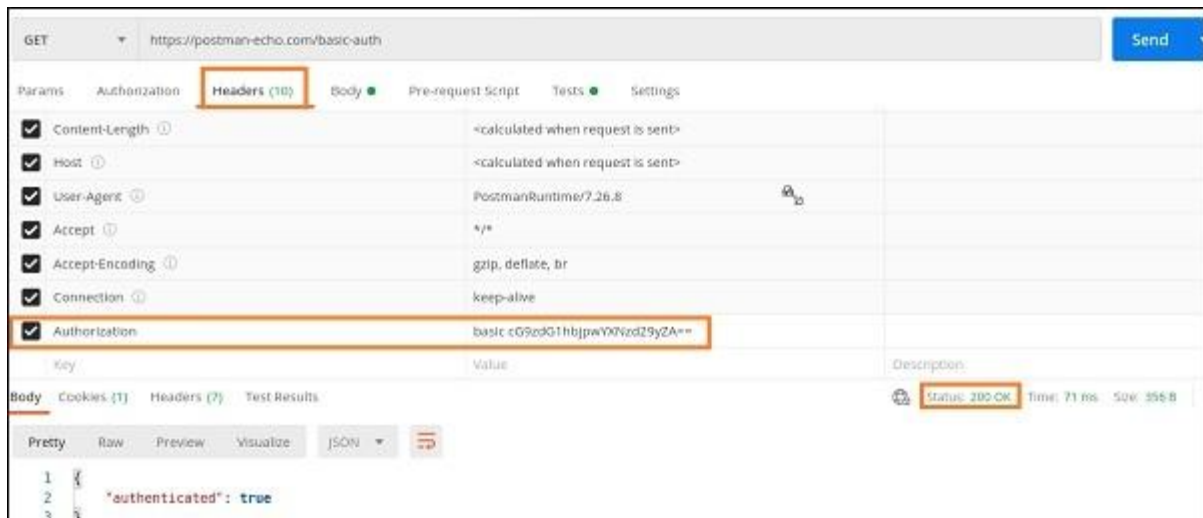
Catatan Penting

- **Spasi** setelah Basic sangat penting. Format penulisan: Basic <encoded> (ada spasi).
- Base64 **bukan enkripsi**, hanya encoding. Jangan pernah menganggapnya aman. Selalu gunakan HTTPS.
- Jika username atau password mengandung karakter non-ASCII atau khusus, pastikan encoding dilakukan dengan benar (biasanya UTF-8 terlebih dahulu).

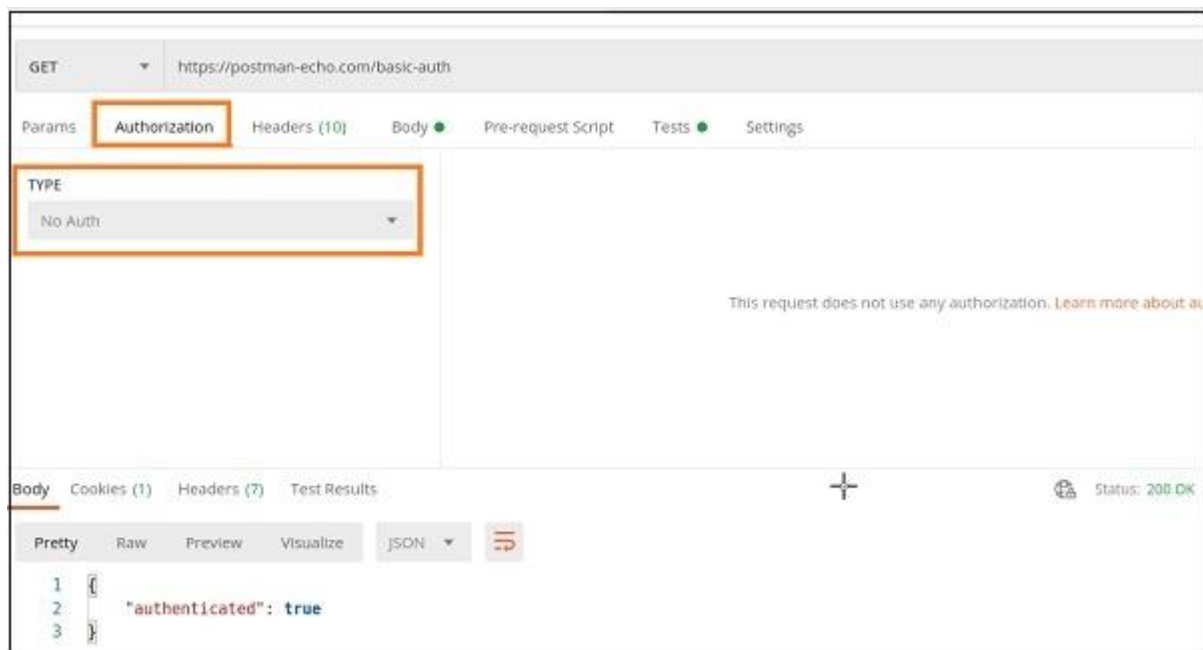


The screenshot shows a web browser window with the URL <https://www.base64encode.org>. The page title is "Encode to Base64 format". Below the title, it says "Simply enter your data then push the encode button." A text input field contains the text "postman:password". Below the input field, there is a section for encoding options. The "Destination character set" is set to "UTF-8". The "Destination newline separator" is set to "LF (Unix)". There are three checkboxes: "Encode each line separately (useful for when you have multiple entries)", "Split lines into 76 character wide chunks (useful for MIME)", and "Perform URL-safe encoding (uses Base64URL format)". The "Live mode" is set to "OFF". A green button labeled "ENCODE" is visible. Below the button, the encoded result "cG9zdG1hbJpwYXNzd29yZA==" is displayed in a text field.

Kita akan menambahkan Username dan Password yang telah di-encode, yaitu `cG9zdG1hbJpwYXNzd29yZA==`, ke dalam Header dengan format basic `cG9zdG1hbJpwYXNzd29yZA ==`. Kemudian, klik **Send**.



No Auth dipilih dari dropdown TYPE.

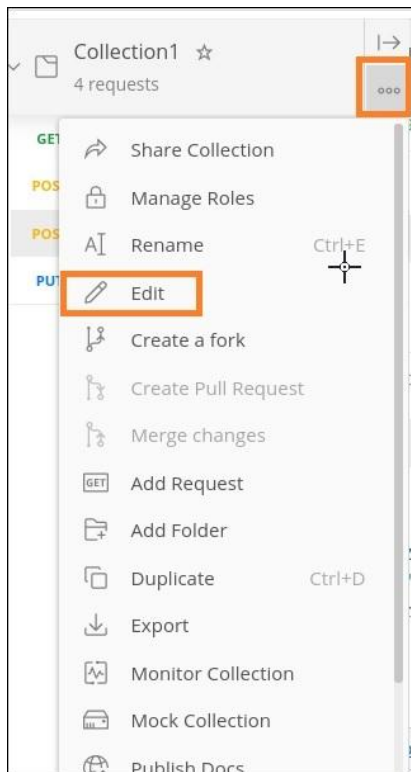


Kode Respons yang diperoleh adalah **200 OK**, yang berarti permintaan kita telah berhasil dikirim.

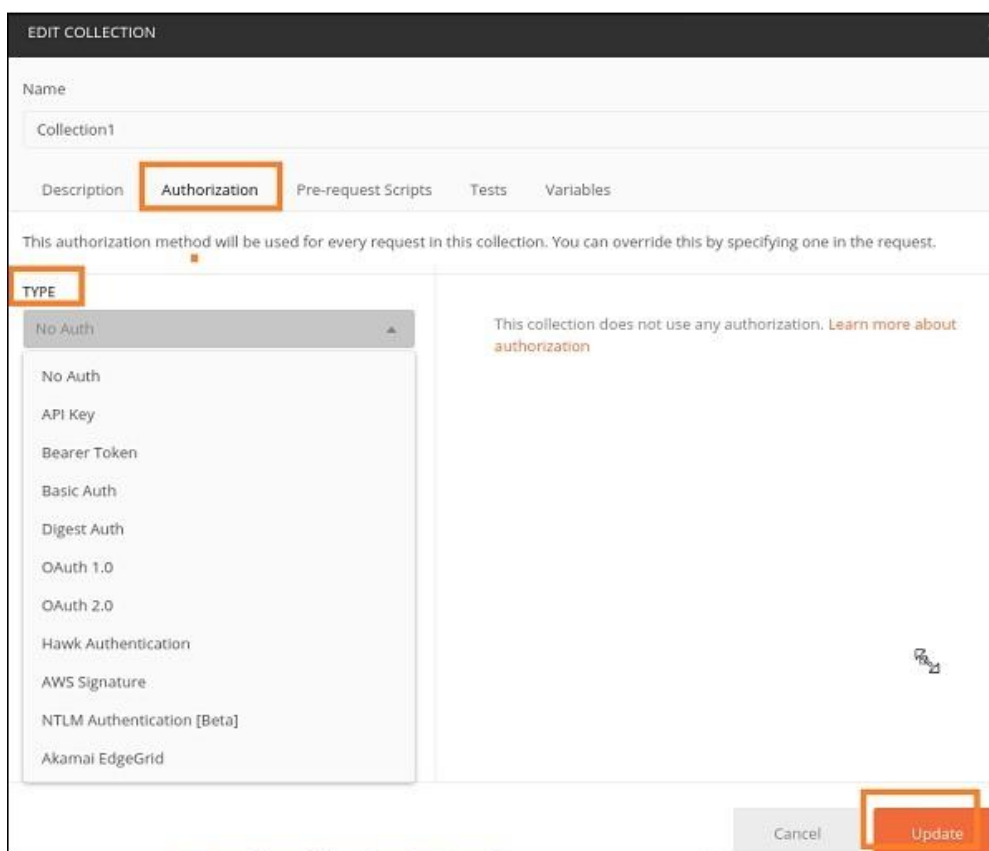
Otorisasi pada Koleksi (Authorization at Collections)

Untuk menambahkan Otorisasi pada sebuah Koleksi, ikuti langkah-langkah di bawah ini:

Langkah 1 – Klik tiga titik di samping nama Koleksi di Postman dan pilih opsi **Edit**.



Langkah 2 – Munculan *pop-up EDIT COLLECTION* akan terbuka. Pindah ke tab **Authorization** lalu pilih salah satu opsi dari dropdown **TYPE**. Klik **Update**.

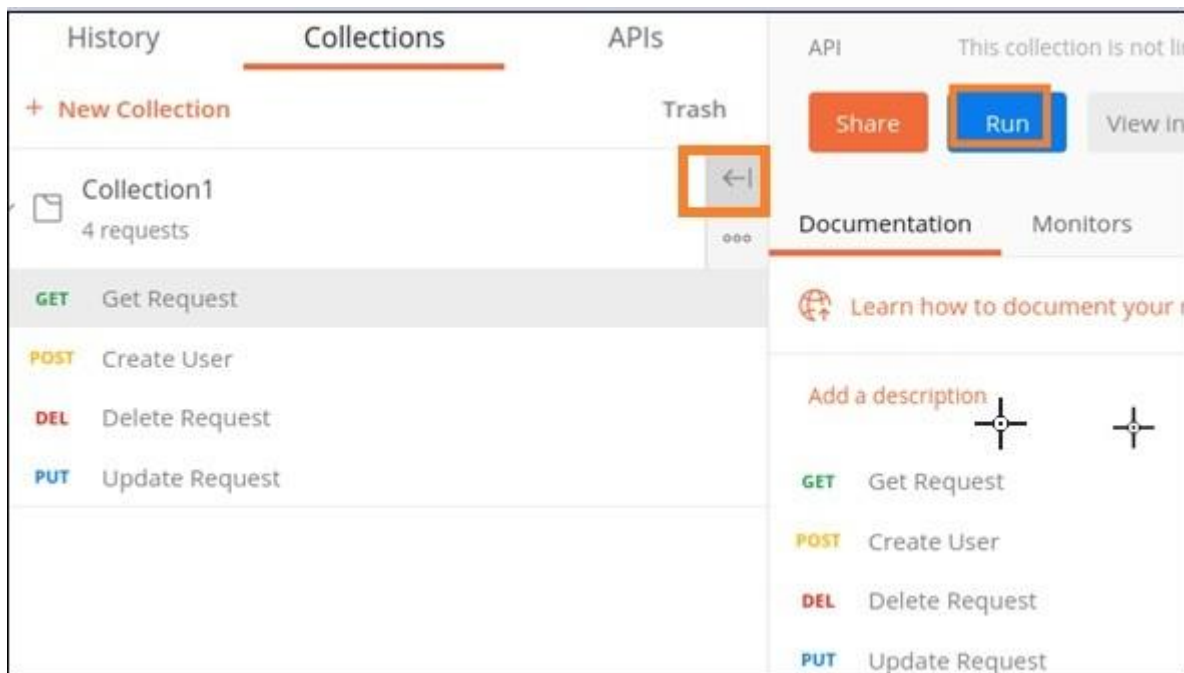


Postman - Workflows

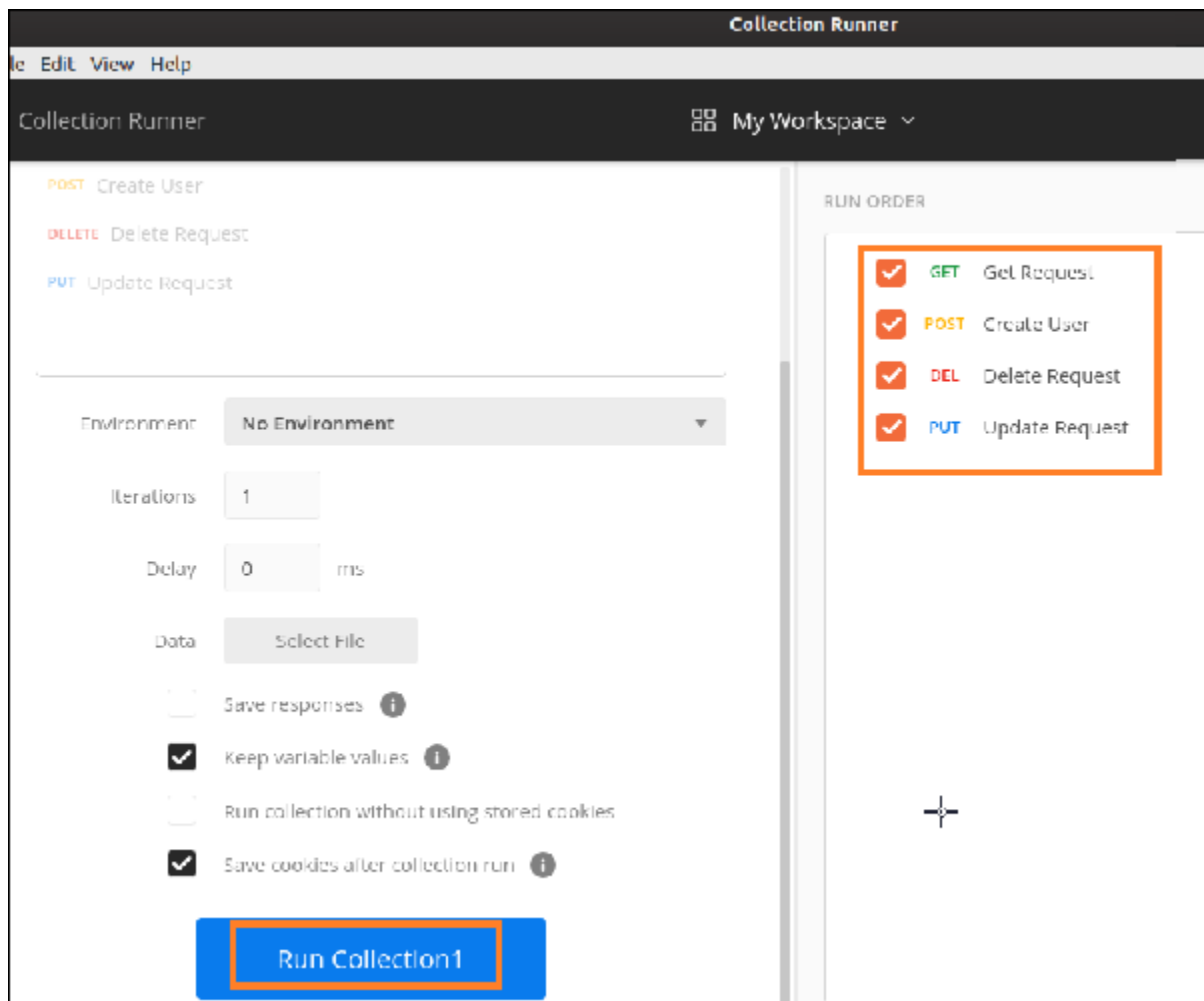
Di dalam sebuah Koleksi Postman, permintaan (request) dieksekusi sesuai urutan kemunculannya. Setiap request dijalankan pertama kali berdasarkan urutan folder, diikuti oleh request lain yang berada di akar Koleksi.

Mari kita buat sebuah Koleksi (Collection1) dengan empat request. Detail tentang cara membuat Koleksi telah dibahas pada bab *Create Collections*.

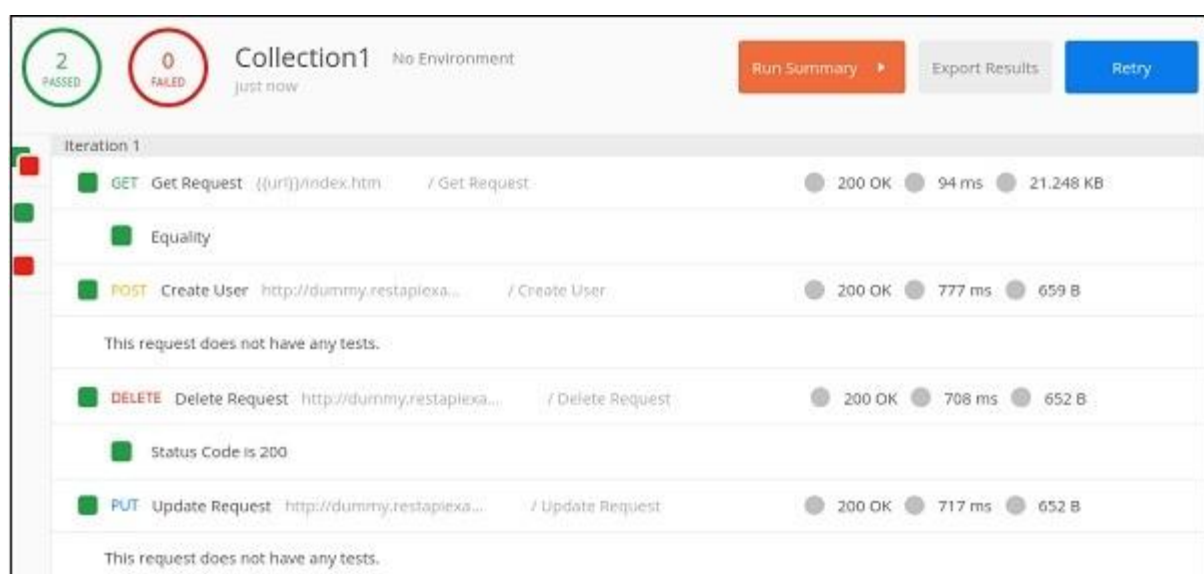
Langkah 1 – Klik tanda panah di sebelah kanan nama Koleksi di sidebar. Kemudian, klik tombol **Run** untuk memicu eksekusi request di dalam Koleksi tersebut.



Langkah 2 – Munculan *pop-up Collection Runner* terbuka. Bagian **RUN ORDER** menunjukkan urutan request yang akan dieksekusi dari atas ke bawah (GET → POST → DEL → PUT). Klik tombol **Run Collection1**.



Langkah 3 – Hasil eksekusi menunjukkan bahwa request GET dijalankan pertama, diikuti POST, lalu DEL, dan terakhir PUT, seperti yang disebutkan di bagian RUN ORDER pada Langkah 2.



Jika kita ingin mengubah urutan eksekusi request (misalnya: request Get dijalankan pertama, lalu Create User, kemudian Update Request, dan terakhir Delete Request), kita harus menggunakan fungsi `postman.setNextRequest()`.

Fungsi ini memiliki fitur untuk menentukan request mana yang akan dijalankan berikutnya. Nama request yang akan dijalankan selanjutnya diteruskan sebagai parameter ke fungsi ini. Sesuai dengan alur kerja, kita harus menambahkan fungsi ini baik di tab **Tests** atau **Pre-request Script** di bawah bilah alamat endpoint di Postman.

Sintaks untuk eksekusi request di Postman adalah sebagai berikut:

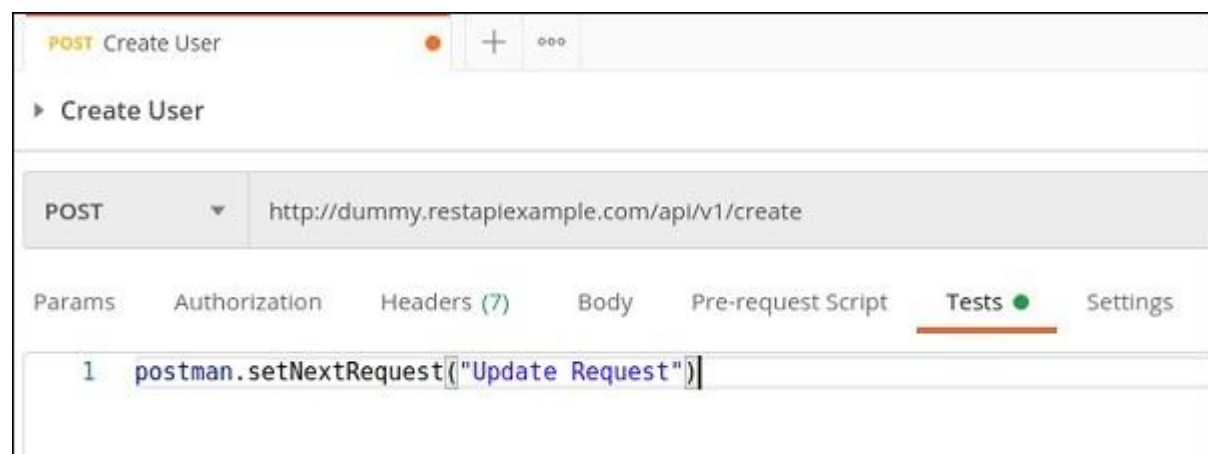
```
postman.setNextRequest("nama request")
```

Implementasi Alur Kerja (Workflow)

Implementasi alur kerja di Postman dijelaskan di bawah ini secara langkah demi langkah:

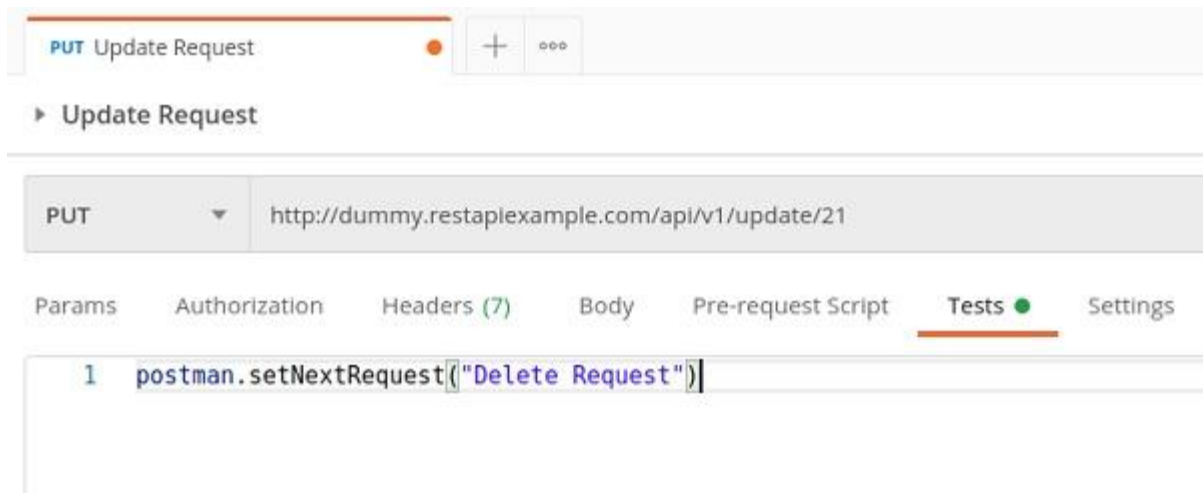
Langkah 1 – Tambahkan skrip di bawah ini di tab **Tests** untuk request **Create User**:

```
postman.setNextRequest("Update Request")
```



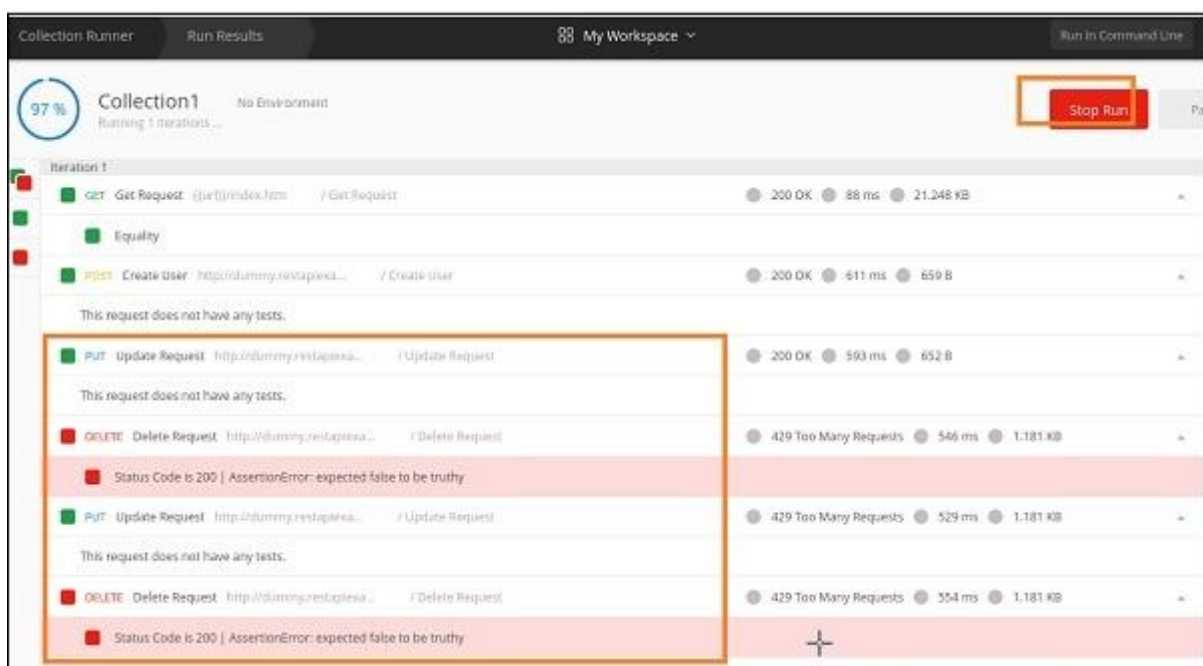
Langkah 2 – Tambahkan skrip di bawah ini di tab **Tests** untuk request **Update Request**:

```
postman.setNextRequest("Delete Request")
```



Keluaran (Output) Alur Kerja

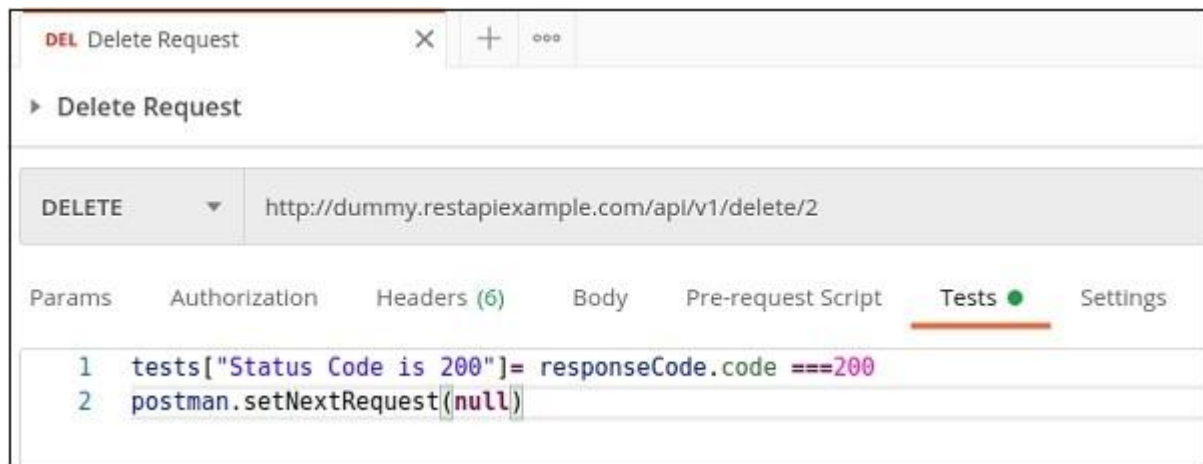
Keluaran menunjukkan bahwa **Update Request** dan **Delete Request** berjalan dalam putaran tak terbatas (*infinite loop*) hingga kita menghentikannya dengan menekan tombol **Stop Run**.



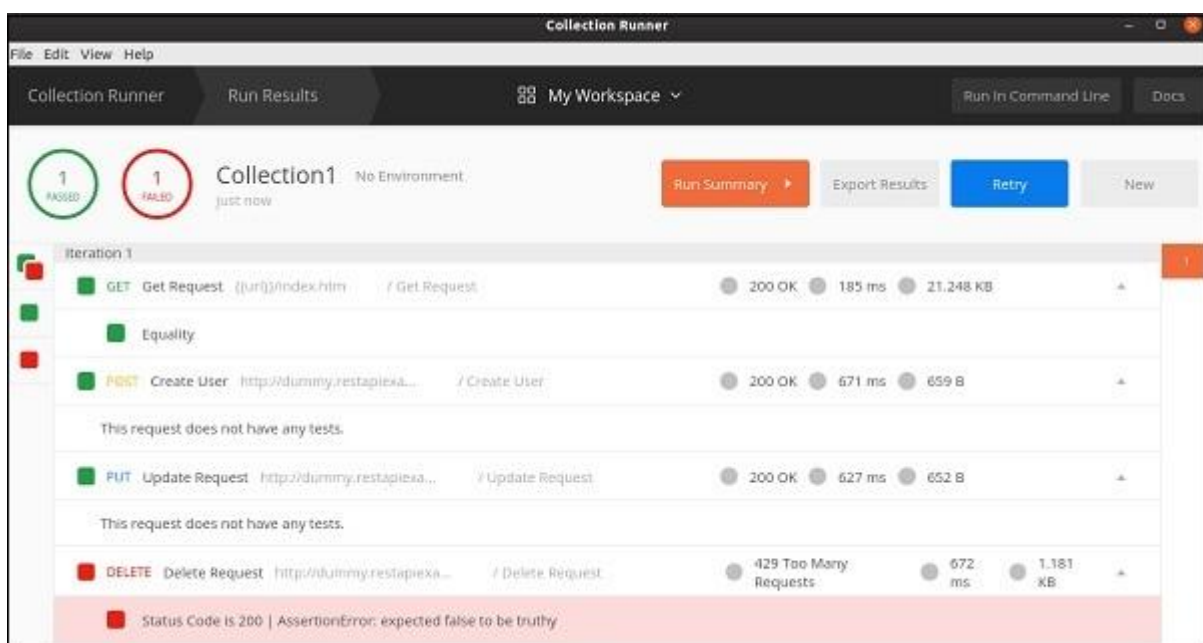
Putaran Alur Kerja Tak Terbatas (Infinite Workflow Loop)

Jika kita ingin menghentikan putaran alur kerja tak terbatas melalui skrip, kita harus menambahkan skrip di bawah ini untuk request **Delete Request**:

```
postman.setNextRequest(null)
```



Jalankan lagi Koleksi yang sama dan keluarannya akan sebagai berikut:



Keluaran menunjukkan urutan eksekusi sebagai: **Get Request, Create User, Update Request, dan terakhir Delete Request.**

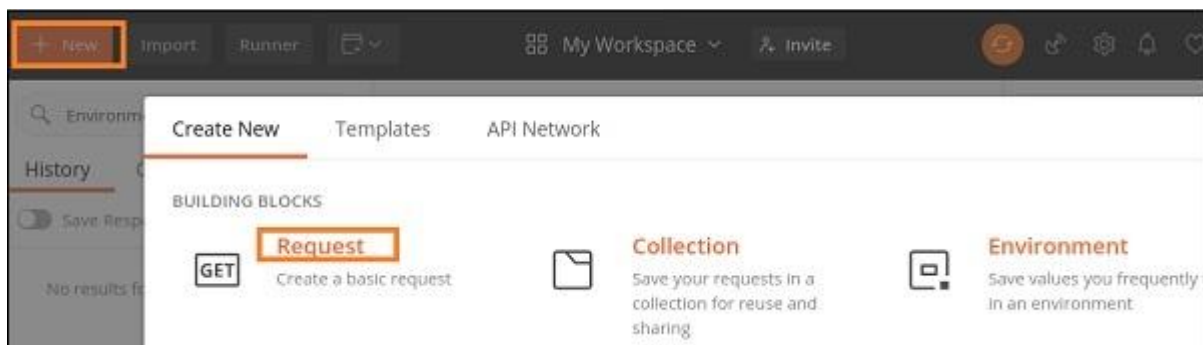
Postman - GET Requests

Permintaan GET digunakan untuk mendapatkan data dari server dan tidak memberikan dampak apa pun pada server. Permintaan GET tidak memperbarui data server saat dipicu. Server hanya mengirimkan Responsnya terhadap permintaan tersebut.

Membuat Permintaan GET

Ikuti langkah-langkah di bawah ini untuk berhasil membuat permintaan GET di Postman:

Langkah 1 – Klik menu **New** dari aplikasi Postman. Munculan *Create New* terbuka. Kemudian klik tautan **Request**.



Langkah 2 – Munculan *SAVE REQUEST* terbuka. Masukkan nama Request, lalu klik **Save**.

SAVE REQUEST

Requests in Postman are saved in collections (a group of requests).
[Learn more about creating collections](#)

Request name

Request description (Optional)

Make things easier for your teammates with a complete request description.

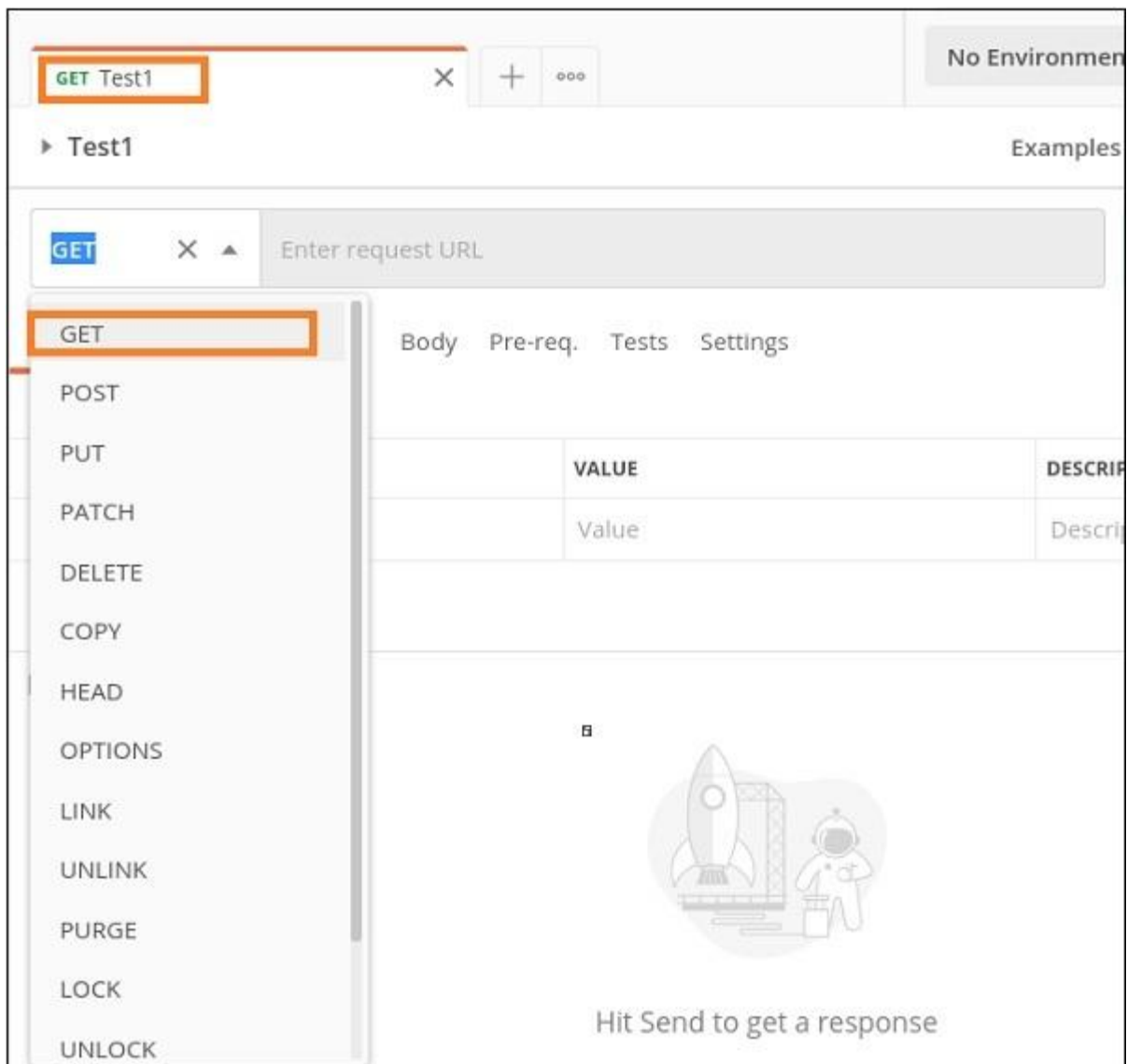
Descriptions support [Markdown](#)

Select a collection or folder to save to:

Cancel

Save to FirstTest

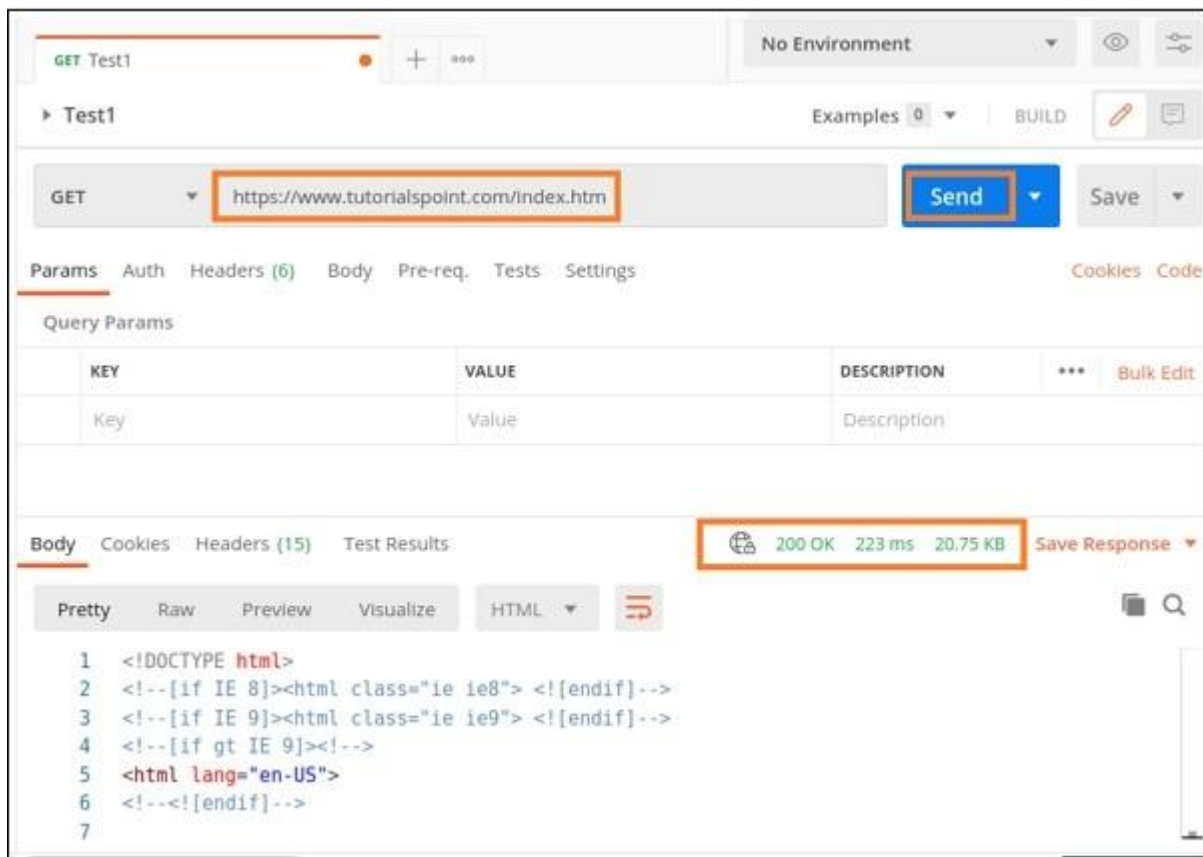
Langkah 3 – Nama Request (Test1) akan muncul di tab Request. Kemudian kita pilih opsi **GET** dari dropdown permintaan HTTP.



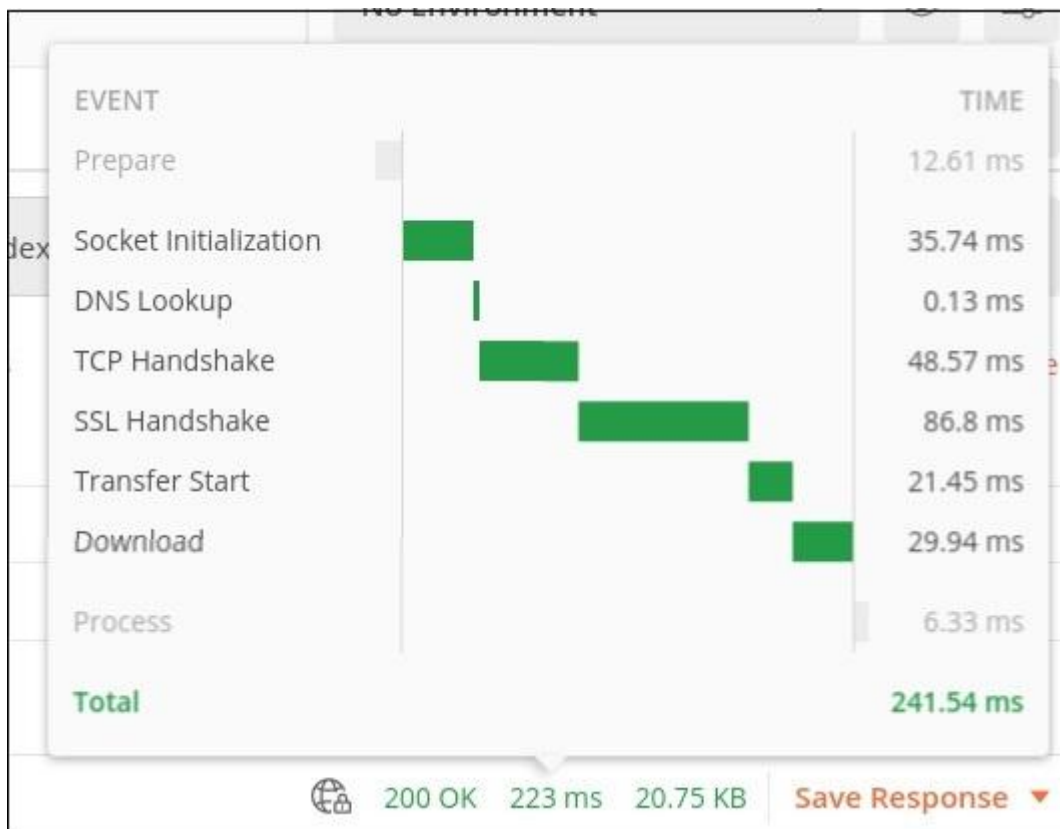
Langkah 4 – Masukkan URL - <https://www.tutorialspoint.com/index.htm> di bilah alamat dan klik **Send**.

Respons

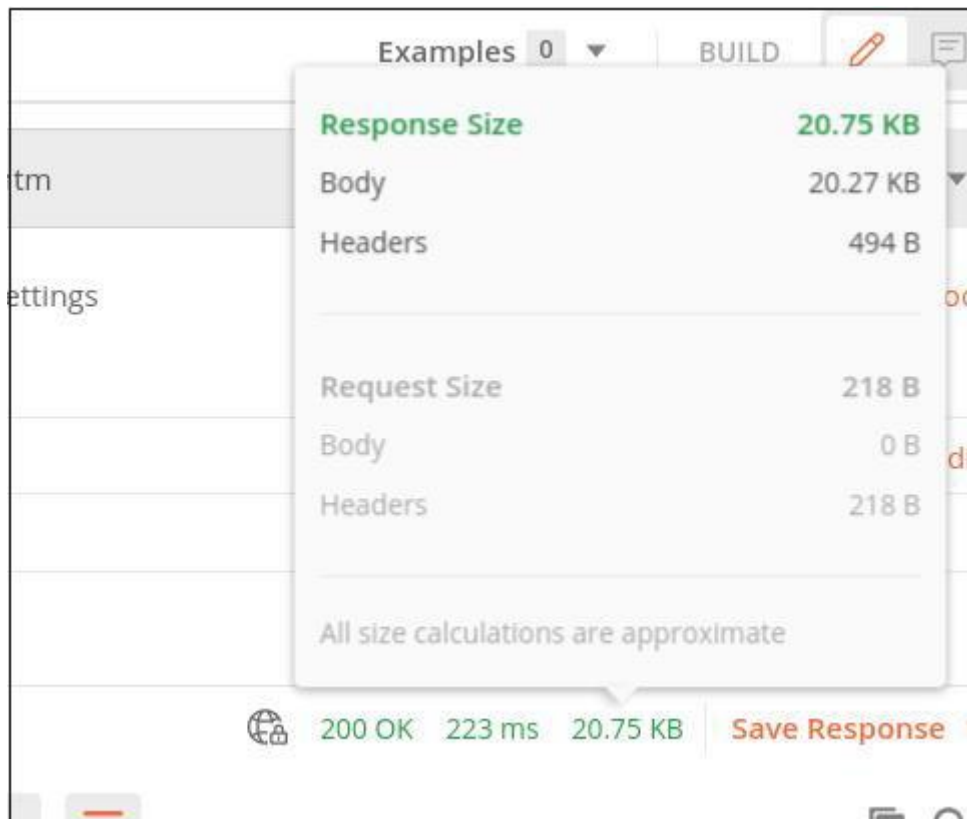
Setelah permintaan dikirim, kita dapat melihat kode respons **200 OK** muncul di bagian Response. Ini menandakan permintaan berhasil dan endpoint benar. Selain itu, informasi tentang waktu yang diperlukan untuk menyelesaikan permintaan (223 ms) dan ukuran *payload* (20.75 KB) juga ditampilkan.



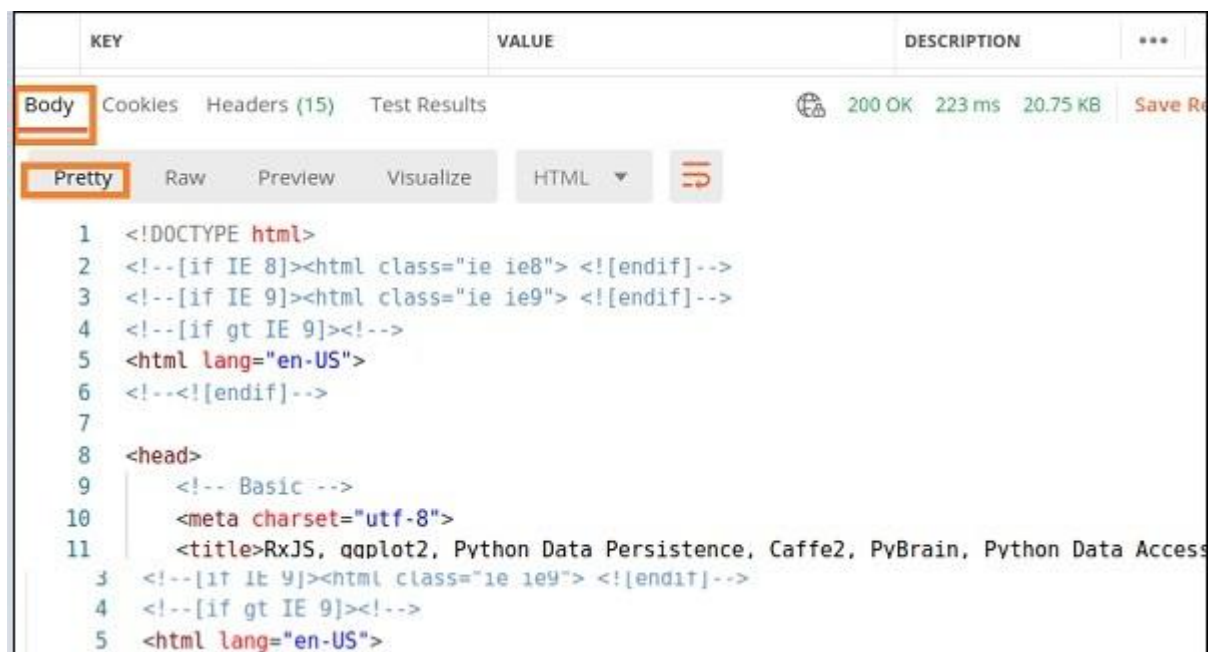
Saat mengarahkan kursor ke waktu respons, kita dapat melihat waktu yang dibutuhkan oleh berbagai peristiwa seperti *DNS Lookup*, *SSL Handshake*, dan sebagainya.



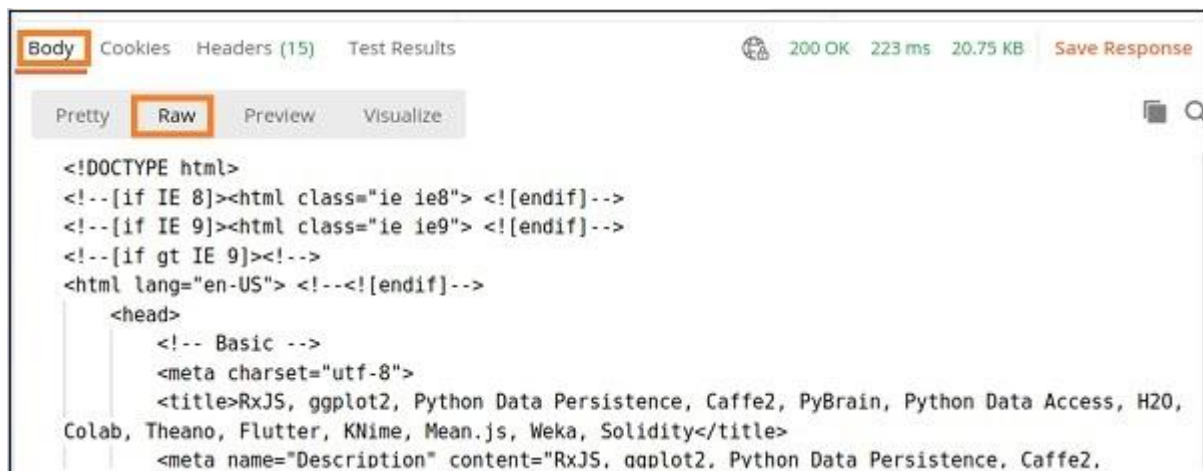
Saat mengarahkan kursor ke ukuran *payload*, detail tentang ukuran respons, *headers*, *Body*, dan sebagainya ditampilkan.



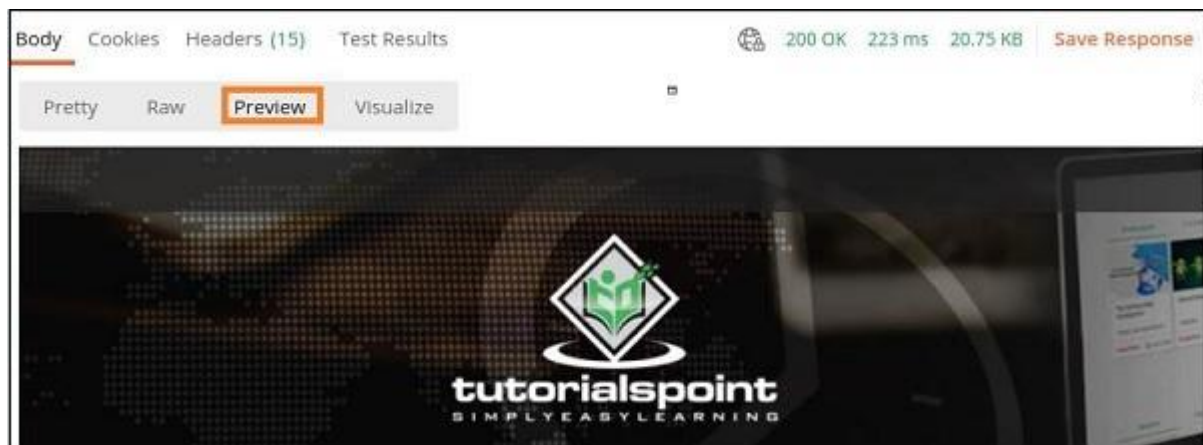
Response Body berisi sub-tab: **Pretty**, **Raw**, dan **Preview**. Format *Pretty* menampilkan pemformatan warna untuk kata kunci dan indentasi agar mudah dibaca. Format *Raw* menampilkan data yang sama seperti di tab *Pretty* tetapi tanpa warna atau indentasi. Tab *Preview* menampilkan pratinjau halaman.



Tab Raw – Tampilan berikut akan muncul.



Tab Preview – Tampilan berikut akan muncul.



Response juga berisi **Cookies, Headers,** dan **Test Results.**

KEY	VALUE	DESCRIPTION
body	Cookies Headers (15) Test Results	200 OK 223 ms 20.75 KB
strict-transport-security	max-age=63072000; includeSubdomains	
Vary	Accept-Encoding	
X-Cache	HIT	
X-Content-Type-Options	nosniff	
X-Frame-Options	SAMEORIGIN	
X-XSS-Protection	1; mode=block	
Content-Length	20753	

Postman - POST Requests

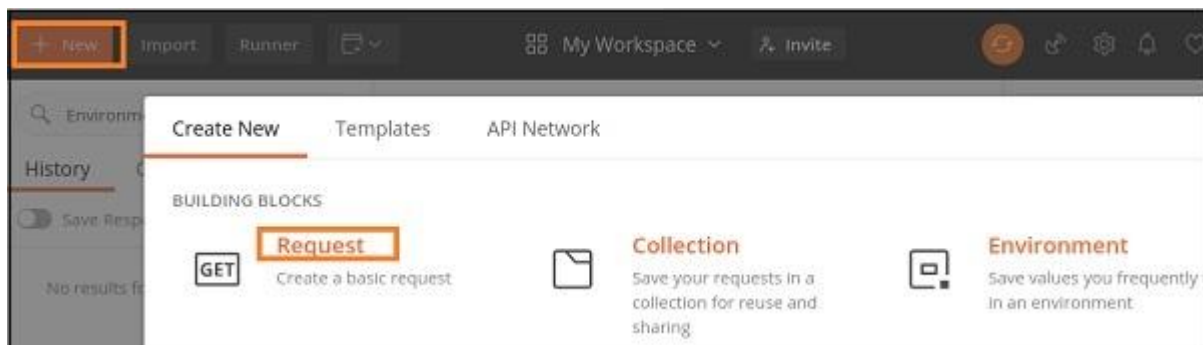
Permintaan POST Postman memungkinkan penambahan data ke *endpoint*. Ini adalah metode yang digunakan untuk menambahkan informasi ke dalam *body* permintaan di server. Biasanya digunakan untuk mengirimkan data yang sensitif.

Setelah kita mengirimkan beberapa *body* permintaan melalui metode POST, API kemudian memberikan informasi tertentu kepada kita dalam Response. Dengan demikian, permintaan POST selalu disertai dengan *body* dalam format yang tepat.

Membuat Permintaan POST

Ikuti langkah-langkah di bawah ini untuk berhasil membuat permintaan POST di Postman:

Langkah 1 – Klik menu **New** dari aplikasi Postman. Munculan *Create New* terbuka. Kemudian klik tautan **Request**.



Langkah 2 – Munculan **SAVE REQUEST** terbuka. Masukkan nama Request, lalu klik Save.

SAVE REQUEST

Requests in Postman are saved in collections (a group of requests).
[Learn more about creating collections](#)

Request name

Test1|

Request description (Optional)

Make things easier for your teammates with a complete request description.

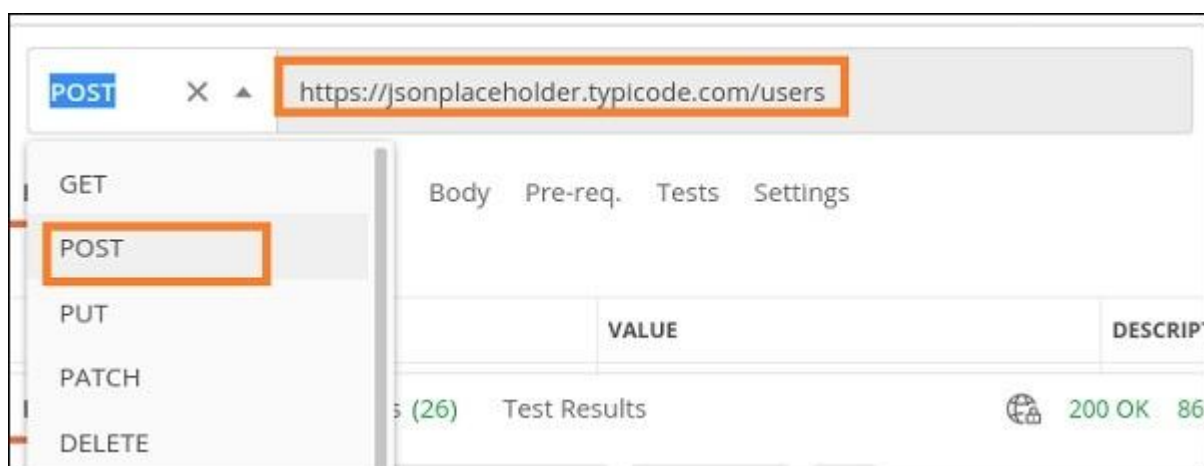
Descriptions support [Markdown](#)

Select a collection or folder to save to:

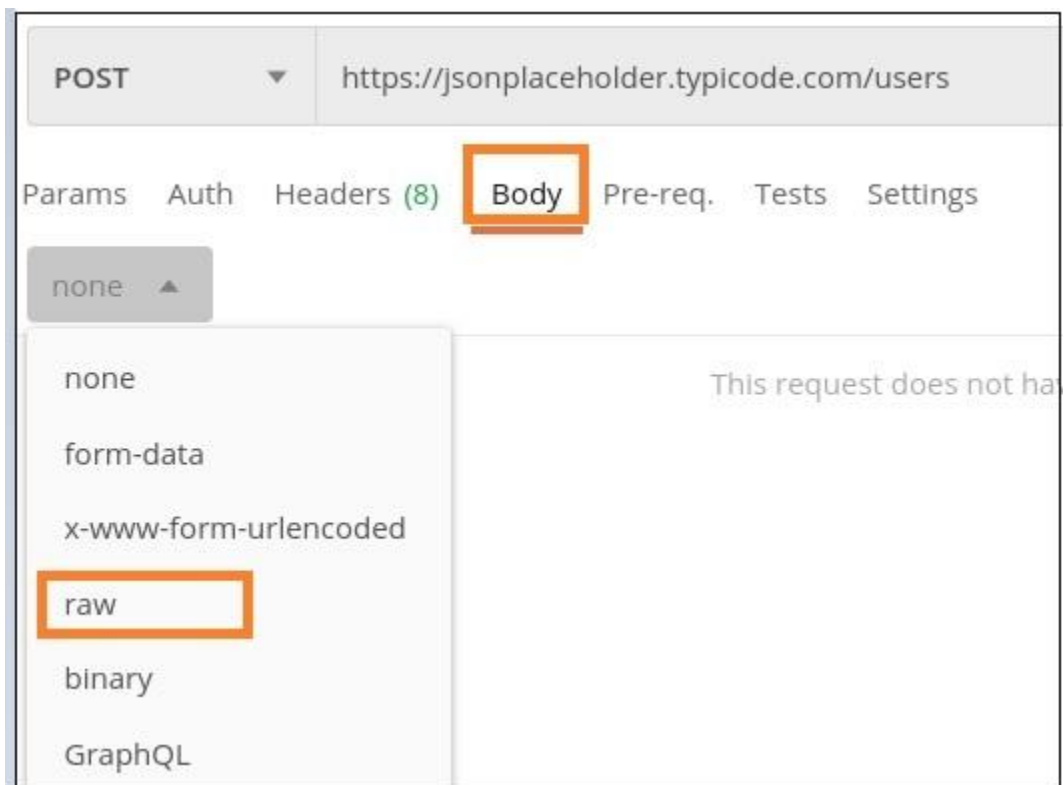
Cancel

Save to FirstTest

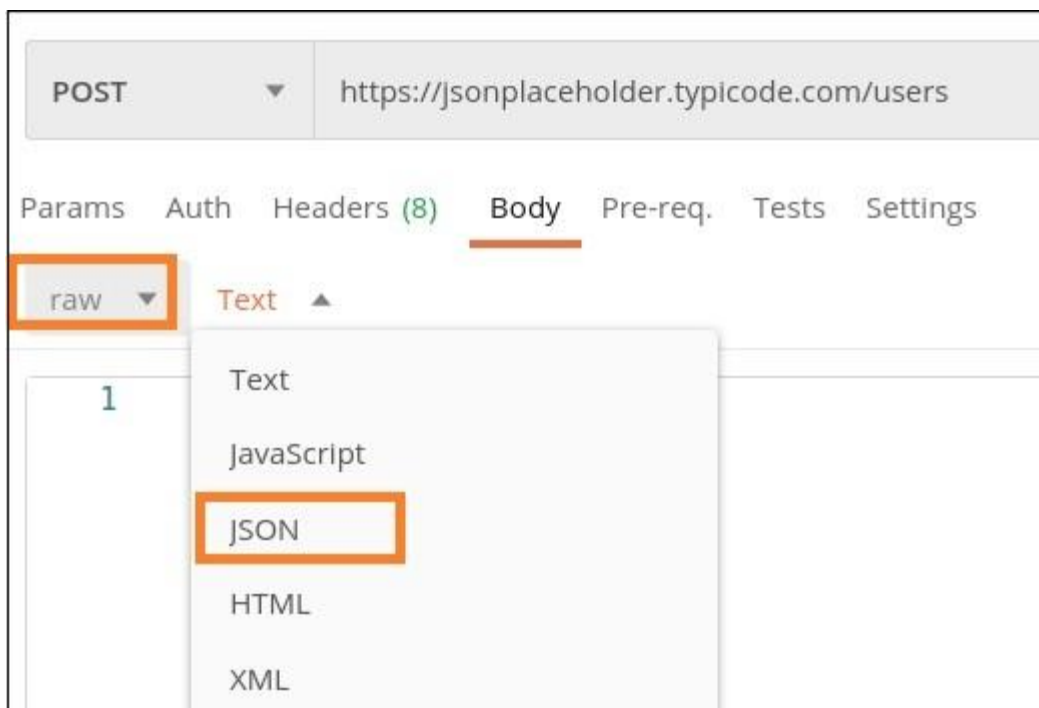
Langkah 3 – Nama Request (Test1) akan muncul di tab Request. Juga, kita pilih opsi **POST** dari dropdown permintaan HTTP. Kemudian, masukkan URL <https://jsonplaceholder.typicode.com/users> di bilah alamat.



Langkah 4 – Pindah ke tab **Body** di bawah bilah alamat dan pilih opsi **raw**.



Langkah 5 – Kemudian, pilih **JSON** dari dropdown *Text*.

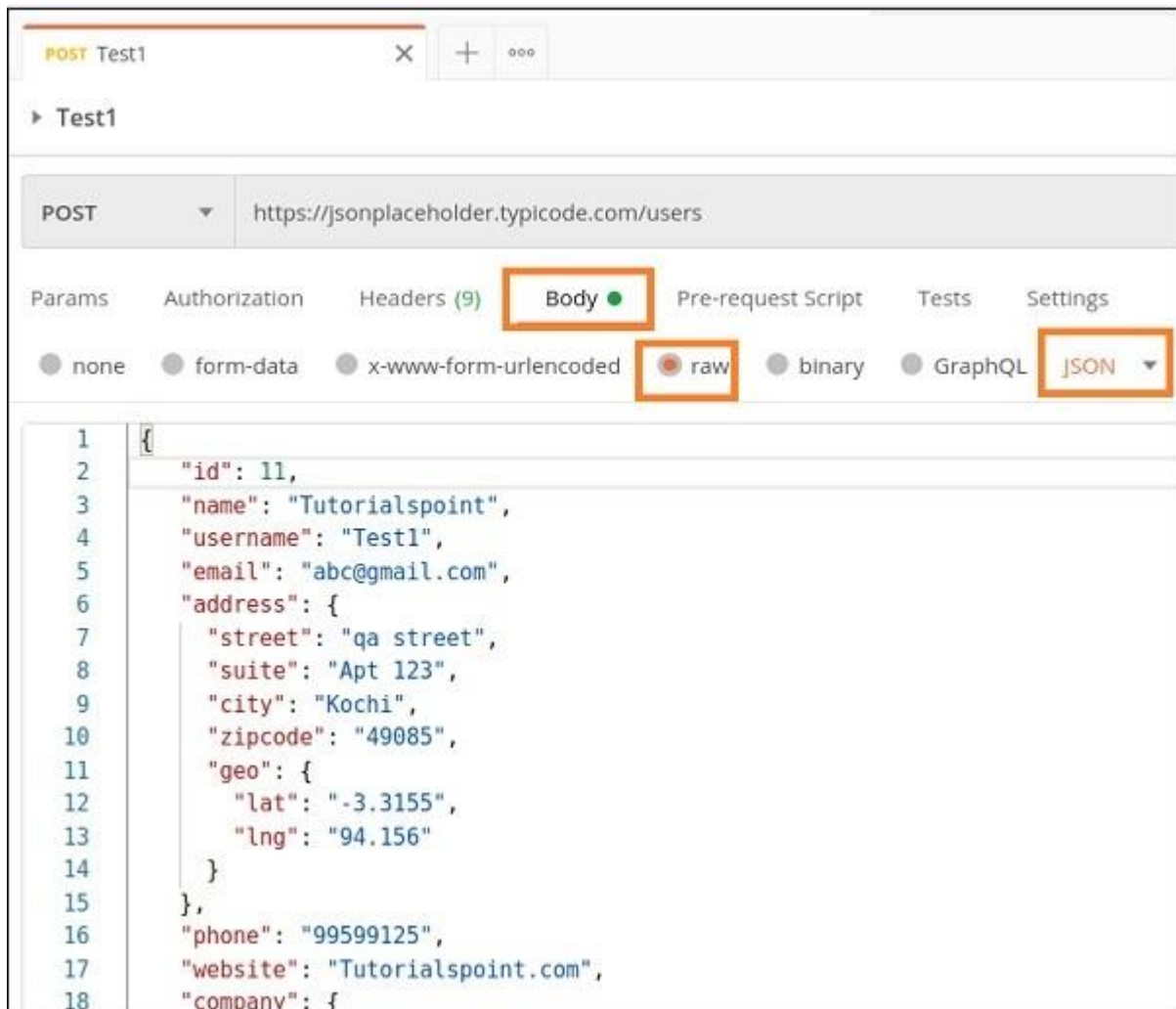


Langkah 6 – Salin dan tempel informasi di bawah ini ke tab Body Postman:

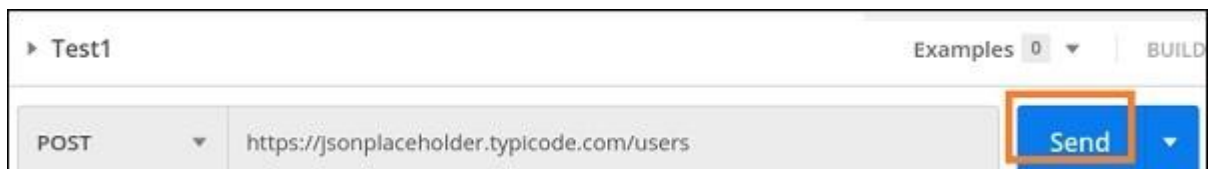
```
{
  "id": 11,
  "name": "Tutorialspoint",
  "username": "Test1",
  "email": "abc@gmail.com",
  "address": {
    "street": "qa street",
    "suite": "Apt 123",
    "city": "Kochi",
    "zipcode": "49085",
    "geo": {
      "lat": "-3.3155",
      "lng": "94.156"
    }
  },
  "phone": "99599125",
  "website": "Tutorialspoint.com",
  "company": {
    "name": "Tutorialspoint",
    "catchPhrase": "Simple Easy Learning",
    "bs": "Postman Tutorial"
  }
}
```

Data di atas yang dikirim melalui metode POST hanya berlaku untuk endpoint: <https://jsonplaceholder.typicode.com/users>.

Untuk meneruskan data dalam format JSON yang benar, kita dapat menggunakan *Jsonformatter* yang tersedia di tautan berikut: <https://jsonformatter.curiousconcept.com/>



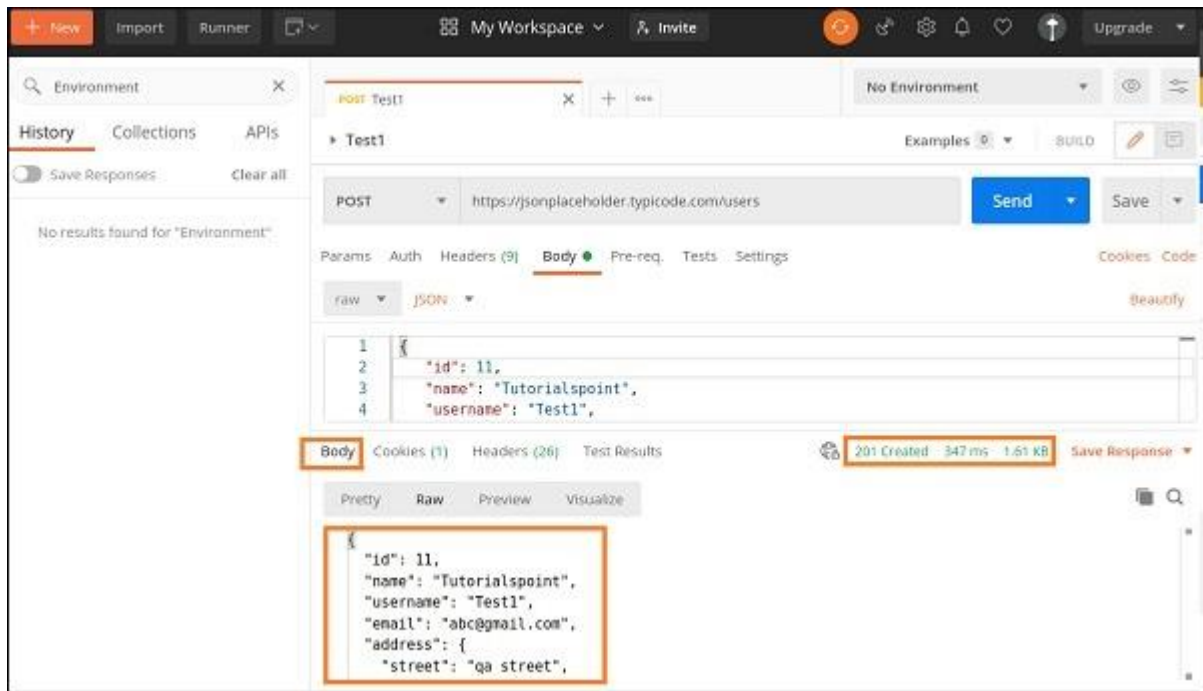
Langkah 7 – Klik tombol Send.



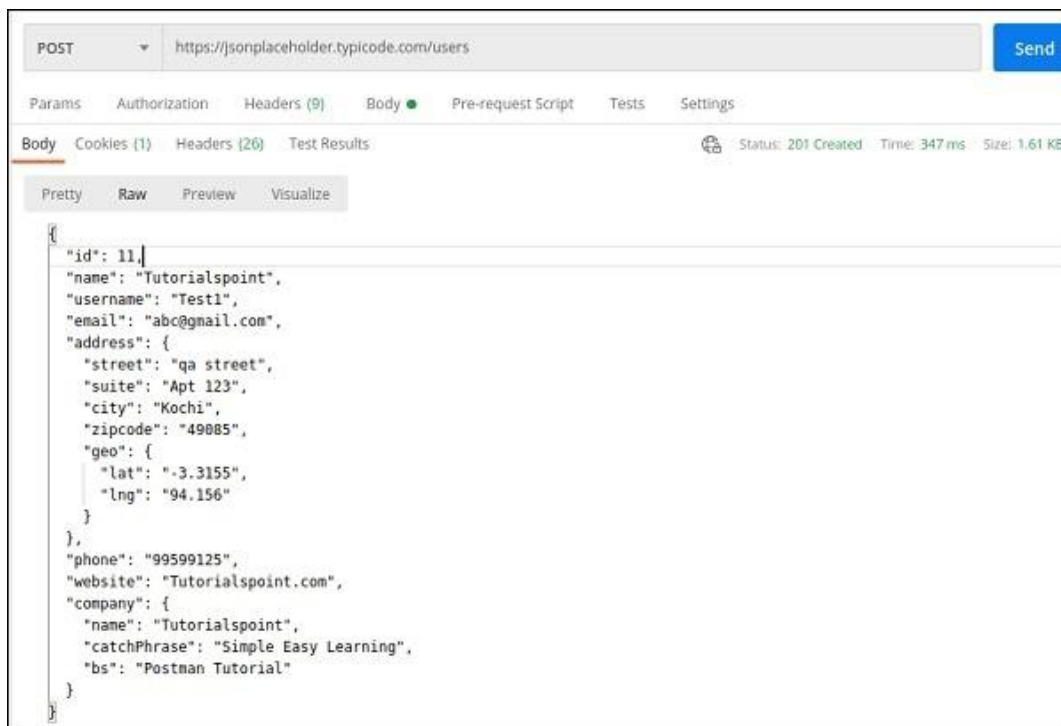
Response

Setelah permintaan dikirim, kita dapat melihat kode respons **201 Created** muncul di Response. Ini menandakan permintaan berhasil dan permintaan yang kita kirim telah diterima oleh server.

Selain itu, informasi tentang waktu yang diperlukan untuk menyelesaikan permintaan (347 ms) dan ukuran *payload* (1.61 KB) juga ditampilkan.



Kita dapat melihat bahwa *Response body* sama dengan *request body* yang telah kita kirim ke server.



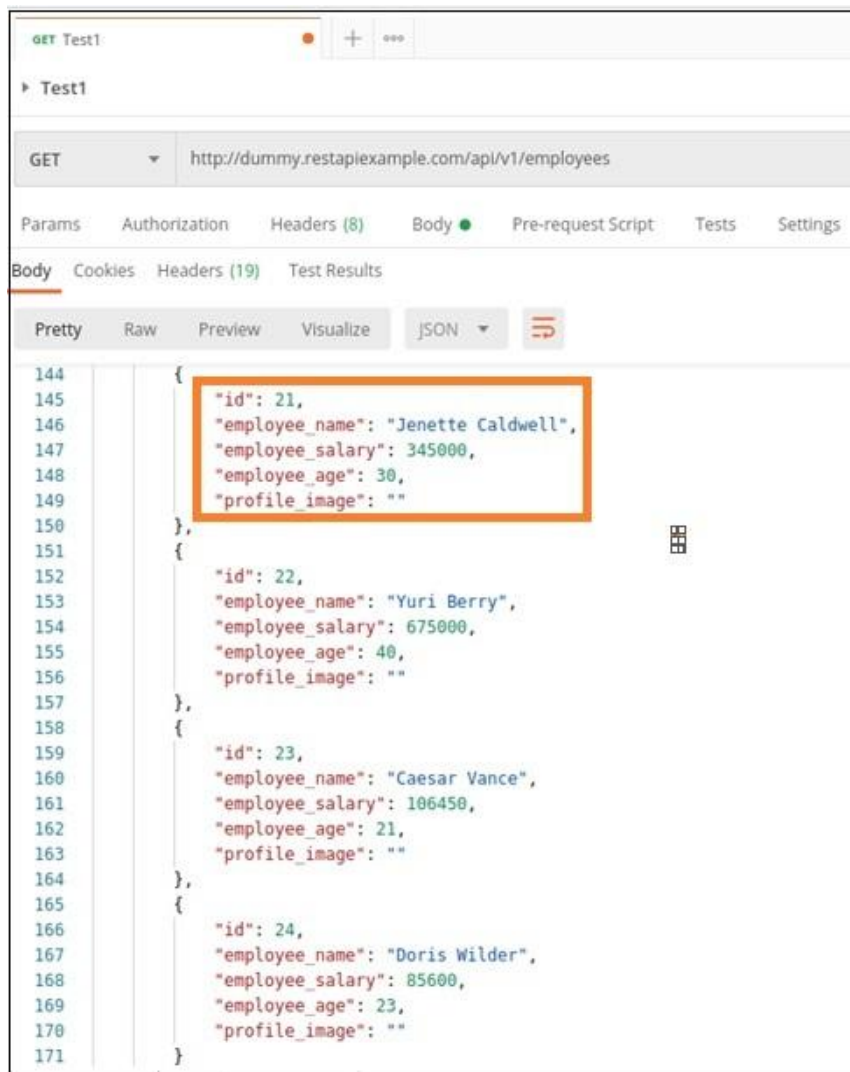
Postman - PUT Requests

Permintaan PUT Postman digunakan untuk mengirimkan data ke server guna membuat atau memodifikasi sumber daya. Perbedaan antara POST dan PUT adalah bahwa permintaan PUT bersifat *idempotent* (tidak mengubah hasil jika dipanggil berulang kali), sedangkan POST tidak.

Ini berarti memanggil permintaan PUT yang sama berkali-kali akan selalu menghasilkan keluaran yang sama. Tetapi memanggil permintaan POST yang sama berkali-kali akan membuat sumber daya serupa lebih dari satu kali.

Sebelum membuat permintaan PUT, pertama-tama kita akan mengirim permintaan GET ke server pada endpoint: `http://dummy.restapiexample.com/api/v1/employees`. Detail tentang cara membuat permintaan GET dijelaskan dalam bab *Postman GET Requests*.

Setelah menerapkan metode GET, *Response body* yang diperoleh adalah sebagai berikut:



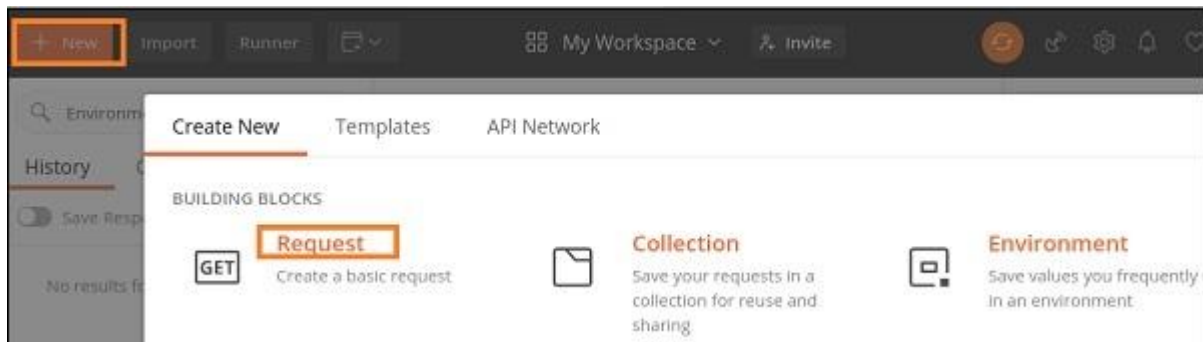
```
144 {
145   "id": 21,
146   "employee_name": "Jenette Caldwell",
147   "employee_salary": 345000,
148   "employee_age": 30,
149   "profile_image": ""
150 },
151 {
152   "id": 22,
153   "employee_name": "Yuri Berry",
154   "employee_salary": 675000,
155   "employee_age": 40,
156   "profile_image": ""
157 },
158 {
159   "id": 23,
160   "employee_name": "Caesar Vance",
161   "employee_salary": 106450,
162   "employee_age": 21,
163   "profile_image": ""
164 },
165 {
166   "id": 24,
167   "employee_name": "Doris Wilder",
168   "employee_salary": 85600,
169   "employee_age": 23,
170   "profile_image": ""
171 }
```

Sekarang, mari kita perbarui employee_salary dan employee_age untuk id 21 dengan bantuan permintaan PUT.

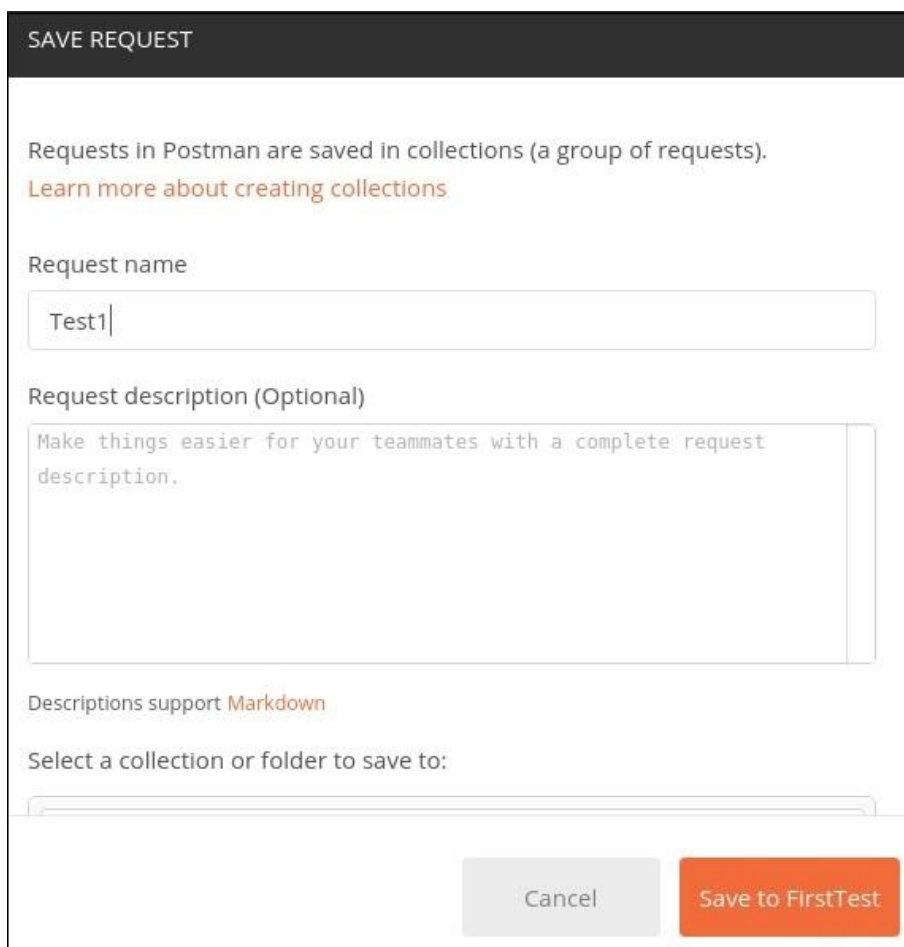
Membuat Permintaan PUT

Ikuti langkah-langkah di bawah ini untuk berhasil membuat permintaan PUT di Postman:

Langkah 1 – Klik menu **New** dari aplikasi Postman. Munculan *Create New* terbuka. Kemudian klik tautan **Request**.



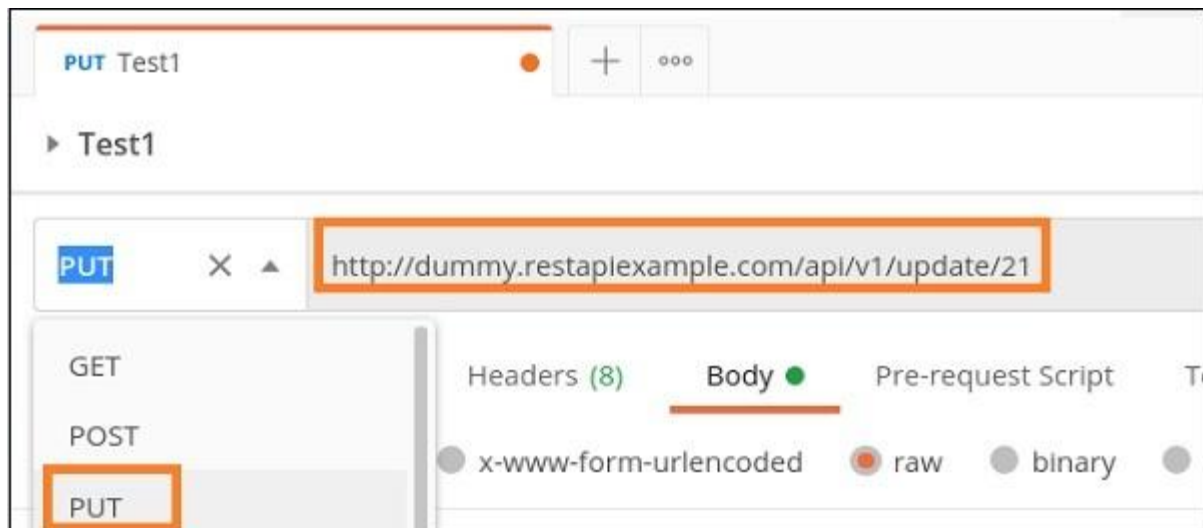
Langkah 2 – Munculan *SAVE REQUEST* terbuka. Masukkan nama Request, lalu klik **Save**.

A screenshot of the 'SAVE REQUEST' dialog box in Postman. The dialog has a dark header with the text 'SAVE REQUEST'. Below the header, there is a text area for 'Request name' containing 'Test1'. Below that is a text area for 'Request description (Optional)' containing 'Make things easier for your teammates with a complete request description.'. Below the description area, there is a note 'Descriptions support Markdown'. At the bottom, there is a section 'Select a collection or folder to save to:' with a dropdown menu. At the very bottom, there are two buttons: 'Cancel' and 'Save to FirstTest' (highlighted with an orange background).

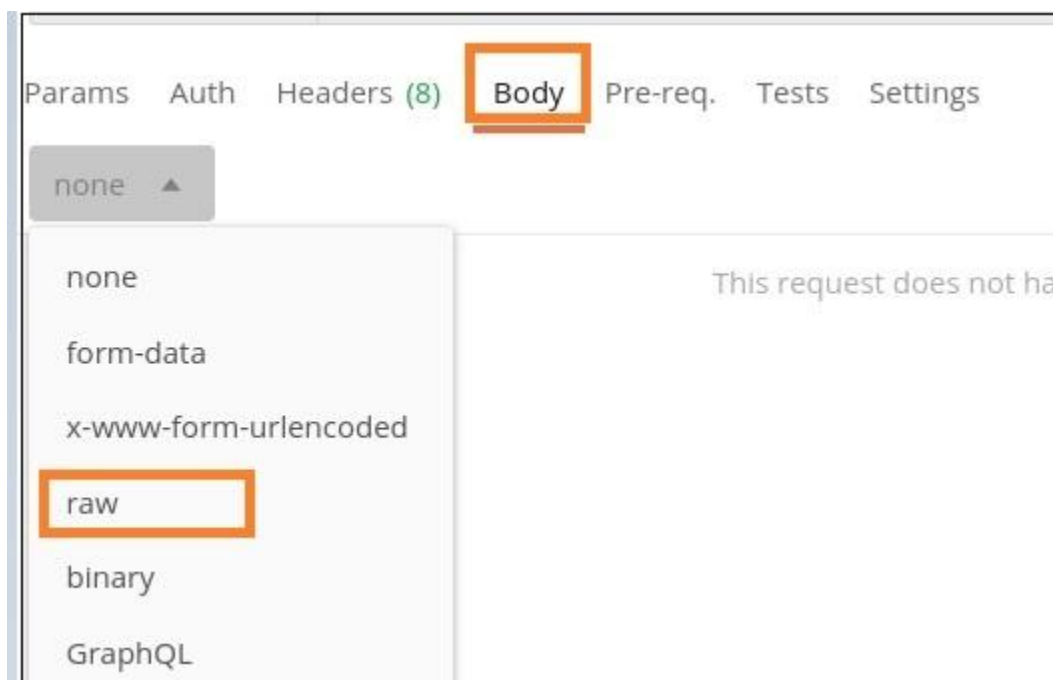
Langkah 3 – Nama Request (Test1) akan muncul di tab Request. Kita pilih opsi **PUT** dari dropdown permintaan HTTP. Kemudian masukkan URL

- <http://dummy.restapiexample.com/api/v1/update/21> (endpoint untuk memperbarui catatan dengan id 21) di bilah alamat.

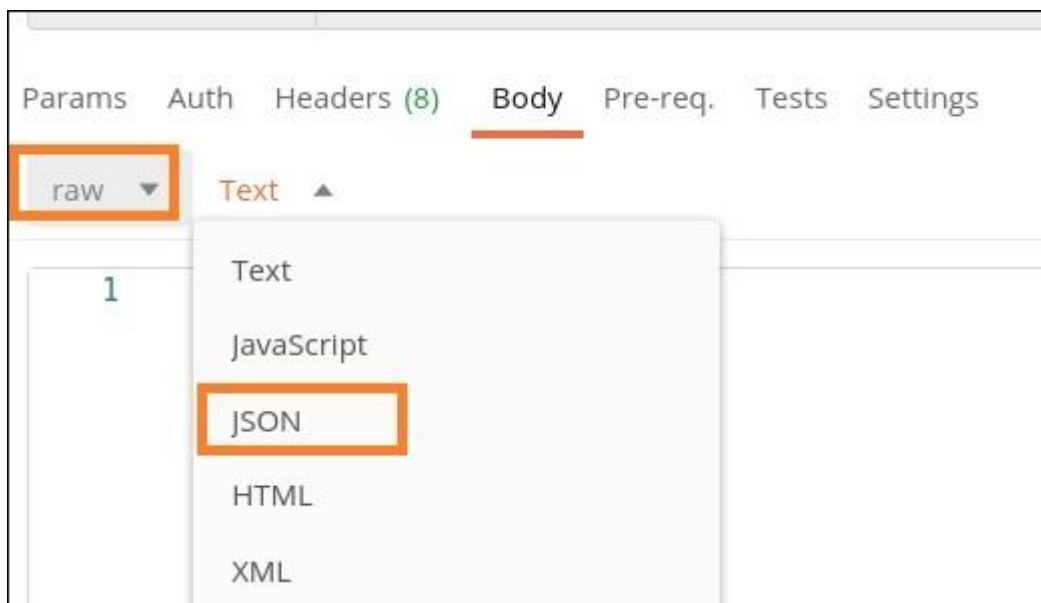
Perlu dicatat bahwa dalam permintaan PUT, kita harus menyebutkan id dari sumber daya di server yang ingin kita perbarui di URL. Sebagai contoh, pada URL di atas kita menambahkan id 21.



Langkah 4 – Pindah ke tab **Body** di bawah bilah alamat dan pilih opsi **raw**.



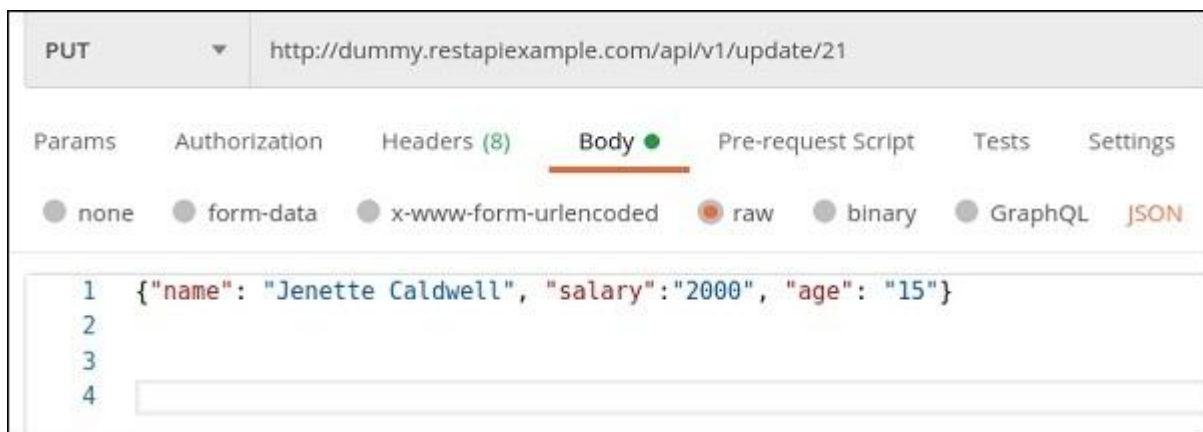
Langkah 5 – Kemudian, pilih **JSON** dari dropdown *Text*.



Langkah 6 – Salin dan tempel informasi di bawah ini ke tab Body Postman:

```
{ "name": "Jenette Caldwell", "salary": "2000", "age": "15" }
```

Parameter keseluruhan yang harus diatur untuk permintaan PUT ditunjukkan di bawah ini.



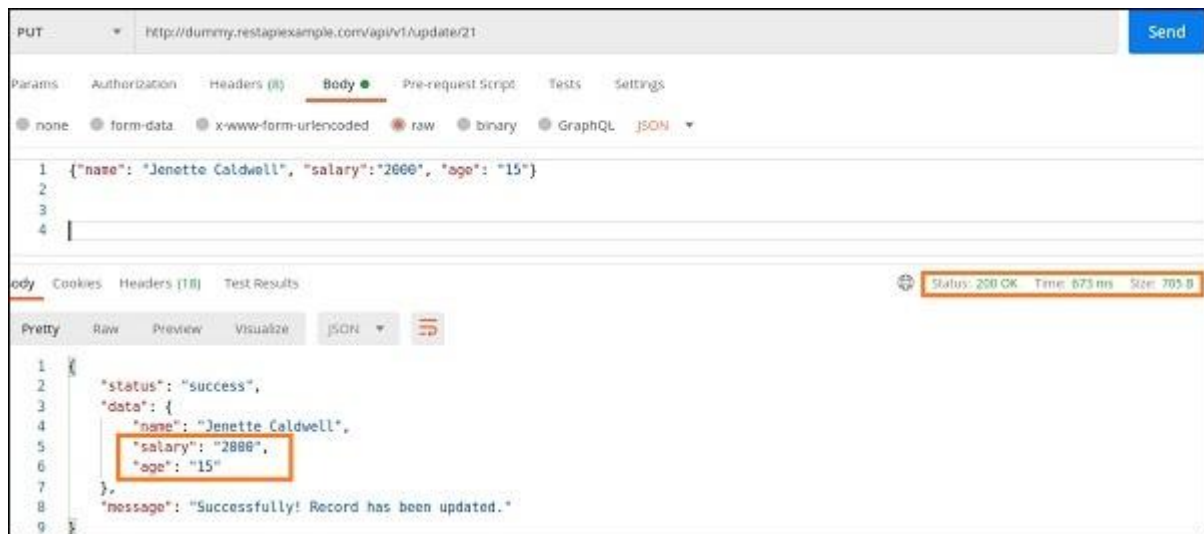
Langkah 7 – Klik tombol **Send**.

Response

Setelah permintaan dikirim, kita dapat melihat kode respons **200 OK** muncul di *Response body*. Ini menandakan permintaan berhasil dan permintaan yang kita kirim telah diterima oleh server.

Selain itu, informasi tentang waktu yang diperlukan untuk menyelesaikan permintaan (673 ms) dan ukuran *payload* (705 B) ditampilkan. *Response body* menunjukkan bahwa

gaji dan usia telah diperbarui masing-masing menjadi 2000 dan 15 untuk karyawan dengan id 21.

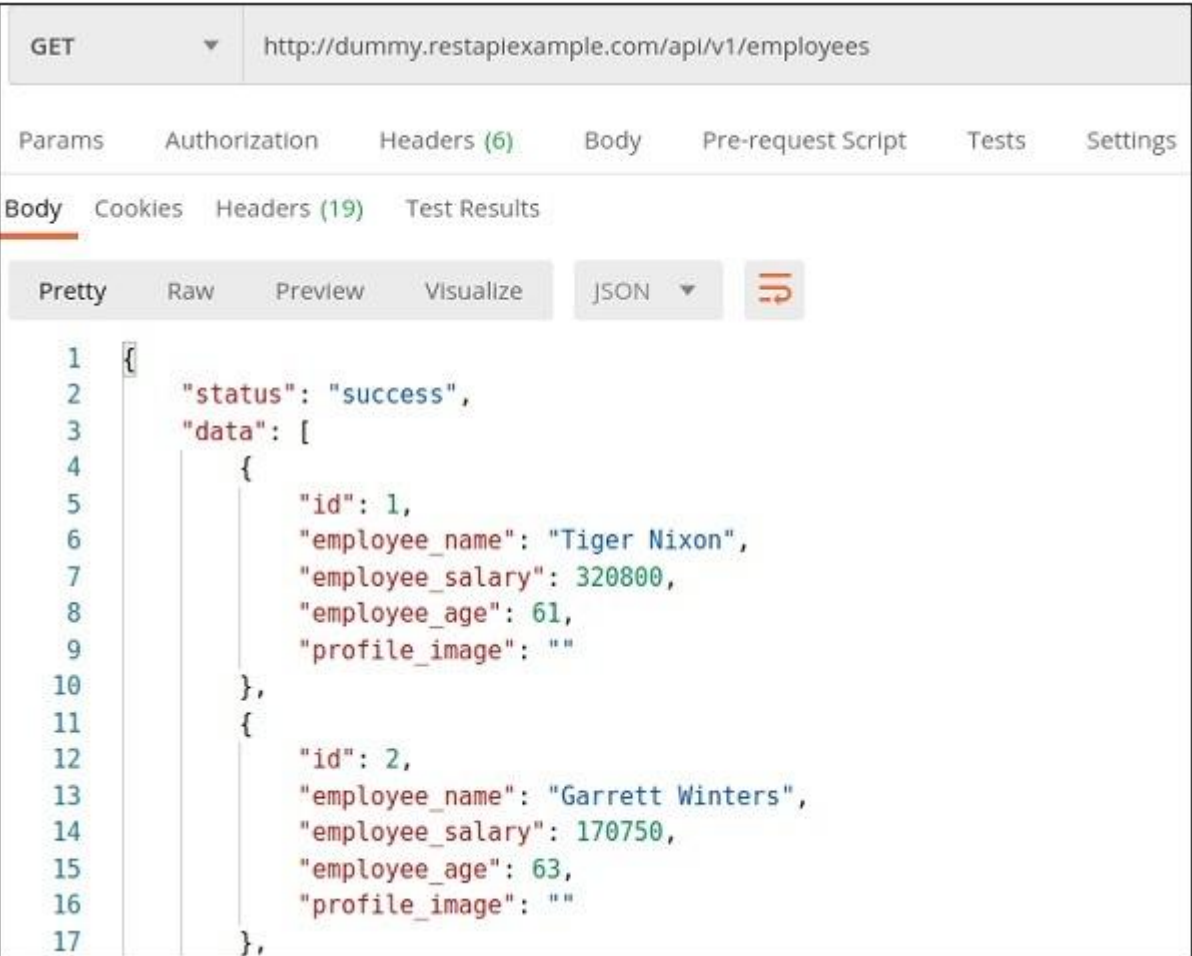


Postman - DELETE Requests

Permintaan DELETE Postman menghapus sumber daya yang sudah ada di server. Metode DELETE mengirimkan permintaan ke server untuk menghapus permintaan yang disebutkan di *endpoint*. Dengan demikian, metode ini mampu memperbarui data di server.

Sebelum membuat permintaan DELETE, pertama-tama kita akan mengirim permintaan GET ke server pada *endpoint*: <https://dummy.restapiexample.com/api/v1/employees>. Detail tentang cara membuat permintaan GET dijelaskan dalam bab *Postman GET Requests*.

Setelah menerapkan metode GET, *Response Body* di bawah ini diperoleh:

A screenshot of the Postman application interface. At the top, the request method is set to 'GET' and the URL is 'http://dummy.restapiexample.com/api/v1/employees'. Below the URL bar, there are tabs for 'Params', 'Authorization', 'Headers (6)', 'Body', 'Pre-request Script', 'Tests', and 'Settings'. The 'Body' tab is selected, and within it, the 'Test Results' sub-tab is active. The response body is displayed in a 'Pretty' JSON format, showing a successful status and a list of two employee records. The first employee has an ID of 1, name 'Tiger Nixon', salary 320800, and age 61. The second employee has an ID of 2, name 'Garrett Winters', salary 170750, and age 63. The interface includes a line number indicator on the left side of the JSON text.

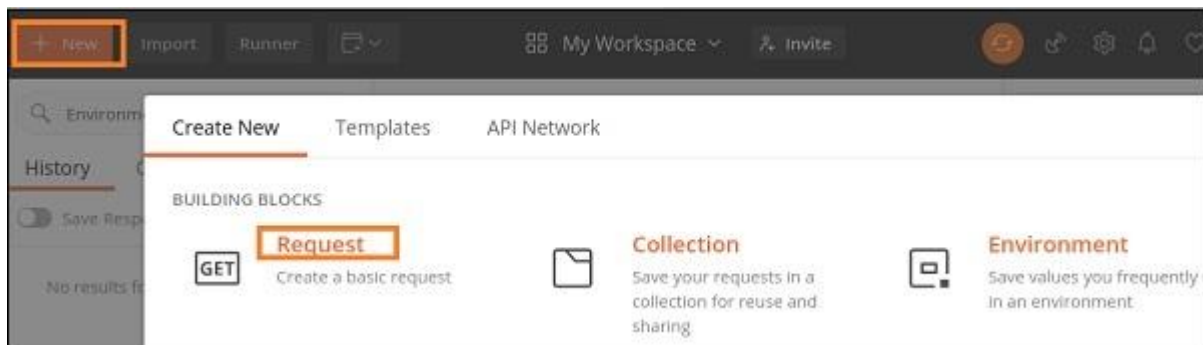
```
1  {
2    "status": "success",
3    "data": [
4      {
5        "id": 1,
6        "employee_name": "Tiger Nixon",
7        "employee_salary": 320800,
8        "employee_age": 61,
9        "profile_image": ""
10     },
11     {
12       "id": 2,
13       "employee_name": "Garrett Winters",
14       "employee_salary": 170750,
15       "employee_age": 63,
16       "profile_image": ""
17     }
18   ]
19 }
```

Mari kita hapus catatan dengan id 2 dari server.

Membuat Permintaan DELETE

Ikuti langkah-langkah di bawah ini untuk berhasil membuat permintaan DELETE di Postman:

Langkah 1 – Klik menu **New** dari aplikasi Postman. Munculan *Create New* terbuka. Kemudian klik tautan **Request**.



Langkah 2 – Munculan *SAVE REQUEST* terbuka. Masukkan nama Request, lalu klik **Save**.

SAVE REQUEST

Requests in Postman are saved in collections (a group of requests).
[Learn more about creating collections](#)

Request name

Request description (Optional)

Make things easier for your teammates with a complete request description.

Descriptions support [Markdown](#)

Select a collection or folder to save to:

Cancel

Save to FirstTest

Langkah 3 – Nama Request (Test1) akan muncul di tab Request. Kita pilih opsi **DELETE** dari dropdown permintaan HTTP. Kemudian masukkan URL - `http://dummy.restapiexample.com/api/v1/delete/2` (*endpoint* untuk menghapus catatan dengan id 2) di bilah alamat.

Di sini, dalam permintaan DELETE, kita telah menyebutkan id dari sumber daya di server yang ingin kita hapus di URL.

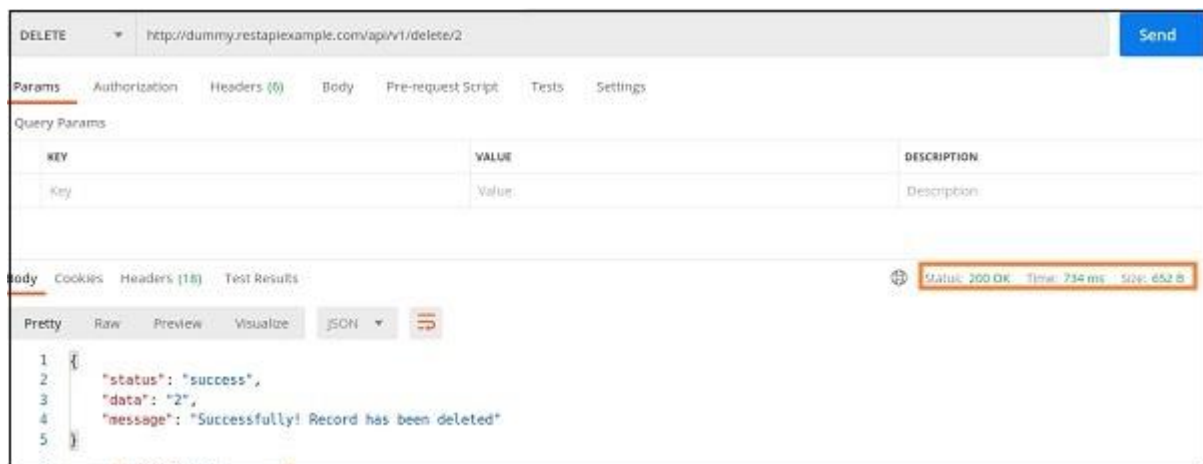


Langkah 4 – Klik tombol Send.

Respons

Setelah permintaan dikirim, kita dapat melihat kode respons **200 OK** muncul di Response. Ini menandakan permintaan berhasil dan permintaan yang kita kirim telah diterima oleh server.

Selain itu, informasi tentang waktu yang diperlukan untuk menyelesaikan permintaan (734 ms) dan ukuran *payload* (652 B) ditampilkan. Response menunjukkan status *success*. Catatan dengan id 2 berhasil dihapus dari server.



Setelah penghapusan catatan dengan id 2, jika kita menjalankan permintaan GET pada *endpoint*: `http://dummy.restapiexample.com/api/v1/employee/2`, kita akan menerima kode status **401 Unauthorized**.

Postman - Create Tests for CRUD

CRUD adalah singkatan dari **Create** (Membuat), **Retrieve** (Mengambil), **Update** (Memperbarui), dan **Delete** (Menghapus) operasi pada situs web yang dibuka di browser. Setiap kali kita meluncurkan aplikasi, operasi *retrieve* dilakukan.

Saat membuat data, misalnya menambahkan pengguna baru untuk situs web, operasi *create* dilakukan. Jika kita memodifikasi informasi, misalnya mengubah detail pelanggan yang sudah ada di situs web, operasi *update* dilakukan. Terakhir, untuk menghilangkan informasi, misalnya menghapus pengguna di situs web, operasi *delete* dilakukan.

Untuk mengambil sumber daya dari server, metode HTTP **GET** digunakan (dibahas dalam bab *Postman GET Requests*). Untuk membuat sumber daya di server, metode HTTP **POST** digunakan (dibahas dalam bab *Postman POST Requests*).

Untuk memodifikasi sumber daya di server, metode HTTP **PUT** digunakan (dibahas dalam bab *Postman PUT Requests*). Untuk menghapus sumber daya di server, metode HTTP **DELETE** digunakan (dibahas dalam bab *Postman DELETE Requests*).

Tes di Postman

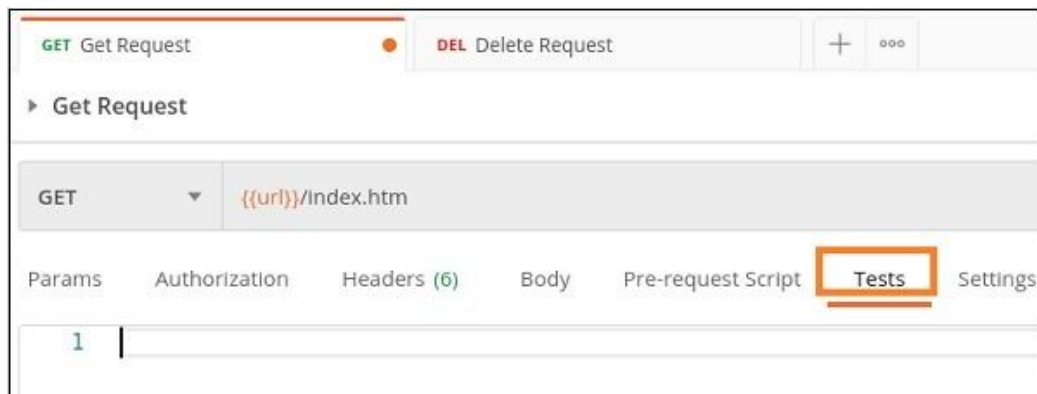
Sebuah tes Postman hanya dieksekusi jika permintaan berhasil. Jika *Response Body* tidak dihasilkan, itu berarti permintaan kita salah dan kita tidak akan dapat mengeksekusi tes apa pun untuk memvalidasi Response.

Di Postman, tes dikembangkan dalam JavaScript dan dapat dikembangkan menggunakan metode JavaScript dan metode Fungsional. Kedua teknik tersebut didasarkan pada bahasa JavaScript.

Metode JavaScript

Ikuti langkah-langkah di bawah ini untuk mengembangkan tes dengan metode JavaScript:

Langkah 1 – Tes yang dikembangkan dengan metode JavaScript ditulis di dalam tab **Tests** di bawah bilah alamat.



Langkah 2 – Tambahkan verifikasi JavaScript berikut di dalam tab Tests:

```
tests["Status Code should be 200"] = responseCode.code === 200
tests["Response time lesser than 10ms"] = responseTime<10
```

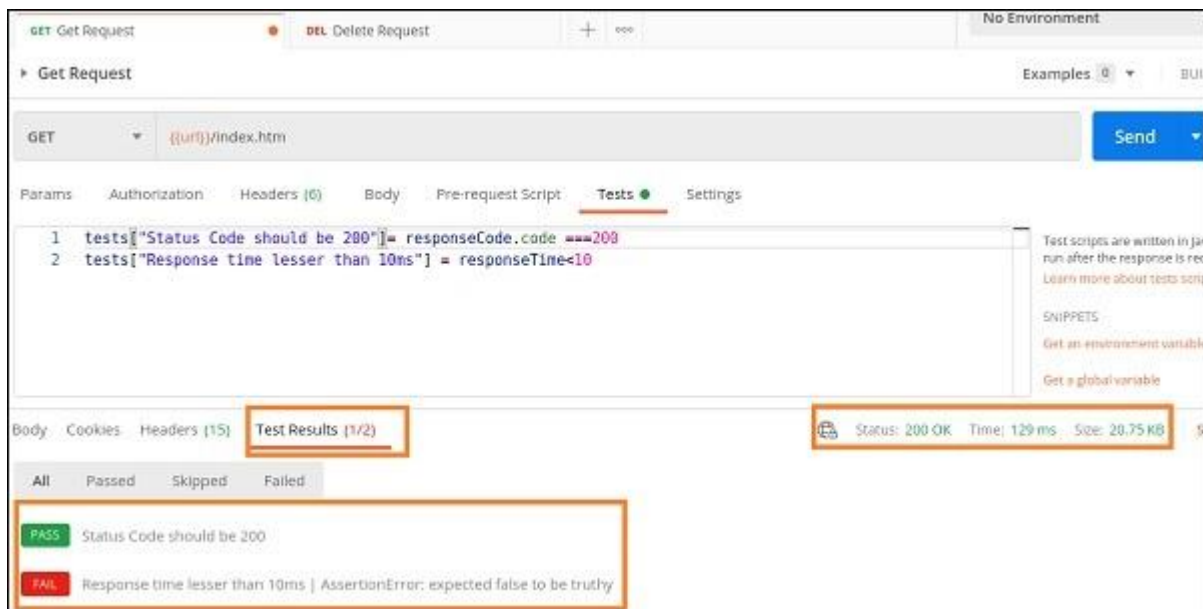
Kita dapat menambahkan satu atau lebih dari satu tes untuk permintaan tertentu.

Di sini, tests adalah variabel bertipe array yang dapat menampung tipe data (integer, string, Boolean, dll.). *Status Code should be 200* dan *Response time lesser than 10ms* adalah nama-nama tes. Disarankan untuk memberikan nama tes yang bermakna.

responseCode.code adalah kode respons yang diperoleh dalam Response dan responseTime adalah waktu yang dibutuhkan untuk mendapatkan Response.

Langkah 3 – Pilih metode **GET** dan masukkan *endpoint*, lalu klik **Send**.

Response – Di bagian Response, klik tab **Test Results**.



Tab *Test Results* menunjukkan tes yang **lulus** berwarna hijau dan tes yang **gagal** berwarna merah. *Test Results (1/2)* berarti satu dari dua tes telah lulus.

Response menunjukkan status **200 OK** dan waktu respons **129ms** (tes kedua memeriksa apakah waktu respons kurang dari 10ms). Maka, tes pertama lulus dan tes kedua gagal beserta *Assertion error*.

Metode Fungsional (Functional Method)

Ikuti langkah-langkah di bawah ini untuk mengembangkan tes dengan metode fungsional:

Langkah 1 – Tes yang dikembangkan dengan metode fungsional ditulis di dalam tab **Tests** di bawah bilah alamat.

Langkah 2 – Tambahkan kode berikut di dalam tab Tests:

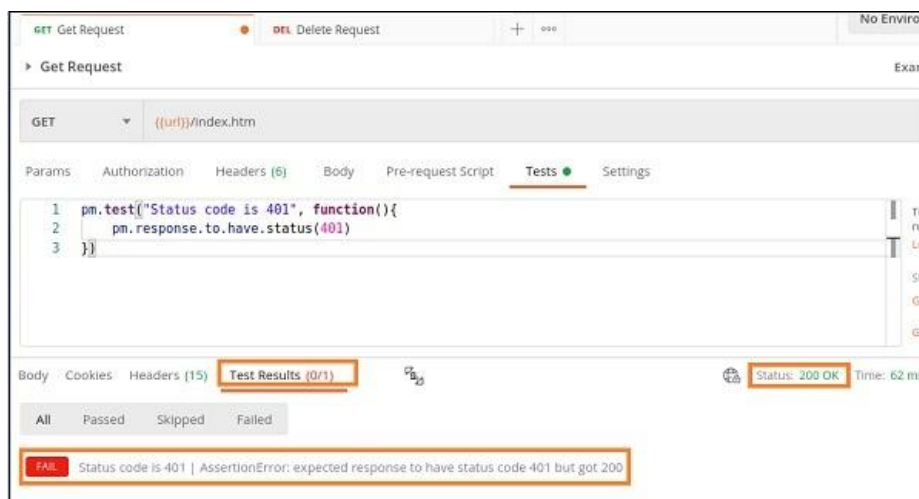
```
pm.test("Status Code is 401", function(){
  pm.response.to.have.status(401)
})
```

Di sini, `pm.test` adalah fungsi untuk tes yang dilakukan. *Status Code is 401* adalah nama tes yang akan terlihat di *Test Result* setelah eksekusi.

`pm.response` digunakan untuk mendapatkan respons dan menambahkan asersi untuk memverifikasi header, kode, status, dan sebagainya.

Langkah 3 – Pilih metode **GET** dan masukkan *endpoint*, lalu klik **Send**.

Response – Di bagian Response, klik tab **Test Results**.



Tab *Test Results* menunjukkan tes berwarna merah karena tes gagal. *Test Results (0/1)* berarti nol dari satu tes telah lulus. Response menunjukkan status **200 OK** (tes memeriksa apakah kode respons adalah 401). Maka tes menunjukkan gagal beserta *Assertion error*.

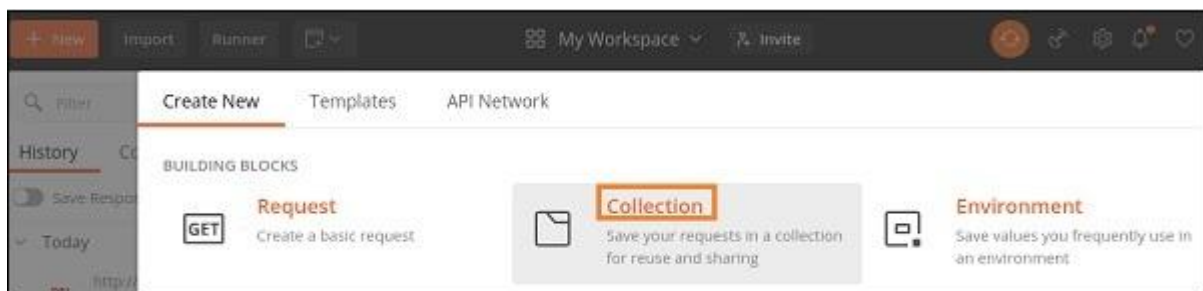
Postman - Create Collections

Sekelompok permintaan yang telah disimpan dan diatur ke dalam folder dikenal sebagai **Koleksi (Collections)**. Ini mirip dengan repositori. Dengan demikian, Koleksi membantu memelihara tes API dan juga membaginya dengan mudah antar tim.

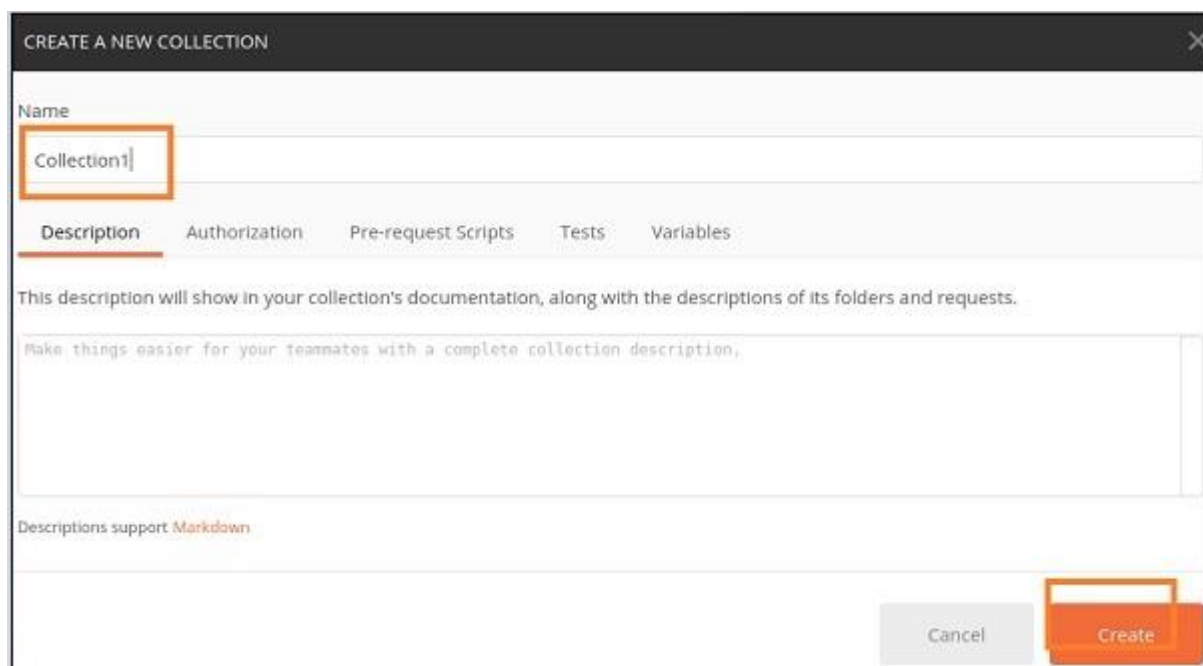
Membuat Koleksi Baru

Ikuti langkah-langkah di bawah ini untuk membuat koleksi baru di Postman:

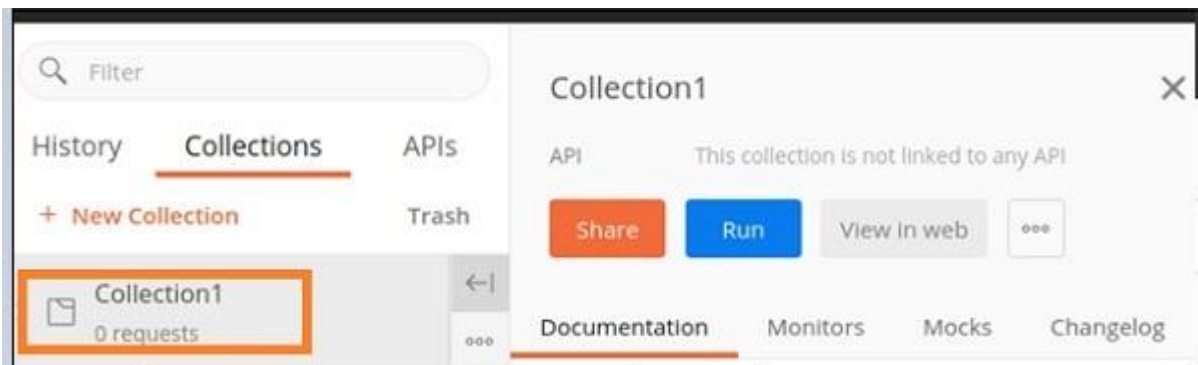
Langkah 1 – Klik ikon **New** dari aplikasi Postman. Munculan *Create New* terbuka. Kemudian klik tautan **Collection**.



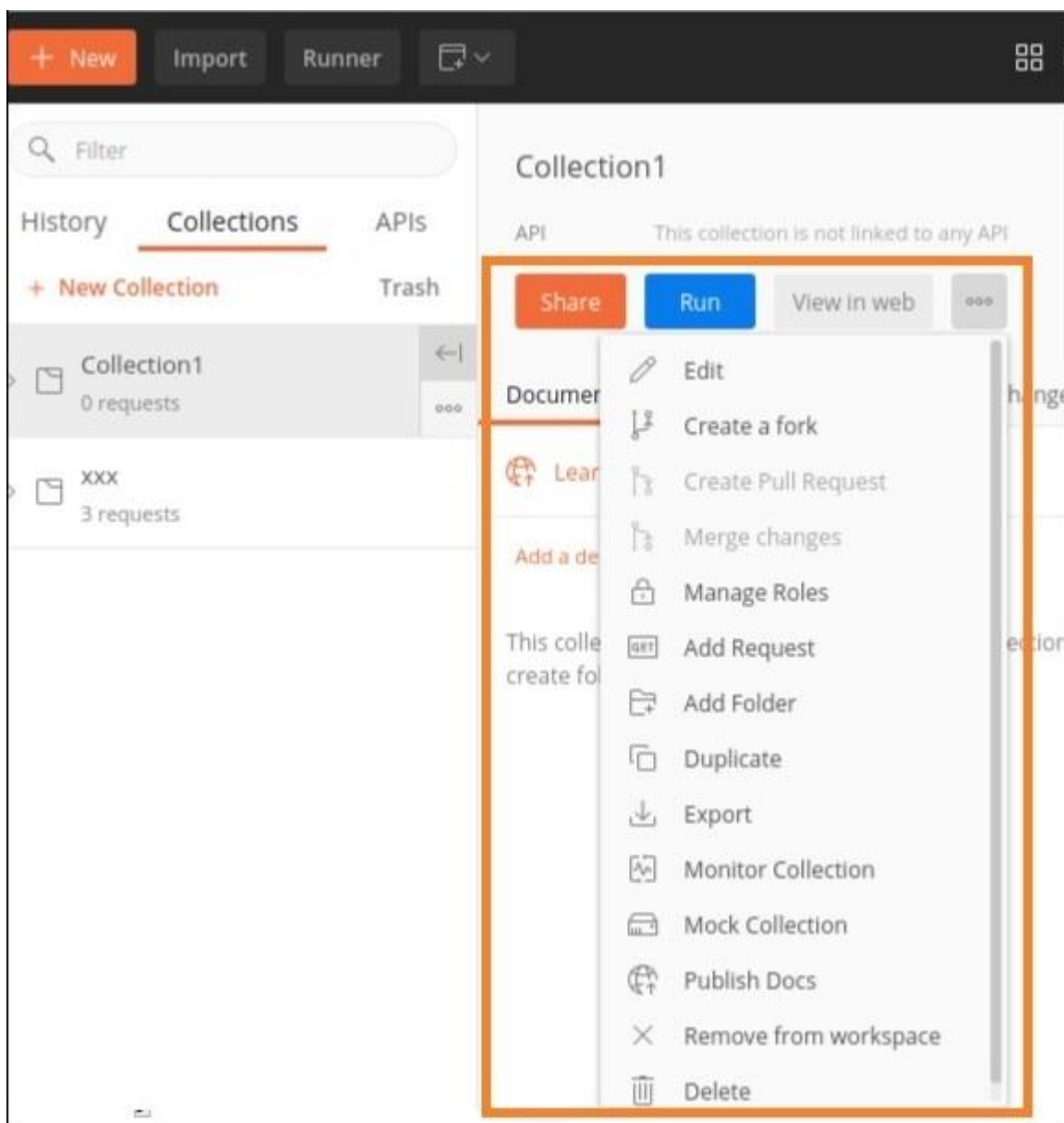
Langkah 2 – Munculan *CREATE A NEW COLLECTION* terbuka. Masukkan **Nama Koleksi** dan klik tombol **Create**.



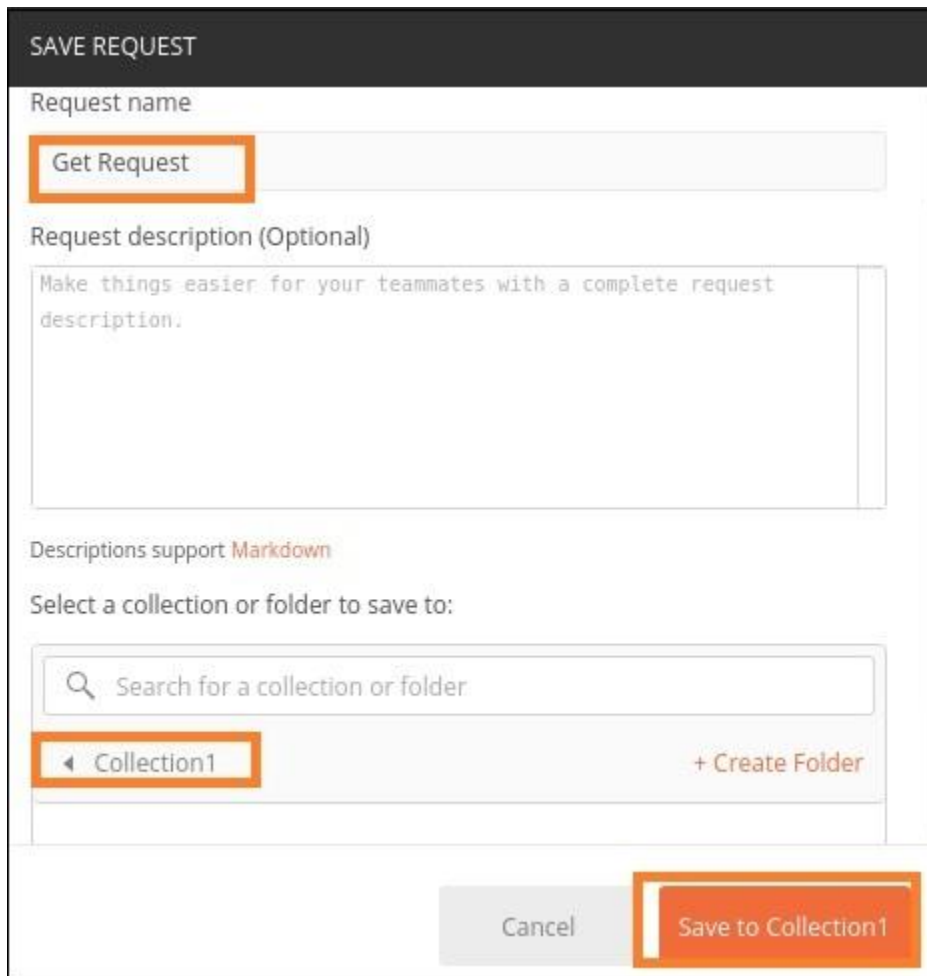
Langkah 3 – Nama Koleksi dan jumlah permintaan yang dikandungnya ditampilkan di sidebar di bawah tab **Collections**.



Langkah 4 – Di sebelah kanan nama Koleksi, tersedia opsi seperti *Share*, *Run*, dan sebagainya. Klik tiga titik untuk mendapatkan opsi lebih lanjut.



Langkah 5 – Klik **Add Request**. Munculan *SAVE REQUEST* terbuka. Masukkan **Nama Request** dan pilih Koleksi yang telah kita buat. Kemudian, klik tombol **Save to Collection1**.



SAVE REQUEST

Request name

Get Request

Request description (Optional)

Make things easier for your teammates with a complete request description.

Descriptions support **Markdown**

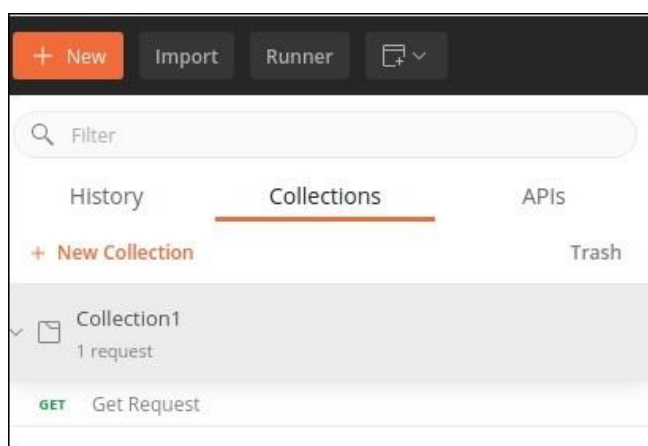
Select a collection or folder to save to:

Search for a collection or folder

Collection1 + Create Folder

Cancel Save to Collection1

Langkah 6 – Koleksi beserta permintaannya ditampilkan di sidebar di bawah tab **Collections**.



Postman - Parameterize Requests

Kita dapat membuat parameter permintaan Postman untuk mengeksekusi permintaan yang sama dengan berbagai kumpulan data. Ini dilakukan dengan bantuan **variabel** beserta **parameter**. Parameter adalah bagian dari URL yang digunakan untuk mengirimkan informasi lebih lanjut ke server.

Data dapat digunakan dalam bentuk **file data** atau **variabel Environment**. Parameterisasi adalah fitur penting Postman dan membantu menghilangkan tes yang berlebihan. Parameter diapit dengan kurung kurawal ganda `{{parameter}}`.

Contoh

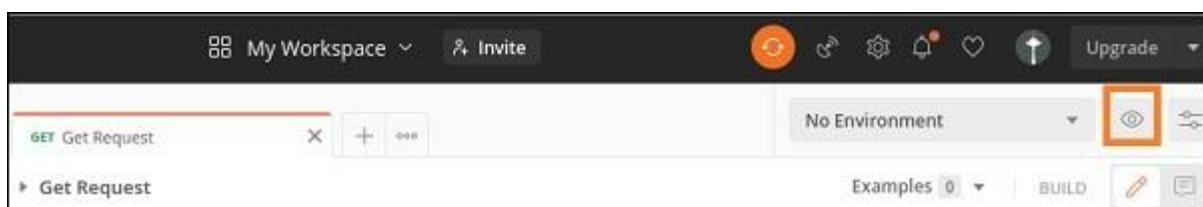
Mari kita ambil contoh URL: `https://www.tutorialspoint.com/index.htm`. Kita akan membuat variabel url lalu menggunakannya untuk parameterisasi permintaan. Kita dapat merujuknya dalam format `{{url}}` di Postman.

Sebuah parameter berbentuk pasangan kunci-nilai. Jadi untuk menunjuk ke URL: `https://www.tutorialspoint.com/index.htm`, kita dapat menyebutnya sebagai `{{url}}/index.htm`. Jadi di sini, url adalah kunci dan nilai yang ditetapkan adalah `https://www.tutorialspoint.com`.

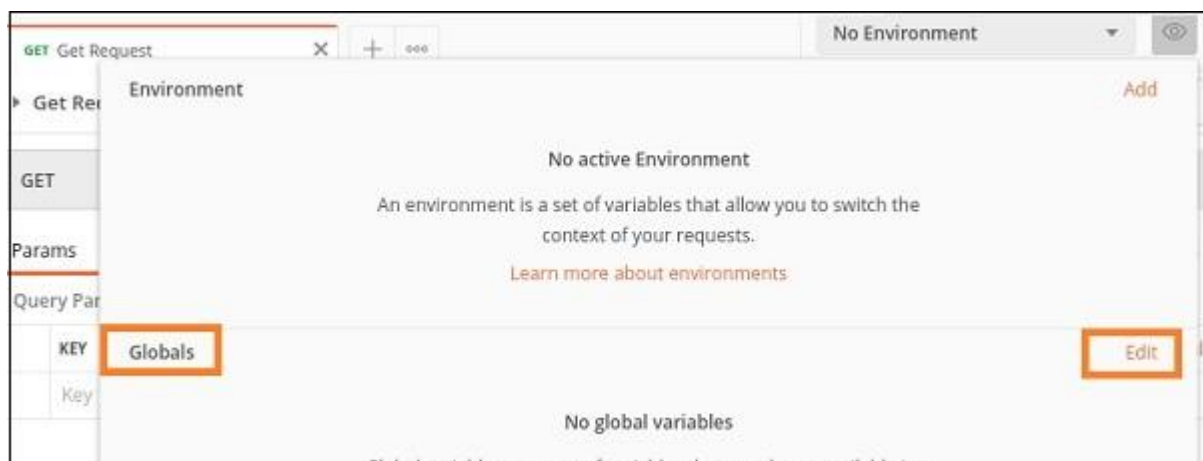
Membuat Permintaan Parameter

Ikuti langkah-langkah di bawah ini untuk membuat permintaan parameter di Postman:

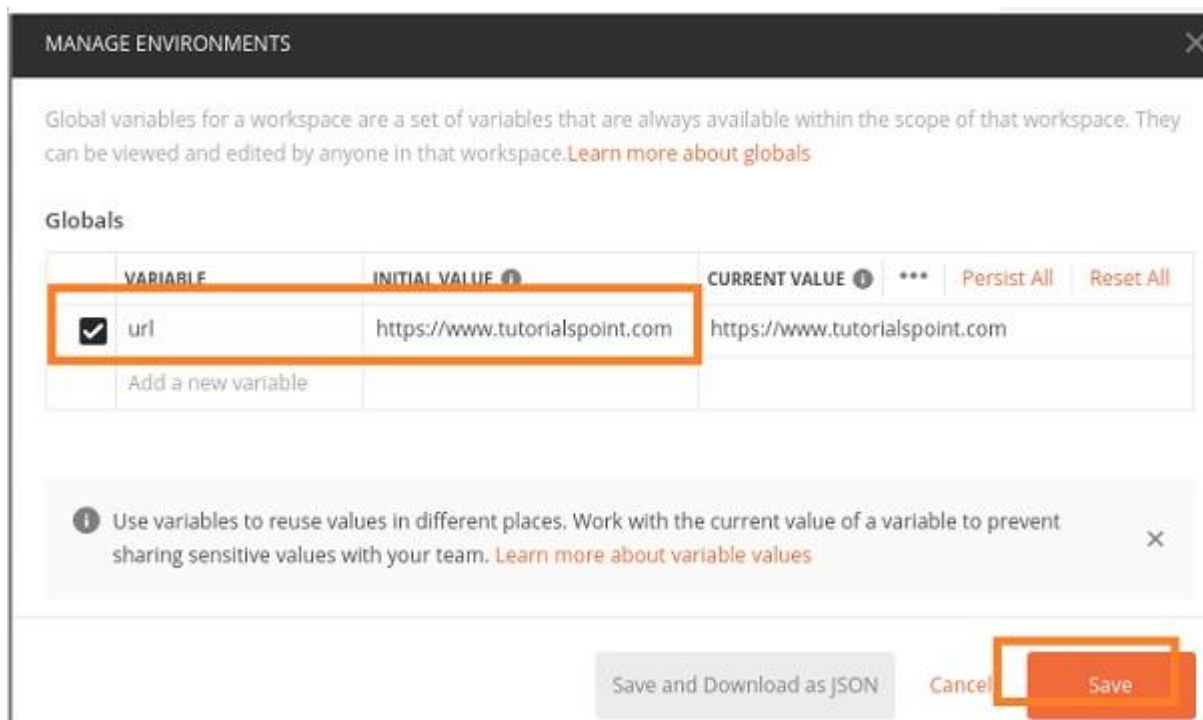
Langkah 1 – Klik ikon mata di sebelah kanan dropdown *Environment* di pojok kanan atas aplikasi Postman.



Langkah 2 – Klik tautan **Edit** di bagian **Globals**.



Langkah 3 – Munculan *MANAGE ENVIRONMENTS* terbuka. Masukkan URL untuk kolom **VARIABLE** dan <https://www.tutorialspoint.com> untuk **INITIAL VALUE**. Kemudian, klik **Save**.



Langkah 4 – Klik **close** untuk beralih ke layar berikutnya.

Langkah 5 – Di tab **Http Request**, masukkan `{{url}}/index.htm` di bilah alamat. Pilih metode **GET** dan klik **Send**.



Response

Setelah permintaan dikirim, kita dapat melihat kode respons **200 OK** muncul di Response. Ini menandakan permintaan berhasil dan *endpoint* benar.

GET Get Request

No Environment

Get Request

Examples 0BUILD

GET{{url}}/index.htmSendSave

ParamsAuthHeaders (6)BodyPre-req. TestsSettingsCookiesCode

Query Params

KEY	VALUE	DESCRIPTION	...	Bulk Edit
Key	Value	Description		

Body

200 OK147 ms20.75 KBSave Response

PrettyRawPreviewVisualizeHTML

```
1 <!DOCTYPE html>
2 <!--[if IE 8]><html class="ie ie8"> <![endif]-->
3 <!--[if IE 9]><html class="ie ie9"> <![endif]-->
4 <!--[if gt IE 9]><!-->
5 <html lang="en-US">
6 <!--<![endif]-->
```

Postman - Collection Runner

Collection Runner Postman digunakan untuk mengeksekusi sebuah Koleksi yang memiliki beberapa permintaan secara bersamaan. Semua permintaan di dalam sebuah Koleksi akan dijalankan secara simultan. Collection Runner tidak menghasilkan *Response Body*.

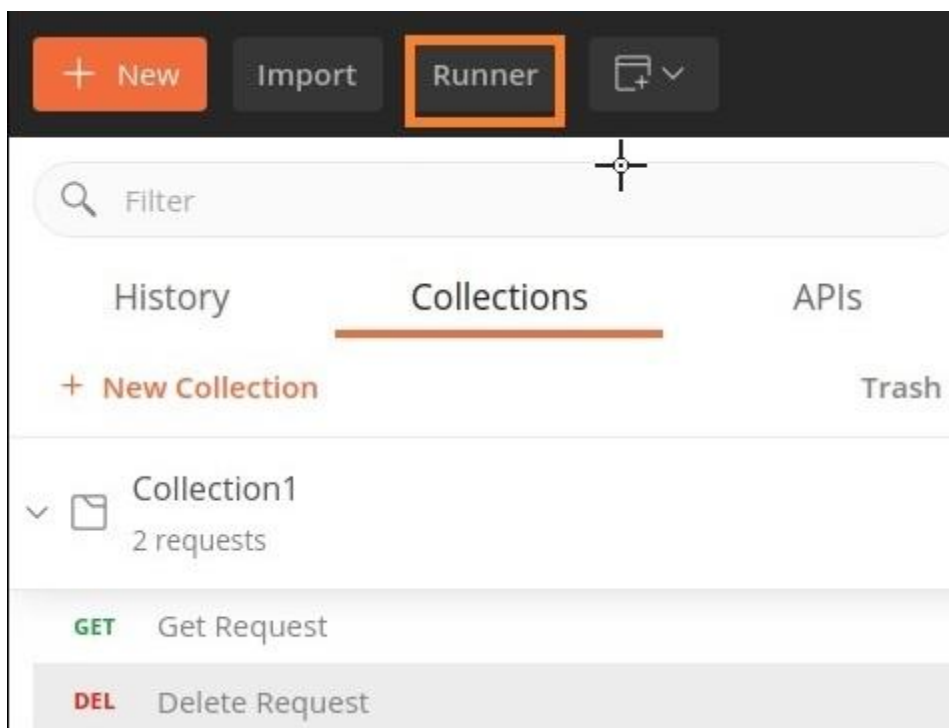
Konsol Collection Runner menampilkan hasil tes untuk setiap permintaan. Wajib memiliki lebih dari satu permintaan di dalam Koleksi untuk dapat menggunakan Collection Runner.

Detail tentang cara membuat Koleksi dibahas dalam bab *Postman Create Collections*.

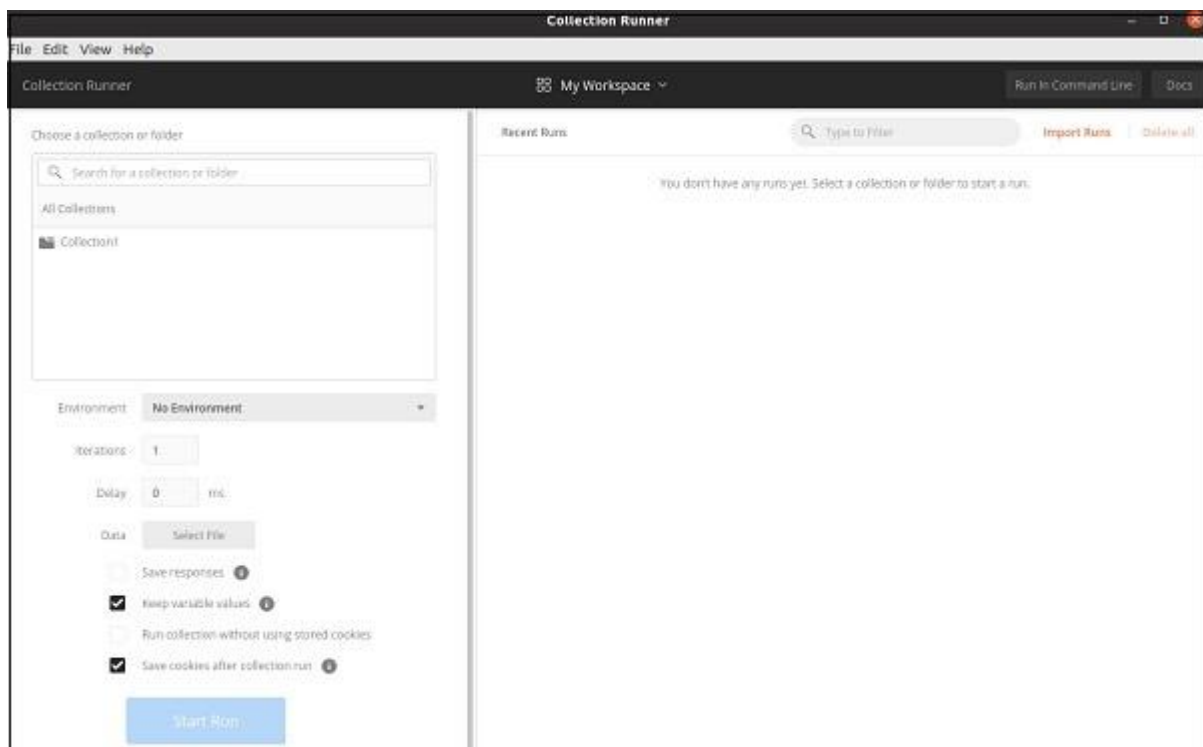
Mengeksekusi Tes dengan Collection Runner

Ikuti langkah-langkah di bawah ini untuk mengeksekusi tes dengan Collection Runner di Postman:

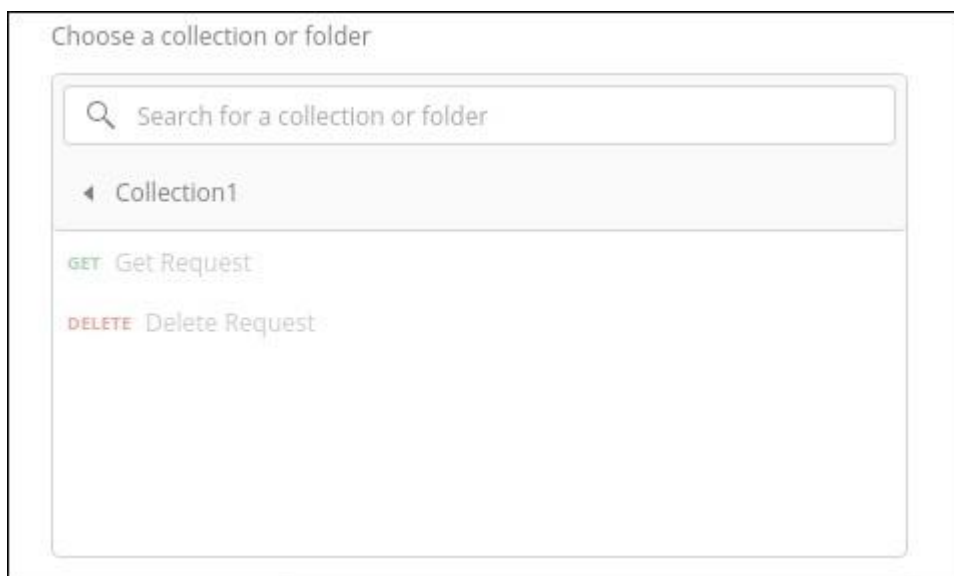
Langkah 1 – Klik menu **Runner** yang ada di bagian atas aplikasi Postman.



Langkah 2 – Layar *Collection Runner* akan muncul.



Langkah 3 – Pilih nama Koleksi dari *Choose a collection or folder*.



Langkah 4 – Pilih lingkungan (*environment*) dari dropdown *Environment* untuk menjalankan permintaan di lingkungan tertentu. Kemudian, tentukan berapa kali kita perlu mengulang permintaan. Kita juga dapat mengatur waktu tunda (*delay*) dalam milidetik untuk permintaan.

Environment: No Environment

Iterations: 2

Delay: 1500 ms

Langkah 5 – Jika kita memiliki data dalam sebuah file, maka kita harus memilih tipe file dari **Data**. Kemudian, klik tombol **Run Collection1**.

Data: Select File

☐ Save responses i

☒ Keep variable values i

☐ Run collection without using stored cookies

☒ Save cookies after collection run i

Run Collection1

Langkah 6 – Halaman *Run Results* akan muncul. Bergantung pada waktu tunda yang diberikan, tes akan dieksekusi.

Hasil tes (Lulus/Gagal) akan ditampilkan untuk setiap iterasi. Status lulus ditampilkan dengan warna hijau dan yang gagal dengan warna merah. Jika tidak ada tes yang diimplementasikan untuk permintaan tertentu, maka akan ditampilkan pesan: *"This request does not have any tests"*.

Ini adalah lingkungan tempat tes dijalankan dan nama-nama Koleksi terlihat di bagian atas Collection Runner. Untuk setiap permintaan, kode status, waktu yang dibutuhkan, ukuran *payload*, dan verifikasi tes juga ditampilkan.

Collection Runner

File Edit View Help

Collection Runner Run Results My Workspace Run in Command Line Docs

4 PASSED 0 FAILED Collection1 No Environment

Run Summary Export Results Retry New

Iteration 1

GET Get Request ([url])index.htm / Get Request 200 OK 117 ms 21,248 KB

Status Code is 200

DELETE Delete Request http://www.my-restapi.com / Delete Request 200 OK 640 ms 652 B

Status Code is 200

Iteration 2

GET Get Request ([url])index.htm / Get Request 200 OK 19 ms 21,248 KB

Status Code is 200

DELETE Delete Request http://www.my-restapi.com / Delete Request 200 OK 706 ms 652 B

Status Code is 200

Postman - Assertion

Assertions digunakan untuk memverifikasi apakah nilai aktual dan nilai yang diharapkan telah cocok setelah eksekusi tes. Jika tidak cocok, tes akan gagal dan kita akan mendapatkan alasan kegagalan dari keluaran tes.

Sebuah *assertion* mengembalikan nilai Boolean, yaitu *true* atau *false*. Di Postman, kita dapat menggunakan bantuan *JavaScript Chai Assertion Library* untuk menambahkan *assertion* ke dalam tes kita. Library ini tersedia secara otomatis di aplikasi Postman.

Chai Assertions mudah dipahami karena didefinisikan dalam format yang dapat dibaca manusia. *Assertions* di Postman ditulis di dalam tab **Tests** di bawah bilah alamat.

Dokumentasi untuk Chai tersedia di tautan berikut: <https://www.chaijs.com/>



Writing Assertions

Mari kita tulis sebuah *assertion* untuk memeriksa apakah sebuah teks tertentu – misalnya "Postman" – berada di dalam sebuah larik (*array*) string.

Contoh:

```
pm.test("Text is present", function(){
  pm.expect(['Java', 'Postman']).to.include('Postman')
})
```

Keluaran: Assertion lulus.



Mari kita tulis *assertion* untuk memeriksa apakah sebuah larik kosong.

Contoh:

```
pm.test("Array contains element", function(){
  pm.expect(['Java', 'Postman']).to.be.an('array').that.is.not.empty
})
```

Keluaran: Assertion lulus.



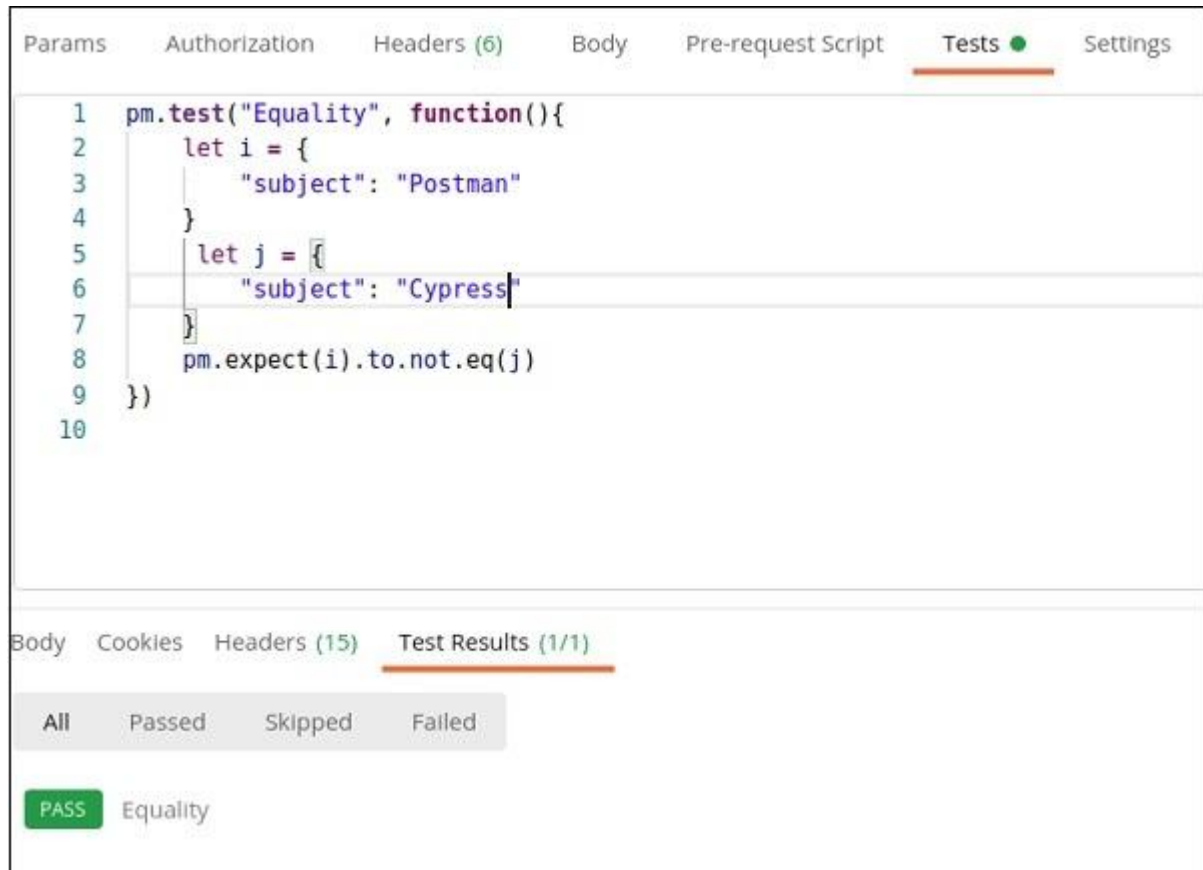
Assertion for Object Verification

Mari kita tulis *assertion* untuk verifikasi objek dengan eql. eql digunakan untuk membandingkan properti objek.

Contoh:

```
pm.test("Equality", function(){
  let i = { "subject" : "Postman" };
  let j = { "subject" : "Cypress" };
  pm.expect(i).to.not.eql(j);
})
```

Output



Keluaran: Assertion lulus karena properti objek i adalah "Postman" sedangkan properti objek j adalah "Cypress", sehingga `not.eql` menghasilkan *true*.

Jenis-Jenis Assertion

Di Postman, kita dapat menerapkan *assertion* pada berbagai bagian dari Response. Dijelaskan di bawah ini:

Kode Status (Status Code)

```
pm.test("Status Code is 401", function(){ pm.response.to.have.status(401) })
```

– Lulus jika kode status Response yang diperoleh adalah 401.

```
pm.test("Status is Forbidden", function(){ pm.response.to.have.property('status', 'Forbidden') })
```

– Diterapkan pada properti *status* Response dengan nilai "Forbidden".

Waktu yang Dibutuhkan Response

```
pm.test("Response time above 500 milliseconds", function () {  
    pm.expect(pm.response.responseTime).to.be.above(500)  
})
```

Lulus jika waktu Response di atas 500 ms.

Tipe Format Response

```
pm.test("Response type is JSON", function(){  
    pm.response.to.be.json;  
})
```

Lulus jika Response bertipe JSON.

Header Response

```
pm.test("Header Content-Encoding is available", function () {  
    pm.response.to.have.header("Content-Encoding")  
})
```

Lulus jika Response memiliki header *Content-Encoding*.

Teks Response

```
pm.test("Response Text", function () {  
    pm.expect(pm.response.text()).to.include("Tutorialspoint")  
})
```

Lulus jika teks Response mengandung teks "Tutorialspoint".

Postman - Mock Server

Mock server bukanlah server sungguhan; ia dibuat untuk menyimulasikan dan berfungsi sebagai server nyata guna memverifikasi API dan responsnya. Mock server umumnya digunakan jika respons tertentu perlu diverifikasi tetapi tidak tersedia di server web karena masalah keamanan pada server asli.

Tujuan Mock Server

- Dibuat jika API yang akan digunakan di *Production* masih dalam pengembangan.
- Digunakan jika kita ingin menghindari pengiriman permintaan pada data *real-time*.

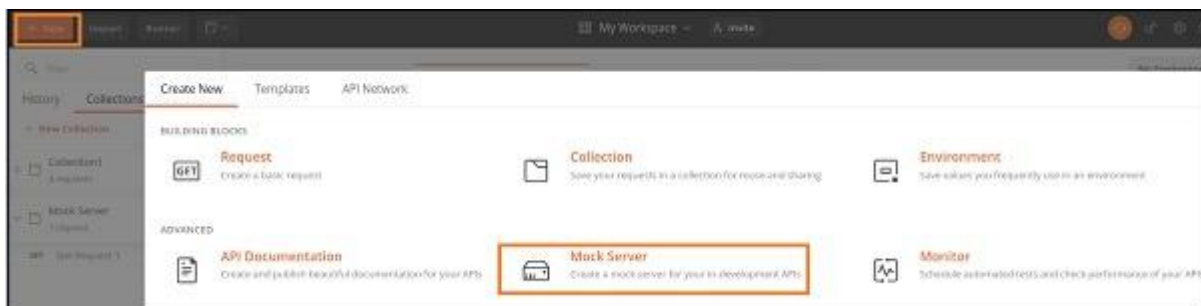
Manfaat Mock Server

- Simulasi fitur API nyata dengan contoh-contoh.
- Mock server dapat ditambahkan ke sebuah Koleksi.
- Memverifikasi API dengan data tiruan (*mocking data*).
- Mengidentifikasi kesalahan dan cacat sejak dini.
- Mengidentifikasi ketergantungan API sebelum dirilis untuk penggunaan sebenarnya.
- Digunakan oleh para insinyur untuk membuat prototipe konsep dan menunjukkannya kepada manajemen.
- Saat mengembangkan *front end* aplikasi, pengembang harus memiliki gambaran tentang fitur respons yang akan diperoleh dari server nyata. Mock server sangat membantu saat ini.

Pembuatan Mock Server

Ikuti langkah-langkah di bawah ini untuk membuat *mock server* di Postman:

Langkah 1 – Klik ikon **New** dari aplikasi Postman. Kemudian, klik **Mock Server**.



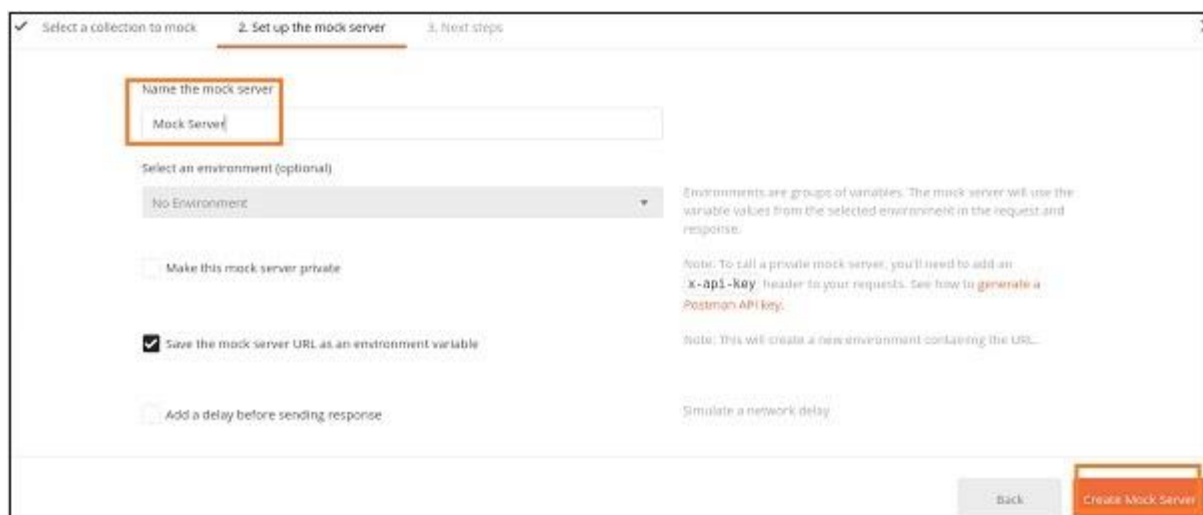
Langkah 2 – Pilih **GET** dari dropdown *Method*, masukkan *Request Path* sebagai `/user/home`, *Response Code* sebagai 200, dan sebuah *Response Body*. Kemudian, klik **Next**.



The screenshot shows the 'Set up the mock server' step in Postman. At the top, there are three tabs: '1. Select a collection to mock', '2. Set up the mock server', and '3. Next steps'. Below the tabs, there are two buttons: 'Create a new collection' and 'Select an existing collection'. A text block explains that mock servers simulate API endpoints without a backend and provides instructions on how to create an example. Below this is a table with four columns: 'Method', 'Request Path', 'Response Code', and 'Response Body'. The first row is highlighted with an orange border and contains 'GET', '({url})/user/home', '200', and 'This is Postman Tutor in Tutoraispoint'. The second row contains 'GET', '({url})/ Path', '200', and 'Response Body'. At the bottom right, there are 'Back' and 'Next' buttons, with 'Next' highlighted in orange.

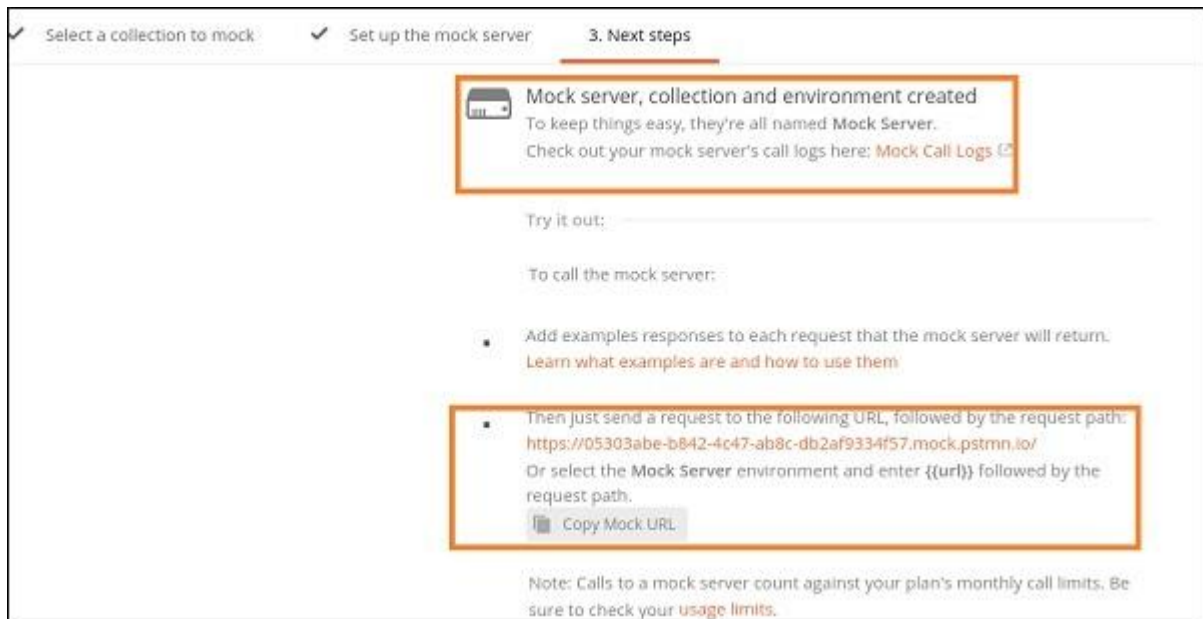
Method	Request Path	Response Code	Response Body
GET	({url})/user/home	200	This is Postman Tutor in Tutoraispoint
GET	({url})/ Path	200	Response Body

Langkah 3 – Masukkan nama *Mock Server* dan klik tombol **Create Mock Server**.

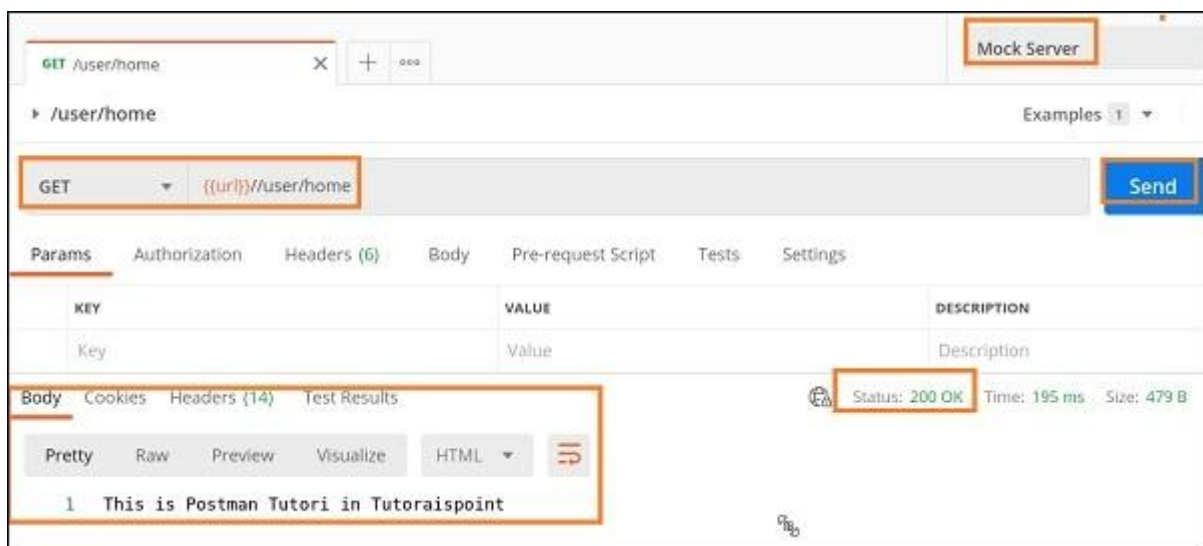


The screenshot shows the 'Set up the mock server' step in Postman. At the top, there are three tabs: '1. Select a collection to mock', '2. Set up the mock server', and '3. Next steps'. Below the tabs, there is a form with the following fields and options: 'Name the mock server' (with 'Mock Server' entered), 'Select an environment (optional)' (with 'No Environment' selected), 'Make this mock server private' (unchecked), 'Save the mock server URL as an environment variable' (checked), and 'Add a delay before sending response' (unchecked). To the right of the form, there is explanatory text about environments and a note about adding an 'x-api-key' header. At the bottom right, there are 'Back' and 'Create Mock Server' buttons, with 'Create Mock Server' highlighted in orange.

Langkah 4 – *Mock Server* akan dibuat beserta *Mock URL*. Tombol **Copy Mock URL** digunakan untuk menyalin tautan Mock. Klik tombol **Close** untuk melanjutkan.



Langkah 5 – Pilih **Mock Server** sebagai *Environment* dari dropdown *No Environment* dan klik **Send**. Kode respons yang diperoleh adalah **200 OK**, yang berarti permintaan berhasil. Response Body juga menampilkan pesan yang sama seperti yang kita berikan di Langkah 2.



Langkah 6 – Nilai URL dapat diperoleh dengan mengklik ikon mata di pojok kanan atas layar



URL

lengkap

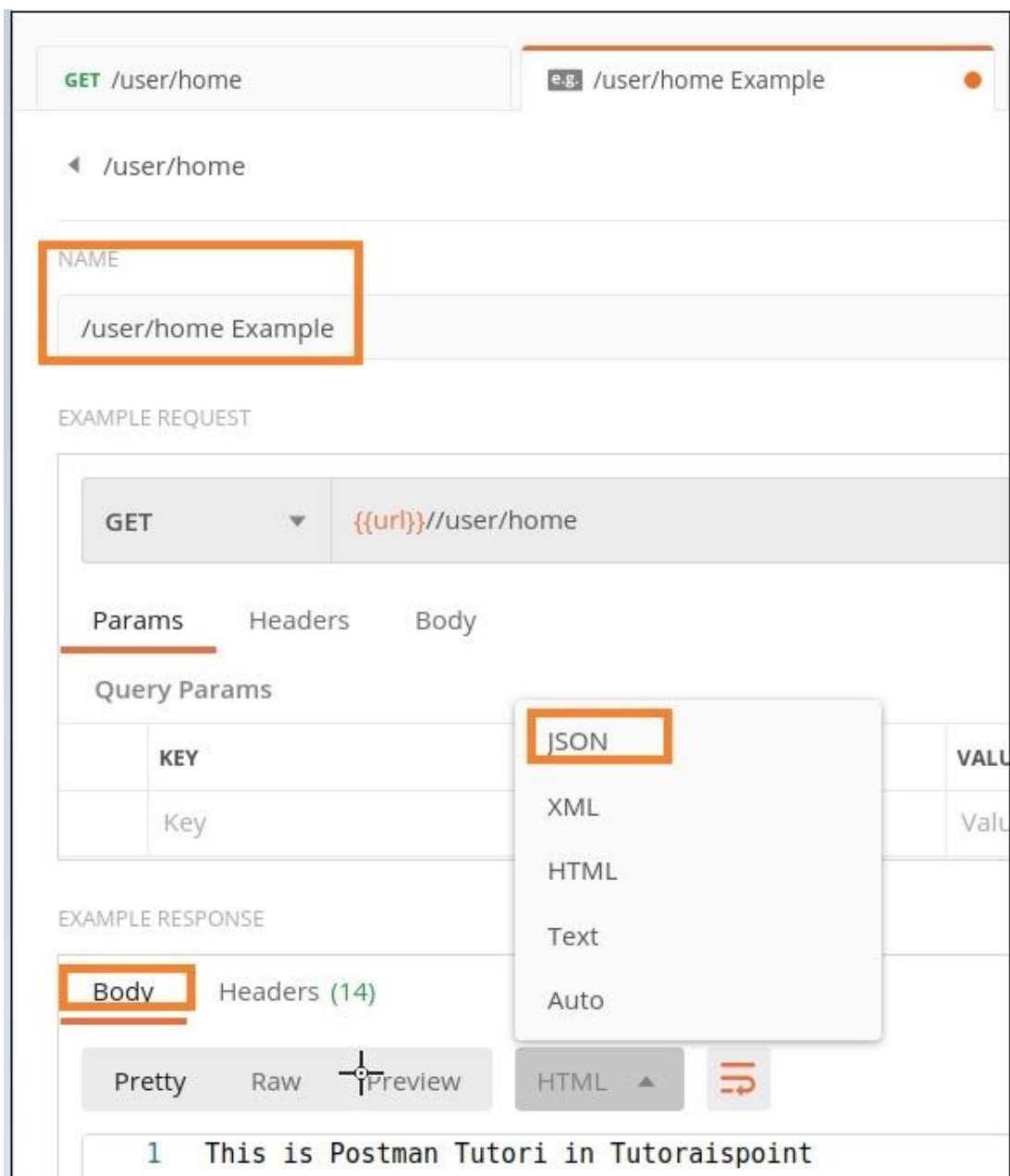
Mock

seharusnya: `https://<ID>.mock.pstmn.io/user/home` (direpresentasikan sebagai `{{url}}/user/home` di bilah alamat). Kita menambahkan `/user/home` di akhir nilai url karena itu adalah *Request Path* yang kita atur untuk Mock Server di Langkah 2.

Langkah 7 – Response Body dalam format teks. Kita dapat memperoleh respons dalam format JSON juga. Untuk mencapai ini, pilih opsi **Save as example** dari dropdown *Save Response*.

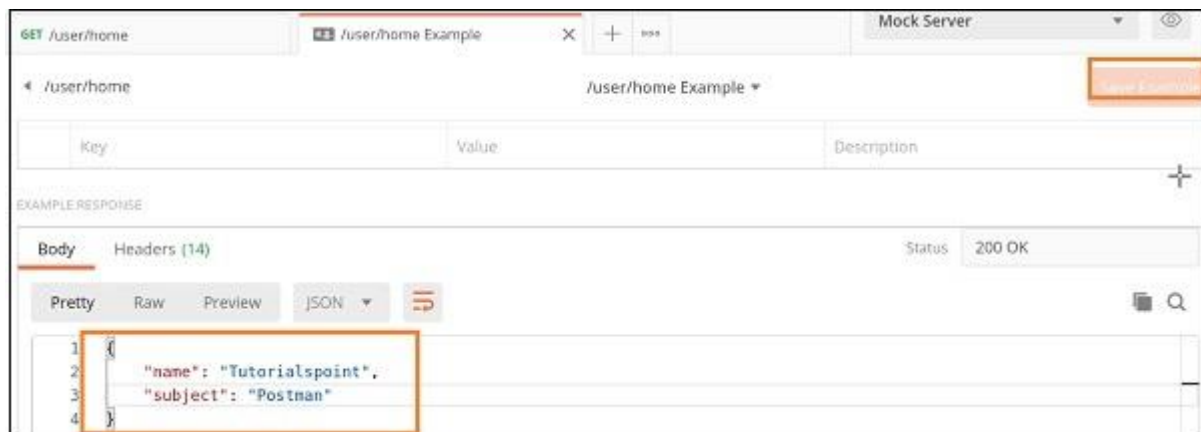


Langkah 8 – Berikan nama *Example* dan pilih **JSON** dari bagian *Response Body*.

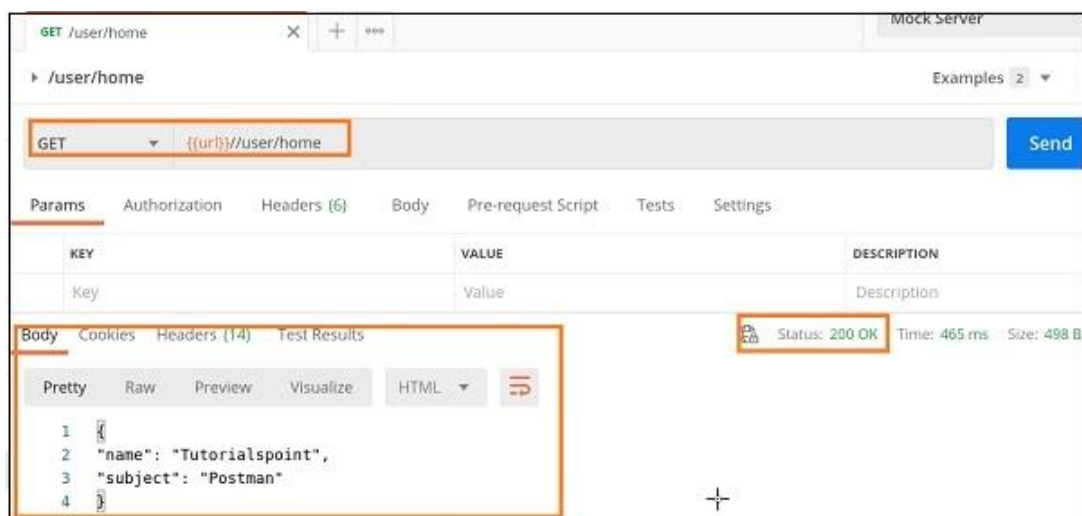


Langkah 9 – Tambahkan Response Body dalam format JSON berikut, lalu klik **Save Example**:

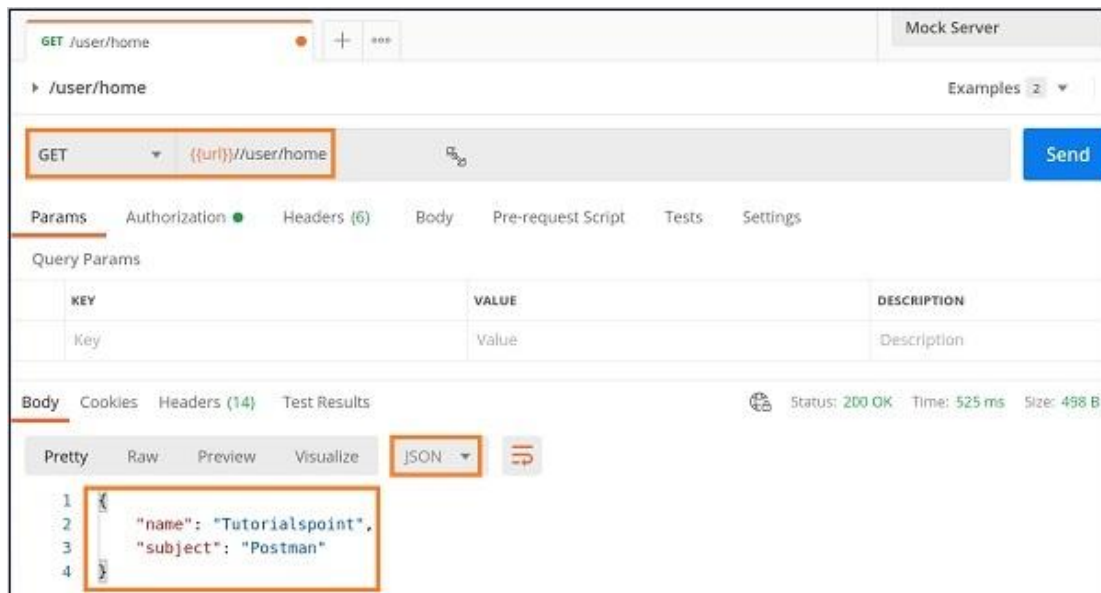
```
{
  "name": "Tutorialspoint",
  "subject": "Postman"
}
```



Langkah 10 – Kirim permintaan GET pada *endpoint* yang sama, dan kita akan menerima Response Body yang sama seperti yang kita berikan pada permintaan Contoh.

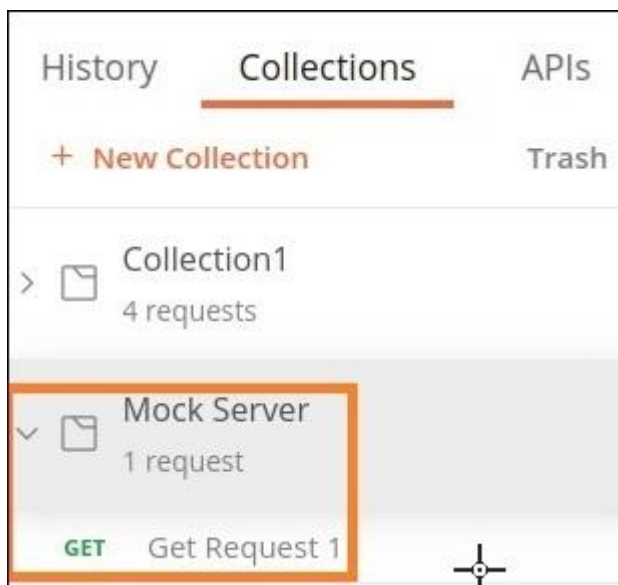


Gambar di bawah menunjukan json format

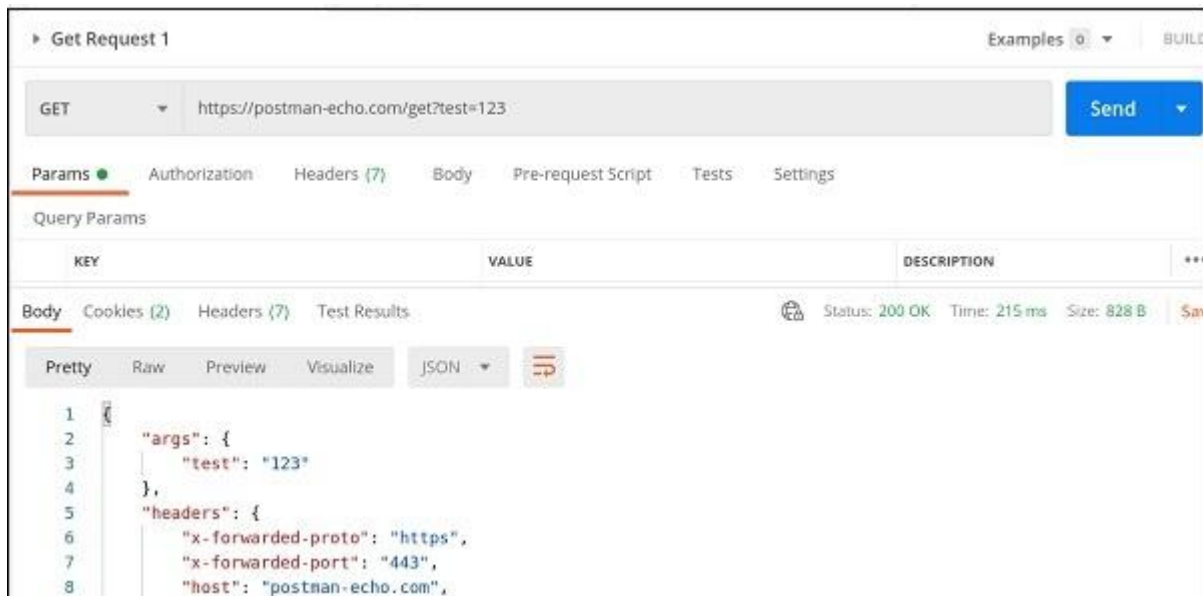


Pembuatan Mock Server Melalui Contoh Permintaan (Example Request)

Langkah 1 – Buat sebuah Koleksi dan tambahkan permintaan ke dalamnya.



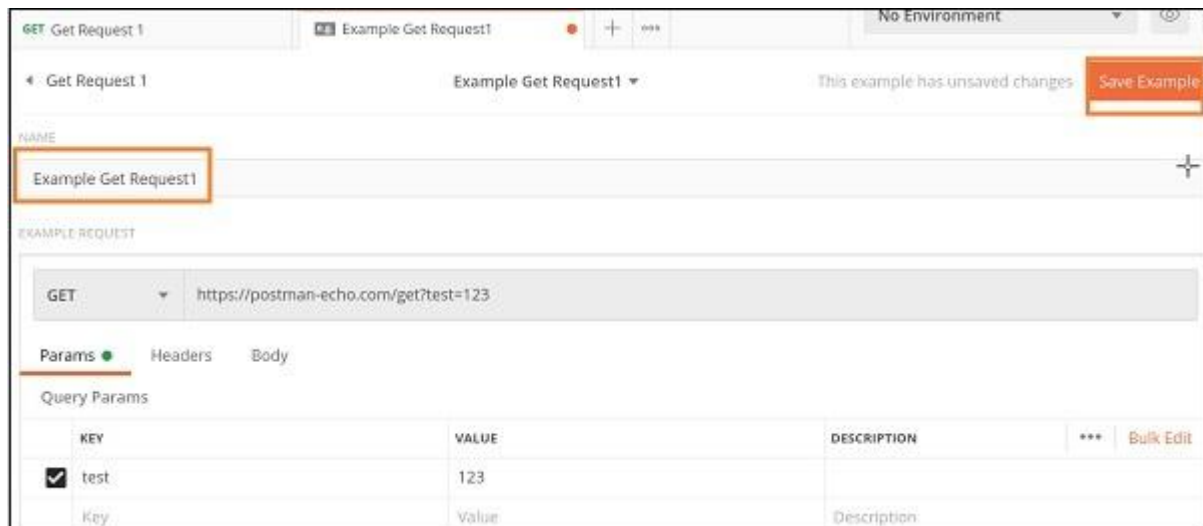
Langkah 2 – Tambahkan *endpoint* `https://postman-echo.com/get?test=123` dan kirim permintaan GET.



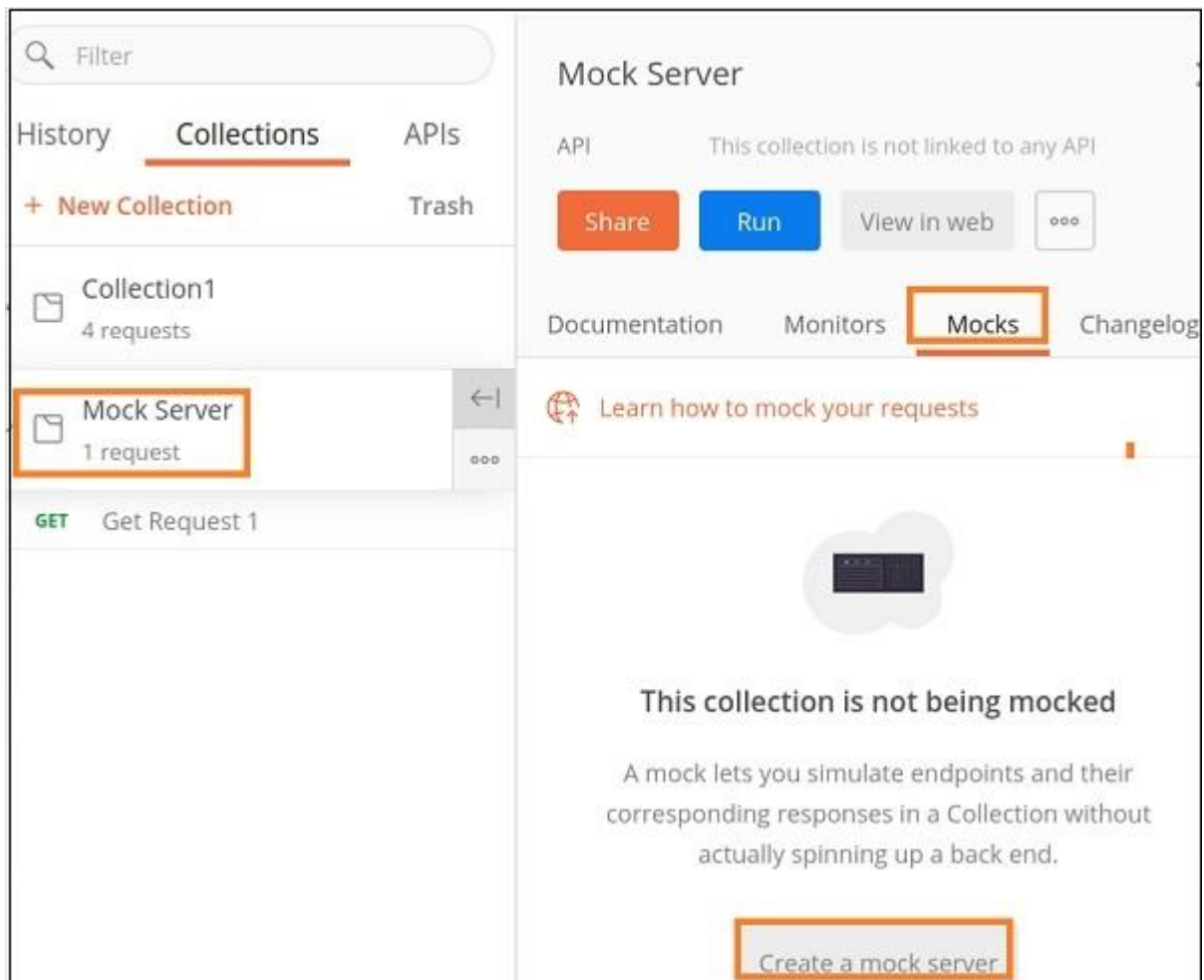
Langkah 3– Dari Response Body, pilih opsi **Save as Example** dari dropdown **Save Response**.



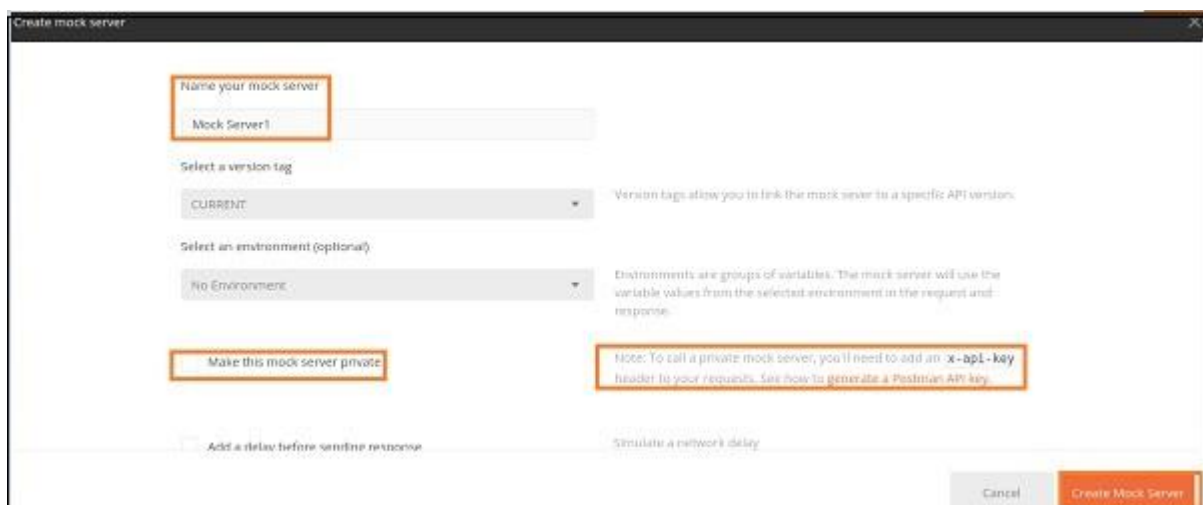
Langkah 4– Berikan nama *Example* dan klik tombol **Save Example**.



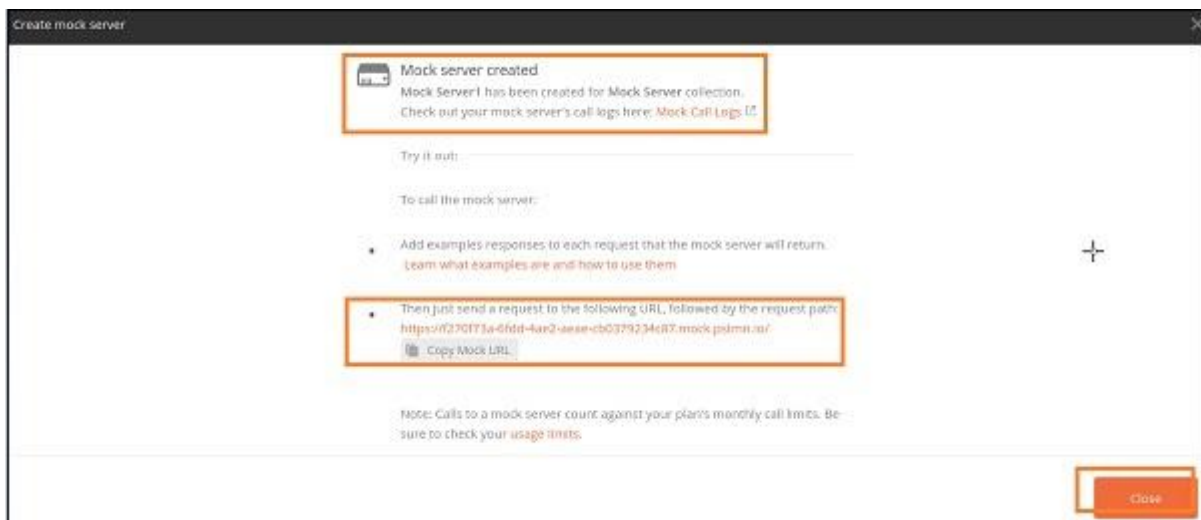
Langkah 5– Klik nama Koleksi (misalnya "Mock Server") dan klik tab **Mock**. Kemudian, klik **Create a mock server**.



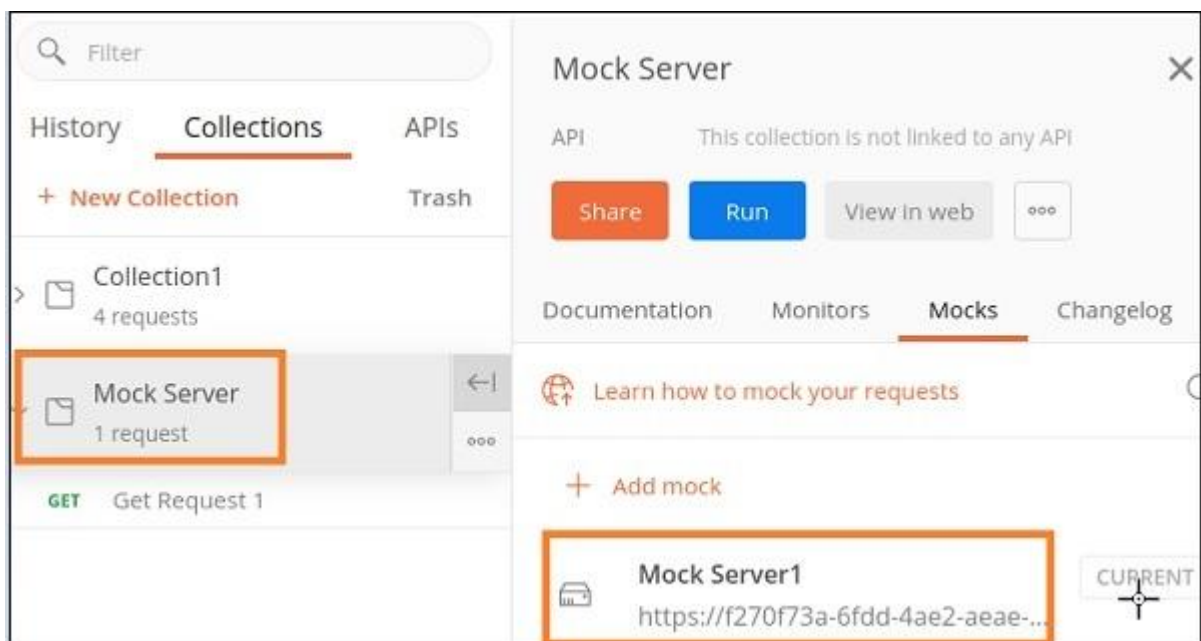
Langkah 6 – Munculan *Create mock server* terbuka. Berikan nama pada Mock Server lalu klik tombol **Create Mock Server**. Catatan: Kita dapat membuat Mock Server menjadi *private* atau *public*. Untuk membuatnya *private*, centang kotak "*Make this mock server private*". Maka kita perlu menggunakan *Postman API key*.



Langkah 7 – Pesan "*Mock server created*" akan muncul. Kita juga akan mendapatkan Mock URL. Kita dapat menyalinnya dengan tombol **Copy Mock URL**. Kemudian, klik **Close**.



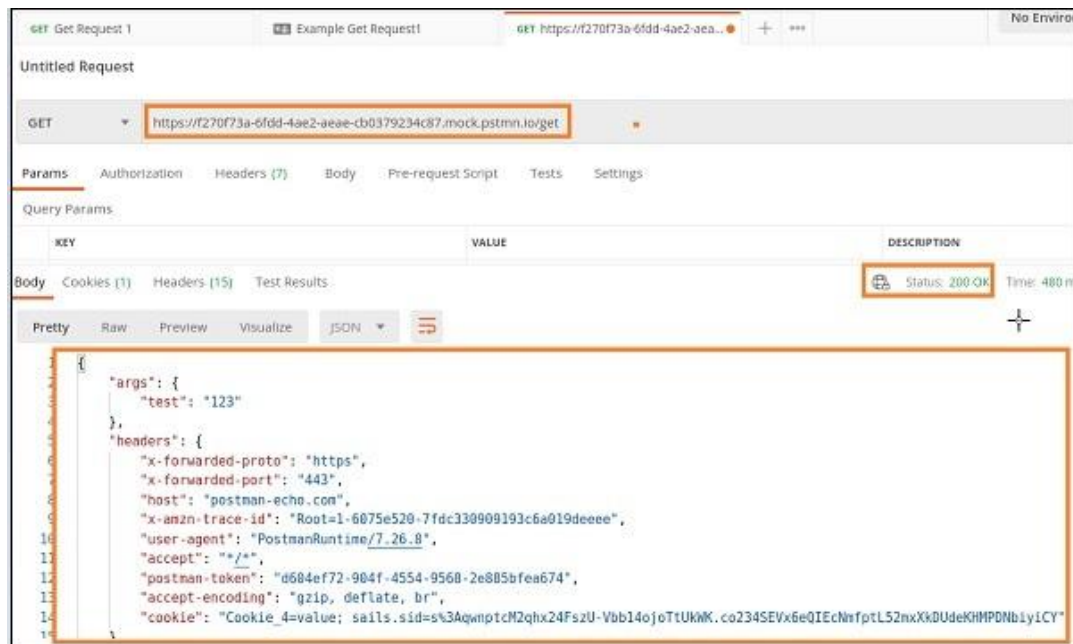
Langkah 8 – Mock Server yang telah dibuat akan muncul di bawah tab **Mock** di sidebar Collections. Klik di atasnya.



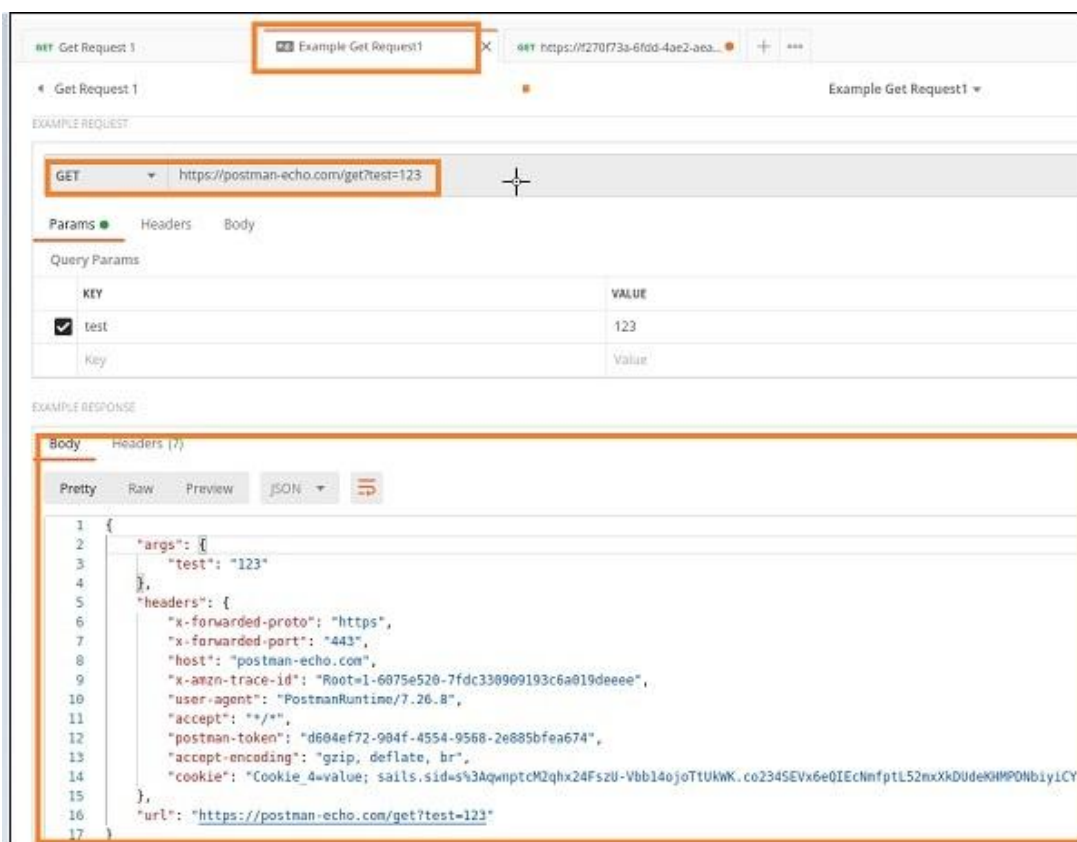
Langkah 9 – Kita akan menambahkan permintaan baru dan menempelkan URL yang telah disalin di Langkah 7. Untuk mengirim permintaan GET, kita tambahkan /get di akhir URL yang ditempel. Response Body yang diterima dari *mocking* server sama dengan permintaan Contoh.

Contoh Mock URL generate – <https://f270f73a-6fdd-4ae2-aeae-cb0379234c87.mock.pstmn.io>.

Sekarang coba kirim GET request, endpoint akan menjadi – <https://f270f73a-6fdd-4ae2-aaee-cb0379234c87.mock.pstmn.io/get>.



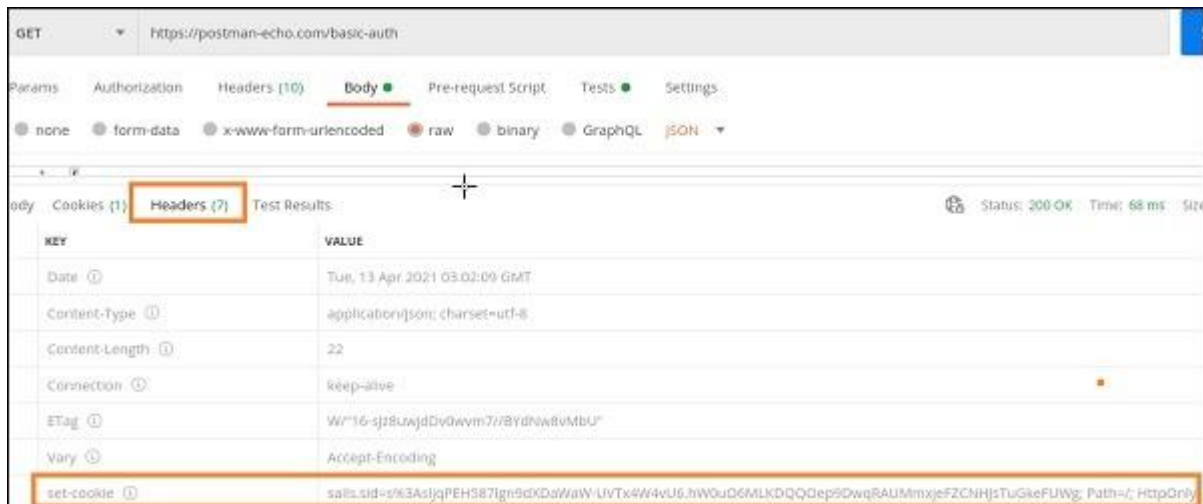
Response Body yang diterima dari *mocking server* sama dengan permintaan Contoh. Response yang diperoleh pada permintaan Contoh adalah sebagai berikut



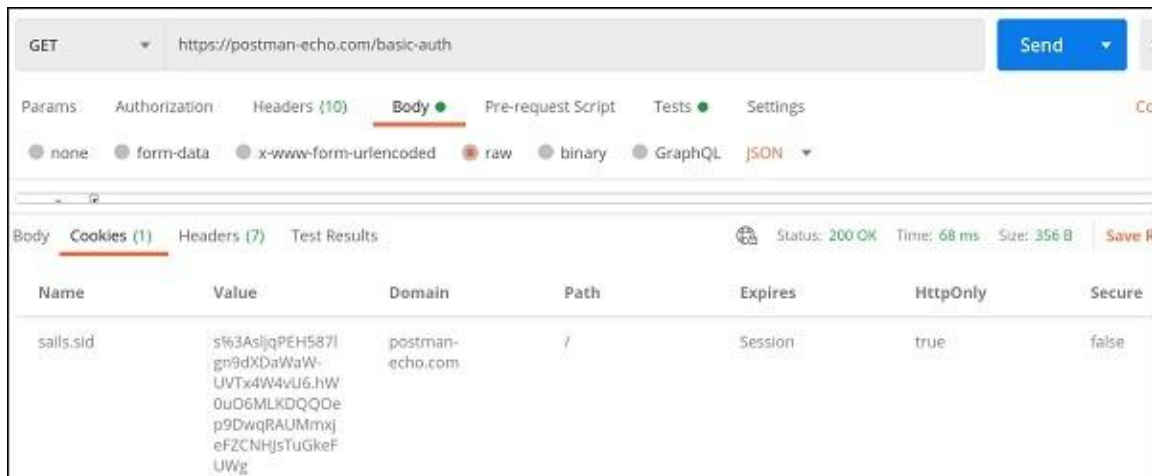
Postman - Cookies

Cookie adalah informasi yang dikirim oleh server dan disimpan di peramban (*browser*). Segera setelah permintaan dikirim, *cookie* dikembalikan oleh server. Di Postman, *cookie* disebutkan di bawah tab **Headers** dan **Cookies** di bagian Response.

Mari kita terapkan permintaan GET pada sebuah *endpoint* dan temukan *cookie*-nya



Di tab **Headers**, *cookie* yang dikirim oleh server diatur dengan kunci set-cookie. Di tab **Cookies**, detail *cookie* yang sama juga akan ditampilkan.

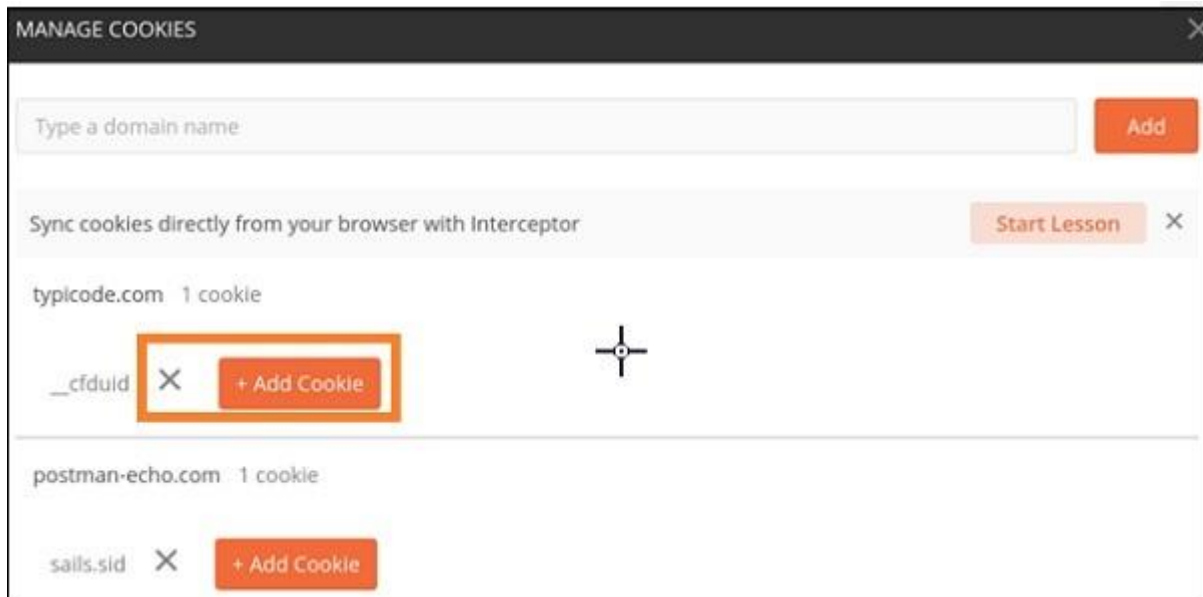


Cookies Management

Di Postman, kita dapat mengelola *cookie* dengan menambah, menghapus, dan memodifikasi *cookie*. Di bawah tab **Params**, terdapat tautan **Cookies** untuk melakukan operasi pada *cookie*.

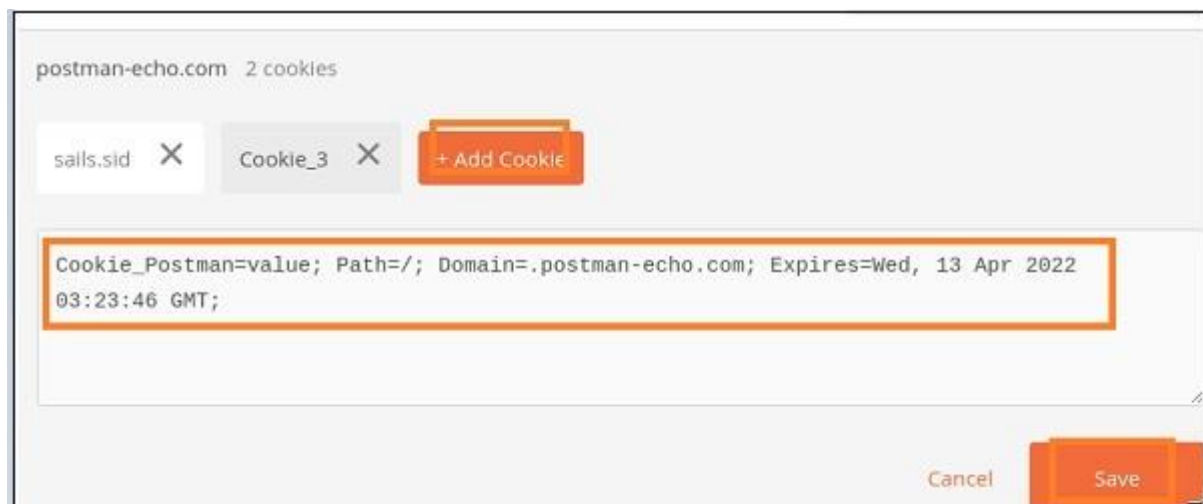


Klik tautan **Cookies**. Munculan *MANAGE COOKIES* akan terbuka di mana semua *cookie* yang tersedia ada beserta opsi untuk menambah dan menghapus *cookie*.



Penambahan Cookie

Langkah 1 – Klik tombol **Add Cookie**. Sebuah kotak teks akan terbuka dengan nilai yang sudah ada di dalamnya. Kita dapat memodifikasi nilainya lalu klik **Save**.



Langkah 2 – Kirim kembali permintaan ke server. Kode respons yang diperoleh adalah **200 OK**. Juga, tab **Cookies** di Response sekarang menampilkan *cookie* yang baru ditambahkan, misalnya *Cookie_Postman*.

KEY	VALUE	DESCRIPTION	***
Cookie_Postman	value	postman-echo.com	/
sails.sid	s%3A_GDZNBL7UHTib9z_7B54w36QTVnu2YGk_A48l1Zqczy%2Bqb%2BxML%2B8QI4LIXGUi3nll00-%2F0l	postman-echo.com	/

Mengakses Cookie Secara Programatis

Cookie dapat ditangani secara terprogram tanpa menggunakan antarmuka grafis di Postman. Untuk bekerja dengan cookie, pertama-tama kita harus menghasilkan *Cookie jar*. *Cookie jar* adalah objek yang memiliki semua cookie dan metode untuk mengaksesnya.

Pembuatan Cookie Jar

```
const c = pm.cookies.jar();
```

Pembuatan Cookie

Fungsi `.set()` menerima URL, nama cookie, nilai cookie sebagai parameter.

```
c.set(URL, name, value, callback(error, cookie));
```

Mendapatkan Cookie

Fungsi `.get()` menerima URL dan nama cookie, mengembalikan nilai cookie.

```
c.get(URL, name, callback(error, cookie));
```

Mendapatkan Semua Cookie

Fungsi `.getAll()` menerima URL, mengembalikan semua nilai cookie untuk URL tersebut.

```
c.getAll(URL, callback(error, cookies));
```

Menghapus Cookie

Fungsi `.unset()` menerima URL dan nama cookie yang akan dihapus.

```
c.unset(URL, name, callback(error, cookie));
```

Menghapus Semua Cookie

Fungsi `.clear()` menerima URL, menghapus semua nilai cookie untuk URL tersebut.

```
c.clear(URL, callback(error));
```

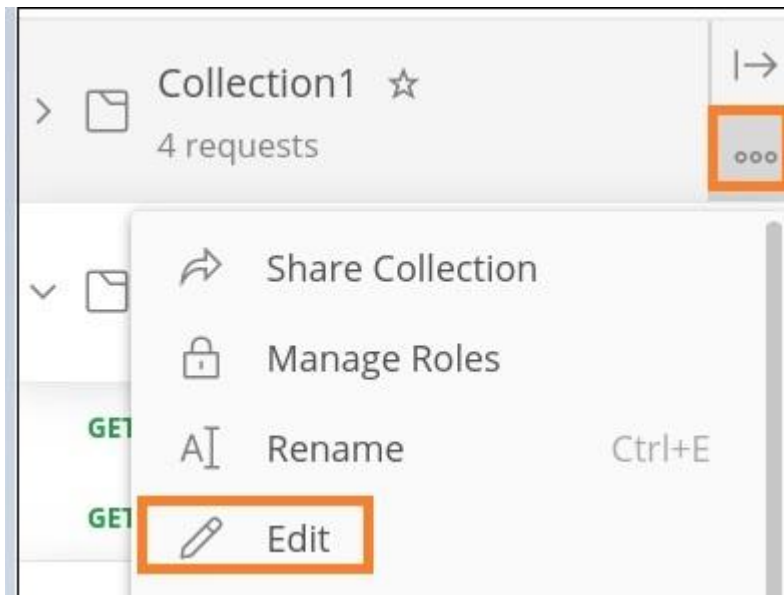
Postman - Sessions

Session adalah lipatan sementara yang menyimpan nilai-nilai variabel. Session digunakan untuk instance saat ini dan memiliki cakupan lokal. Di Postman, kita dapat memodifikasi nilai variabel session untuk berbagi ruang kerja antar tim.

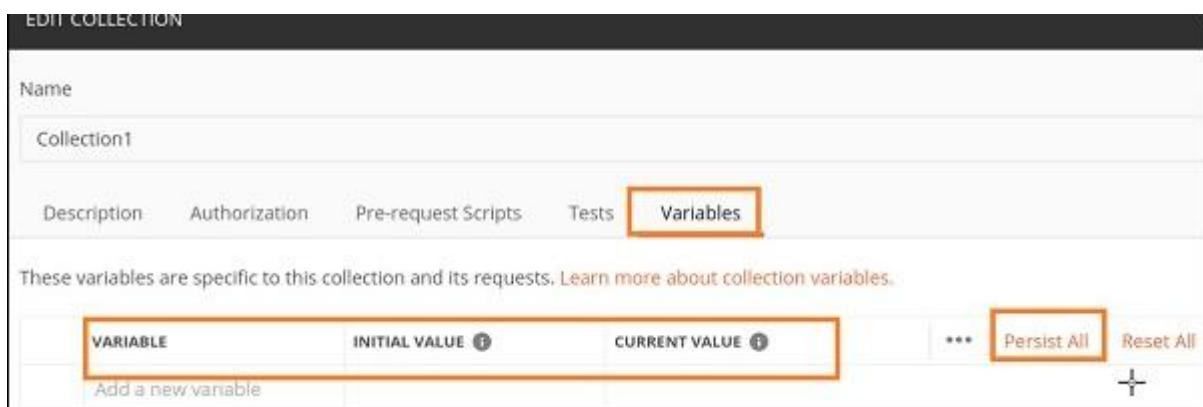
Postman memberikan fitur berbagi session lokal. Meskipun sebuah Koleksi dapat dibagikan antar tim, session tidak pernah dibagikan. Token yang berbeda harus dihasilkan saat suatu tugas akan dilakukan dalam struktur tim.

Session memiliki cakupan lokal untuk seorang pengguna di dalam *Workspace*-nya, dan modifikasi apa pun yang ia lakukan tidak akan tercermin di server. Di Postman, session dapat menyimpan variabel Environment, variabel global, dan sebagainya.

Kita dapat menetapkan nilai saat ini (*current value*) ke variabel Koleksi, variabel global, dan variabel Environment. Untuk menetapkan nilai saat ini ke Koleksi, klik tiga titik di samping nama Koleksi, lalu klik **Edit**.



Di pop-up **EDIT COLLECTION**, pindah ke tab **Variables**.



CURRENT VALUE bersifat lokal untuk pengguna dan tidak pernah sinkron dengan server Postman. Kita juga dapat mengganti atau memodifikasi **INITIAL VALUE** dengan **CURRENT VALUE**.

Juga, perlu diingat bahwa **INITIAL VALUE** hanya terpengaruh jika kita menerapkan opsi **Persist** pada sebuah variabel. Setelah itu, variabel tersebut akan sinkron dengan server Postman.

Postman - Newman Overview

Newman adalah *command-line runner* potensial yang digunakan di Postman. Kita dapat mengeksekusi dan memverifikasi sebuah Koleksi Postman dari baris perintah juga. Newman memiliki fitur yang konsisten dengan Postman.

Kita dapat menjalankan permintaan di dalam sebuah Koleksi dari Newman dengan cara yang sama seperti di Collection Runner. Newman dapat berada di kedua *registry* NPM dan GitHub. Kita juga dapat melakukan *Continuous Integration* atau *Deployment* dengan Newman.

Kode status **0** dikembalikan oleh Newman jika semua eksekusi selesai tanpa kesalahan. Alat *Continuous Integration* membaca kode status dan menentukan gagal/berhasilnya sebuah *build*.

Kita dapat menambahkan flag `--bail` pada Newman untuk berhenti pada kesalahan yang ditemui dalam sebuah tes dengan kode status **1**. Ini dapat diinterpretasikan oleh alat CI. Newman berbasis *node.js* dan menggunakan *npm* sebagai manajer paket.

Instalasi Newman

Instalasi Newman memerlukan Node.js dan npm.

Langkah 1: Navigasi ke tautan <https://nodejs.org/en/download/current/> untuk mengunduh Node.js.

The screenshot shows the Node.js download page. The URL is <https://nodejs.org/en/download/current/>. The page features the Node.js logo and a navigation bar with links: HOME, ABOUT, DOWNLOADS, DOCS, GET INVOLVED, SECURITY, CERTIFICATION, and NEWS. The main content area is titled 'Downloads' and indicates the latest current version is 15.14.0 (including npm 7.7.6). It offers two main options: LTS (Recommended For Most Users) and Current (Latest Features). Under the Current section, there are three download options: Windows Installer (node-v15.14.0.msi), macOS Installer (node-v15.14.0.pkg), and Source Code (node-v15.14.0.tar.gz). Below these, there is a table listing various binary downloads for different operating systems and architectures.

32-bit	64-bit
32-bit	64-bit
	64-bit
	64-bit
	64-bit
ARMv7	ARMv8
node-v15.14.0.tar.gz	

Langkah 2: Setelah unduhan selesai, jalankan perintah berikut untuk memverifikasi bahwa Node.js terinstal dengan benar.

- Windows: `node --v`
- Linux: `node --version`

```
osboxes@osboxes:~/Desktop$ node --version
v10.15.2
osboxes@osboxes:~/Desktop$
```

Langkah 3: npm disertakan dengan Node.js, jadi setelah mengunduh Node.js, npm akan terunduh secara default. Verifikasi ketersediaan npm:

- Windows: `npm --v`
- Linux: `npm --version`

```
osboxes@osboxes:~/Desktop$ node --version
v10.15.2
osboxes@osboxes:~/Desktop$ npm --version
5.8.0
osboxes@osboxes:~/Desktop$
```

Langkah 4: Untuk instalasi Newman, jalankan perintah: `npm install -g newman`

Langkah 5: Verifikasi versi Newman:

- Windows: `newman --v`

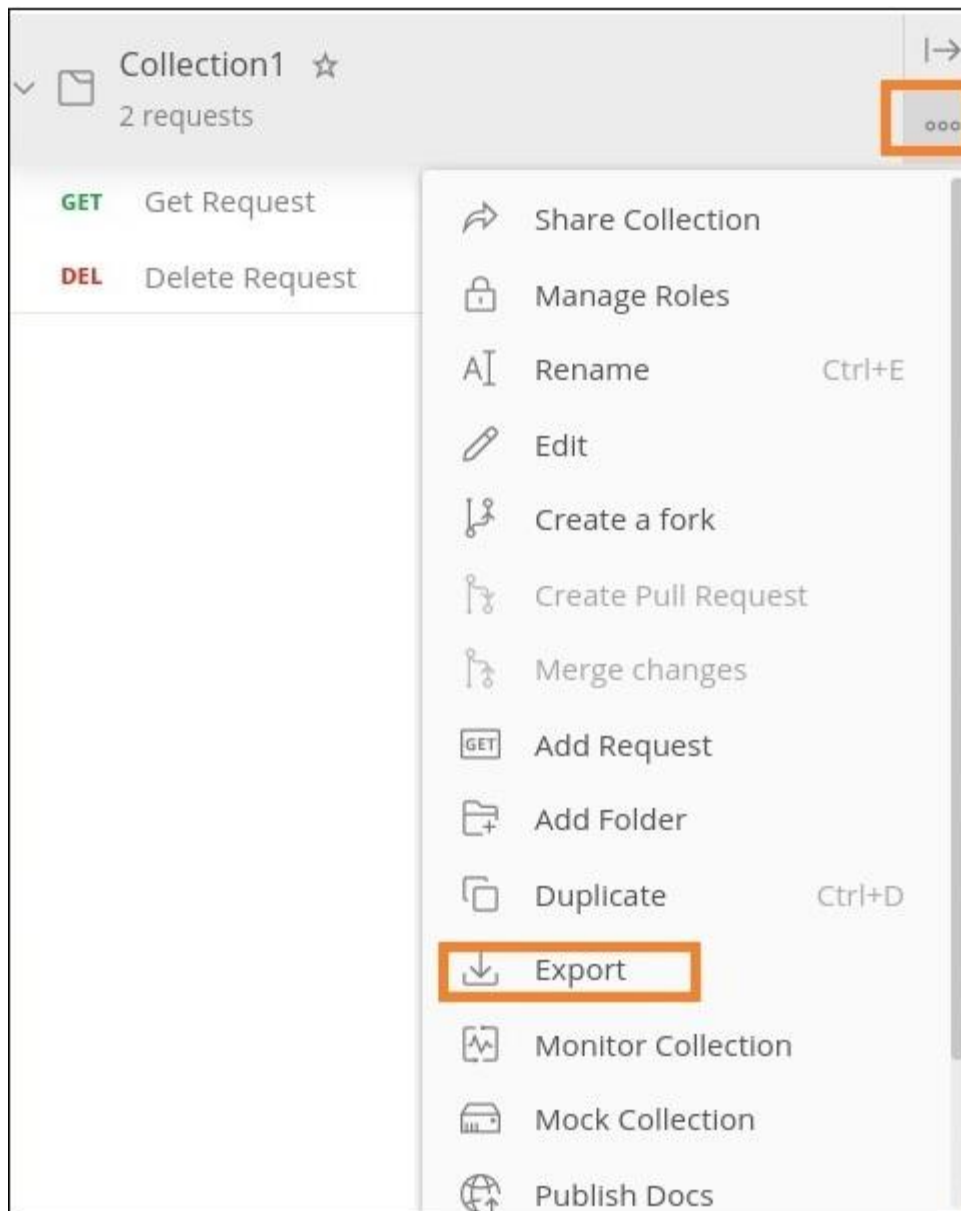
Linux: `newman --version`

Postman - Run Collections using Newman

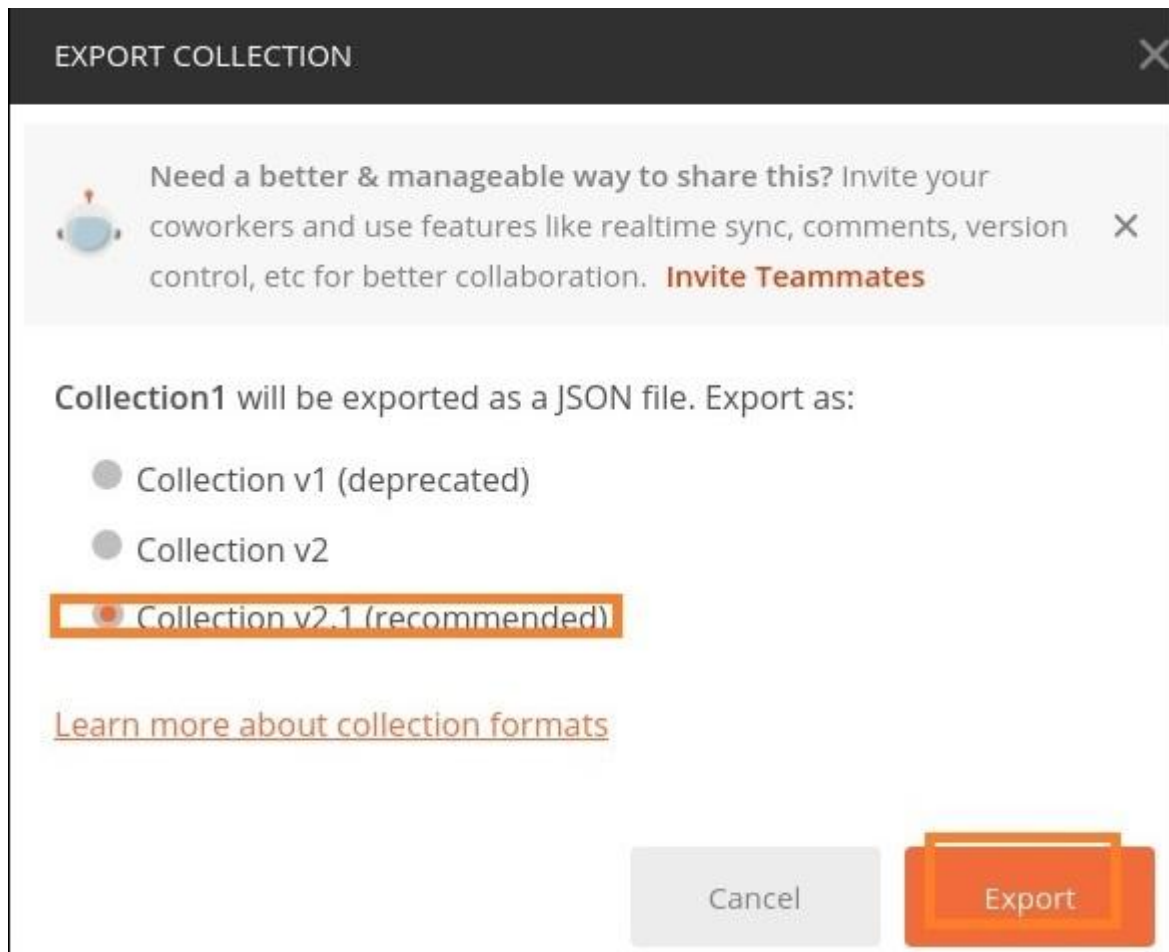
Untuk menjalankan Koleksi menggunakan Newman, pertama-tama kita harus meluncurkan aplikasi Postman dan klik tiga titik yang tersedia di samping nama Koleksi. Detail tentang cara membuat Koleksi dibahas dalam bab *Postman Create Collections*.

Menjalankan Koleksi

Langkah 1 – Klik **Export**.

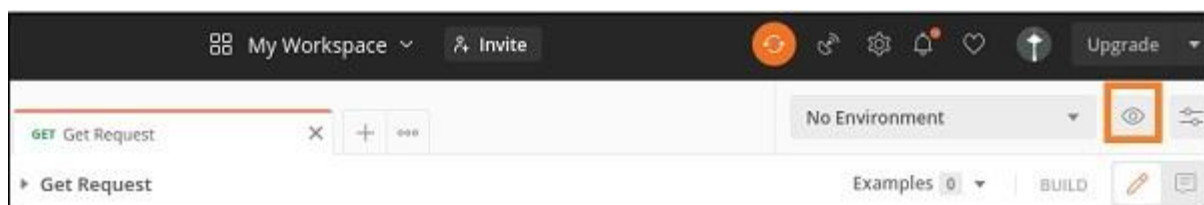


Langkah 2 – Pilih opsi **Collection v2.1 (recommended)** dari pop-up **EXPORT COLLECTION**. Klik **Export**.

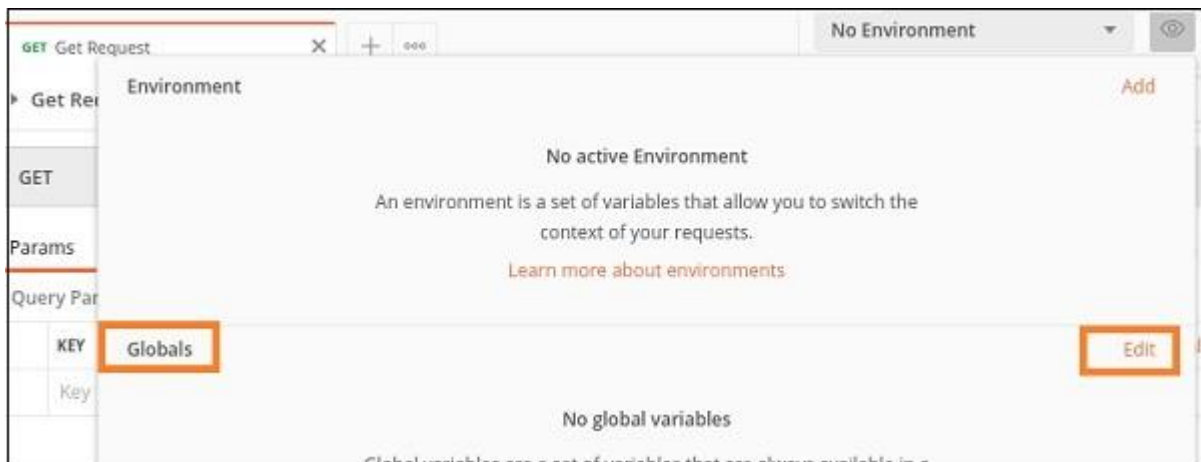


Langkah 3 – Pilih lokasi dan klik **Save**.

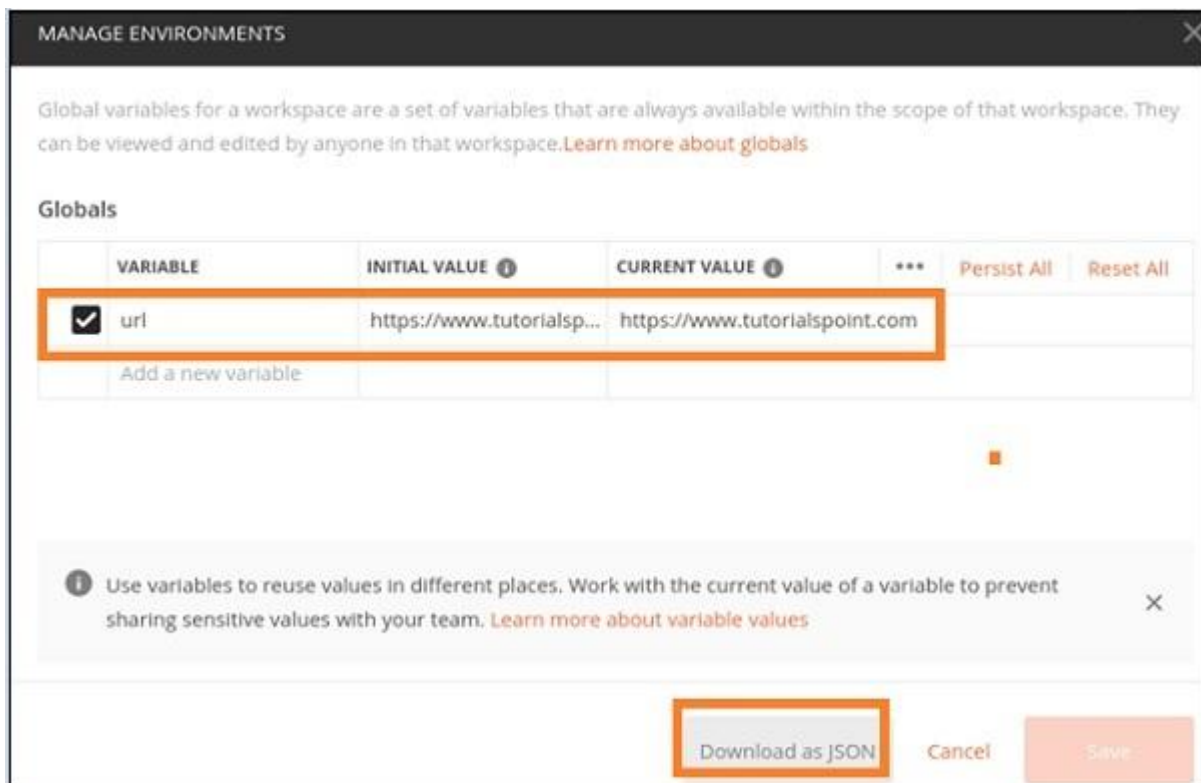
Langkah 4 – Selanjutnya, kita akan mengekspor *Environment*. Klik ikon mata di sebelah kanan dropdown *No Environment*.



Langkah 5 – Klik tautan **Edit** di bagian **Globals**.



Langkah 6 – Munculan **MANAGE ENVIRONMENTS** terbuka. Masukkan URL untuk kolom **VARIABLE** dan **https://www.tutorialspoint.com** untuk **INITIAL VALUE**. Kemudian, klik **Download as JSON**. Terakhir, pilih lokasi yang diinginkan dan klik **Save**.



Langkah 7 – Ekspor *Environment* ke lokasi yang sama dengan tempat Koleksi berada.

Langkah 8 – Dari baris perintah, pindah ke direktori tempat Koleksi dan Environment disimpan. Kemudian, jalankan perintah berikut:

```
newman run "<nama file>"
```

Nama file harus selalu diapit tanda kutip; jika tidak, nama tersebut akan dianggap sebagai nama direktori.

Argumen Baris Perintah Umum untuk Newman

- Menjalankan Koleksi di sebuah Environment:

```
newman run <nama Koleksi> -e <nama Environment>
```

- Menjalankan Koleksi untuk sejumlah iterasi:

```
newman run <nama Koleksi> -n <jumlah iterasi>
```

- Menjalankan Koleksi dengan file data:

```
newman run <nama Koleksi> --data <nama file> -n <jumlah iterasi> -e <nama Environment>
```

- Mengatur waktu tunda di antara permintaan:

```
newman run <nama Koleksi> -d <waktu tunda>
```